SERVIÇO SOCIAL DA INDÚSTRIA - SESI
SESI-SP CENTRO DE ATIVIDADES ESCOLARES
EQUIPE FTC SESI BUG HUNTERS

ANDRÉ LUIZ DE ANDRADE FLOR

MEMORIAL TÉCNICO PEDAGÓGICO E MATRIZ DE APRENDIZAGEM: ODOMETRIA HOLONÔMICA, ALGORITMOS DE NAVEGAÇÃO E TRILHA GAMIFICADA DE PROGRAMAÇÃO NO FTC ODOMETRY SANDBOX

SÃO BERNARDO DO CAMPO - SP
2026

ANDRÉ LUIZ DE ANDRADE FLOR

MEMORIAL TÉCNICO PEDAGÓGICO E MATRIZ DE APRENDIZAGEM: ODOMETRIA HOLONÔMICA, ALGORITMOS DE NAVEGAÇÃO E TRILHA GAMIFICADA DE PROGRAMAÇÃO NO FTC ODOMETRY SANDBOX

Memorial técnico de engenharia de software e diretrizes pedagógicas, elaborado para compor o Caderno de Engenharia (Engineering Notebook) da equipe SESI Bug Hunters, alinhado aos eixos S.T.E.A.M. Versão do software documentada: FTC Odometry Sandbox v26.0 (fase Alpha).

NOTA DE ESCLARECIMENTO: O desenvolvimento do presente software começou no período de off-season da temporada FTC DECODE, sem nenhuma relação ou vínculo com as regras, dinâmicas ou desafios daquela temporada, e continua durante a temporada 2026-2027, BIOBUZZ. O núcleo do software trata de competências universais de navegação; a partir da versão 25, ele também inclui uma representação do campo e das peças do BIOBUZZ, feita pela equipe a partir do Manual de Competição publicado, que serve para treino e não substitui as regras oficiais.

SÃO BERNARDO DO CAMPO - SP
2026

SUMÁRIO

1. INTRODUÇÃO E CONTEXTUALIZAÇÃO PEDAGÓGICA.....6
2. MATRIZ DE COMPETÊNCIAS CIENTÍFICAS E MATEMÁTICAS (S.T.E.A.M.)….7
3. ARQUITETURA DE LOCALIZAÇÃO CINEMÁTICA (DEAD WHEELS + IMU).....8
3.1 SISTEMAS DE COORDENADAS DO SIMULADOR.....8
3.2 COMPENSAÇÃO DO ARCO ROTACIONAL E MATRIZ DE ROTAÇÃO.....9
3.3 MODO DE TRÊS RODAS: HEADING CALCULADO PELOS ENCODERS. .10
3.4 LEITURAS EM TICKS E CALIBRAÇÃO DO DIÂMETRO DA RODA.....11
3.5 PINPOINT ODOMETRY COMPUTER SIMULADO.....11
4. MODOS DE MOVIMENTO E CINEMÁTICA DO CHASSI.....11
4.1 MOVIMENTO IDEAL.....13
4.2 TRAÇÃO DIFERENCIAL (TANK).....13
4.3 TRAÇÃO MECANUM.....13
4.4 TRAÇÃO SWERVE.....14
4.5 INÉRCIA DOS MOTORES E ENCODERS DE TRAÇÃO.....14
5. PROGRAMAÇÃO DO ROBÔ: COMANDOS, MOTORES E GAMEPAD.....15
5.1 INTERFACE DE COMANDOS AUTÔNOMOS.....15
5.2 CONTROLE DIRETO DOS MOTORES E MODELOS DE TELEOP.....15
5.3 SUPORTE A GAMEPAD.....16
6. ALGORITMOS DE NAVEGAÇÃO HOLONÔMICA.....17
6.1 LÓGICA DE APONTAMENTO E ANGLE WRAPPING.....17
6.2 NAVEGAÇÃO PONTO A PONTO (GO TO TARGET).....18
7. METROLOGIA, TELEMETRIA E DIAGNÓSTICO EM AMBIENTE VIRTUAL.....19
8. RESULTADOS: MEDIÇÕES NO SIMULADOR.....21
8.1 ETAPAS DO ALGORITMO DE ODOMETRIA.....21
8.2 DEAD WHEELS E ENCODERS DE TRAÇÃO NOS DIFERENTES CHASSIS 22
9. TELA INICIAL E SISTEMA DE PROGRESSO.....22
9.1 ORGANIZAÇÃO EM TELAS E NAVEGAÇÃO.....23
9.2 PROGRESSO, NÍVEIS E MEDALHAS.....24
10. MAPA DE ESTUDOS INTERATIVO.....24
10.1 FASES, ÁREAS E MAPA.....25
10.2 ESTRUTURA DE UMA LIÇÃO.....26
10.3 DESAFIOS E EXPLORAÇÕES INTERATIVAS.....27
10.4 REGRAS DE PROGRESSÃO.....28
10.5 VALIDAÇÃO AUTOMATIZADA.....28
11. CAMINHO DO ANDRÉ: TELEOP, AUTÔNOMO E PEDRO PATHING.....29

11.1 CAPÍTULOS E ESTRUTURA DOS PASSOS....29
11.2 CHECAGEM DE VOCABULÁRIO....31
11.3 TELEOP E AUTÔNOMO NO SANDBOX....32
11.4 O SEGUIDOR GVF DE SEGUNDA ORDEM....33
11.5 RESULTADOS DO SEGUIDOR NO SIMULADOR....35
11.6 DO SANDBOX À API DO PEDRO PATHING....36
12. DESAFIOS DE CÓDIGO PARA AVANÇAR....37
12.1 COMO É UM DESAFIO DE CÓDIGO....37
12.2 CORREÇÃO AUTOMÁTICA E ISOLAMENTO....39
12.3 CORREÇÃO DO CÓDIGO INICIAL DO SWERVE....40
12.4 BOTÕES DO CONTROLE: ATALHOS, BORDAS E VIBRAÇÃO....41
12.5 EDITOR, CONSOLE E MENSAGENS DE ERRO DOS DESAFIOS....42
12.6 TESTE DE MESA E DEPURADOR....46
12.7 CORES DO ANDROID STUDIO E MENU DE OPÇÕES....50
12.8 O CÓDIGO ORGANIZADO EM CLASSES E PASTAS....52
12.9 FUNÇÕES DE CADA DESAFIO....55
12.10 DICAS EM ESCADA....56
13. ROBÔ COMPLETO E DRIVER HUB....58
13.1 DO PROJETO JAVA AO OPMODE NO SIMULADOR....58
13.2 O FTC SDK SIMULADO....59
13.3 VISÃO COMPUTACIONAL COM APRILTAGS....60
13.4 O DRIVER HUB: CONFIGURAÇÃO, INIT E START....61
13.5 MISSÕES E VALIDAÇÃO....62
13.6 O SANDBOX EM DOIS MODOS....63
13.7 SUGESTÕES NO EDITOR DE CÓDIGO....64
13.8 SALVAR, ABRIR E IMPORTAR PROJETOS....66
13.9 SWERVE NO DESENVOLVIMENTO COMPLETO....68
14. SENSORES DE LOCALIZAÇÃO: PINPOINT V2, PODS E LIMELIGHT 3A....70
14.1 MONTAGEM DO ROBÔ E CONFIGURAÇÃO....70
14.2 GOBILDA PINPOINT V2....71
14.3 PODS DE ODOMETRIA INDIVIDUAIS....72
14.4 LIMELIGHT 3A: APRILTAGS, MEGATAG1 E MEGATAG2....72
14.5 VALIDAÇÃO....73
15. VISÃO EM 3D, CAMPO BIOBUZZ E MODO PONTUAL DE VISÃO....74
15.1 O CAMPO BIOBUZZ E AS APRILTAGS DAS HIVES....74
15.2 O MOTOR 3D: A RETA DE BRESENHAM E O ROTATEPOINT....75
15.3 A VISTA 3D: ARENA, CÂMERAS E ROBÔ ESTIMADO....76
15.4 A INCLINAÇÃO DAS CÂMERAS E O QUE ELAS VEEM....78

15.5 MODO PONTUAL DE VISÃO: LIMELIGHT OU C270.....79
15.6 VALIDAÇÃO.....80
16. PEÇAS DO JOGO, MECANISMOS DO ROBÔ E FÍSICA.....81
16.1 AS HIVES: CELLS EM FORMA DE CASINHA NUMA BARRA RETA.....81
16.2 POLLEN, NECTAR, FLOWERS, GARDENS E PARK.....83
16.3 O ROBÔ: INTAKE, MIDTAKE, SHOOTER E BRAÇO.....84
16.4 A FÍSICA DAS PEÇAS, DO LANÇAMENTO E DAS HIVES.....89
16.5 CÂMERAS EM SEIS GRAUS DE LIBERDADE.....92
16.6 VALIDAÇÃO.....94
17. PARTIDA COM ROBÔS, BATERIA E CALIBRAÇÃO.....96
17.1 OS TRÊS CHASSIS NA VISTA 3D.....96
17.2 A PARTIDA: ALIANÇA, RELÓGIO E AUTÔNOMO DOS OUTROS ROBÔS 97
17.3 COMO OS OUTROS ROBÔS JOGAM.....98
17.4 A REGRA DE VIRADA DA HIVE E A CAPACIDADE DO ROBÔ.....101
17.5 BATERIA REAL E SENSOR DE TENSÃO.....102
17.6 CALIBRAÇÃO DO LANÇADOR.....103
17.7 A C270 A 0°: AS PEÇAS PELA COR.....104
17.8 A TELA DO SANDBOX: CADA OPÇÃO NUM LUGAR SÓ.....105
17.9 VALIDAÇÃO.....107
18. LIMITAÇÕES DO MODELO.....108
19. CONCLUSÃO E DIRETRIZES DE VALIDAÇÃO NO HARDWARE REAL.....111
20. HISTÓRICO DE DESENVOLVIMENTO (CHANGELOG).....113
REFERÊNCIAS.....118

1. INTRODUÇÃO E CONTEXTUALIZAÇÃO PEDAGÓGICA

O desenvolvimento de competências em robótica móvel na FIRST Tech Challenge (FTC) exige que os estudantes apliquem conceitos de matemática e de engenharia de controle em tempo real, sob restrições físicas. Um gargalo recorrente das equipes é a disputa pelo chassi físico para testar software: o tempo de teste é limitado e componentes caros ficam expostos a colisões e desgaste. O FTC Odometry Sandbox é um simulador interativo, executado no navegador — com uma arena vista de cima e, desde a versão 25, também uma vista 3D —, que permite desenvolver e validar algoritmos de localização e navegação antes de levá-los ao robô. Além da localização, ele permite treinar a programação dos quatro tipos de chassi mais usados (movimento ideal, tração diferencial, Mecanum e swerve), inclusive com um controle (gamepad) real. Ele não substitui os testes no hardware, mas reduz o número de tentativas necessárias no robô real e permite que vários integrantes estudem ao mesmo tempo.

A ideia do projeto surgiu no ambiente do curso de Desenvolvimento de Sistemas do SENAI, oportunidade concedida ao autor por ser aluno regular do SESI. Embora o conteúdo inicial do curso trate de lógica de programação introdutória, o contato com esse ambiente mostrou que o projeto era viável e motivou um estudo autodidata de trigonometria, geometria analítica e álgebra linear aplicadas à robótica, que resultou neste software. O simulador foi pensado para servir também como material de estudo para outros integrantes da equipe.

O desenvolvimento começou de forma independente durante o off-season da temporada FTC DECODE, sem relação com as missões, regras ou pontuações daquela temporada, e continua durante a temporada 2026-2027, BIOBUZZ: a versão 25 acrescentou o campo, as peças e um robô de exemplo desse jogo (Seções 15 e 16), num campo que se escolhe na tela, ao lado do campo de treino das versões anteriores. Em linha com o Gracious Professionalism (Profissionalismo Gracioso), a plataforma compartilha o conhecimento para que qualquer integrante domine os conceitos antes de modificar o robô físico. Como o núcleo do simulador é a movimentação e a localização do robô, que não dependem do tema do jogo, os algoritmos testados aqui podem ser reaproveitados em qualquer temporada.

A partir da versão 18, o simulador passou a ser uma das partes de um aplicativo de aprendizagem. A versão 18 acrescentou uma tela inicial com sistema de progresso (Seção 9); a versão 19, o Mapa de Estudos, que transforma em jogo os 86 encontros do plano de estudo do programador novo de FTC (Seção 10); a versão 20, o Caminho do André (Seção 11); a versão 21, um desafio de código ao fim de cada lição e de cada passo, que a versão 24 equipou com editor, console, teste de mesa e depurador e a

versão 25 com dicas em escada (Seção 12); a versão 22, a tela Robô Completo, em que o estudante escreve os arquivos Java inteiros do robô e o opera por um Driver Hub simulado (Seção 13); a versão 23, os sensores de localização dessa tela — o Pinpoint V2, pods de odometria individuais e a Limelight 3A (Seção 14); a versão 25, a vista 3D do campo, o campo BIOBUZZ da temporada 2026-2027 e o modo pontual de visão (Seção 15), com as peças do jogo, os mecanismos do robô e a física (Seção 16); e a versão 26, a partida com outros robôs, a bateria real, a calibração do lançador e a visão das peças pela C270 (Seção 17). O objetivo dessas partes não é preparar o estudante para resolver provas de matemática ou de física, e sim mostrar onde cada conteúdo aparece no código do robô e como aplicá-lo.

2. MATRIZ DE COMPETÊNCIAS CIENTÍFICAS E MATEMÁTICAS (S.T.E.A.M.)

O FTC Odometry Sandbox foi estruturado como uma ferramenta de transposição didática: ele transforma problemas de engenharia em objetos de estudo acessíveis. Ao programar o robô no simulador, o estudante mobiliza os seguintes conhecimentos:

- **Geometria Analítica:** coordenadas cartesianas no plano da arena, em dois sistemas de referência (o da tela e o do campo FTC), relacionando a posição atual do robô com pontos de destino.
- **Trigonometria:** seno e cosseno para projetar vetores entre referenciais, normalização de ângulos e a função arcotangente de dois argumentos (atan2), usada para calcular a direção de um alvo e o ângulo das rodas swerve sem indeterminação quando $\Delta x = 0$.
- **Álgebra Linear:** a matriz de rotação 2D que converte deslocamentos do referencial do robô para o referencial do campo, e o efeito de mudar a convenção dos eixos.
- **Cálculo Numérico:** integração discreta dos deslocamentos a cada ciclo do programa e o acúmulo de erro (drift) que essa integração produz.
- **Cinemática de Chassis:** cinemática inversa (do movimento desejado para os motores) e direta (dos motores para o movimento) dos chassis de tração diferencial, Mecanum e swerve, incluindo o produto vetorial $\omega \times r$ que descreve a velocidade de cada roda durante um giro.
- **Metrologia e Calibração:** conversão entre ticks de encoder e distância, efeito de um diâmetro de roda incorreto, influência do ruído dos sensores e comparação entre dead wheels e encoders das rodas de tração.

- **Aprendizagem por domínio e gamificação:** cada encontro do plano de estudo é uma lição com desafios verificados automaticamente; a lição seguinte só é liberada quando todos os desafios são resolvidos, e o progresso é recompensado com pontos de experiência (XP), níveis e medalhas.
- **Cálculo e Métodos Numéricos aplicados:** derivadas de curvas paramétricas, vetores tangente e normal, curvatura, o método de Newton para o ponto mais próximo e a integração numérica da aceleração, usados para construir um seguidor de caminho por campo vetorial guia (GVF).

O diferencial dessa abordagem é o retorno imediato: um sinal trocado, uma equação mal formulada ou um eixo invertido aparecem na tela como um comportamento visível do robô virtual. O erro passa a ser um ponto de partida para investigação, e o estudante se prepara para explicar suas decisões técnicas às bancas da FIRST.

3. ARQUITETURA DE LOCALIZAÇÃO CINEMÁTICA (DEAD WHEELS + IMU)

O sistema de localização simulado combina duas rodas passivas de medição (dead wheels), montadas perpendicularmente entre si, com o heading fornecido pela Unidade de Medida Inercial (IMU) do Control Hub. Como as dead wheels não são tracionadas, elas não sofrem o deslizamento das rodas motrizes. No simulador, o ruído de deslizamento é aplicado ao movimento do chassi antes da leitura: as rodas de medição acompanham o deslocamento real e apenas o ruído próprio dos encoders se acumula (ver Seção 8). Isso vale para os quatro modos de movimento da Seção 4. Opcionalmente, uma terceira roda, paralela à primeira, permite calcular o heading sem a IMU.

3.1 SISTEMAS DE COORDENADAS DO SIMULADOR

O simulador oferece dois modos de coordenadas, escolhidos na aba "Odometria" (Tabela 1). A troca de modo reinicia a simulação.

Tabela 1 – Modos de coordenadas do simulador

Grandeza	Modo Sandbox	Modo FTC
Unidade	Unidade da arena (1000 un. = largura do campo)	Polegada (1 un. = 0,144 pol.)
Origem e início	Canto superior esquerdo; robô inicia em (500, 500)	Centro do campo; robô inicia em (0, 0)
Eixo Y	Para baixo (convenção de tela)	Para cima
Heading	Graus, sentido horário; 0° = robô voltado para cima	Graus, sentido anti-horário; 0° = voltado para +X; robô inicia em 90°
Roda perpendicular (x)	Positiva para a direita	Positiva para a esquerda
Roda paralela (y)	Positiva para trás	Positiva para a frente
Comandos autônomos	`dx`, `dy`, `turn`	`forward`, `left`, `turn`

Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

O modo FTC segue a mesma convenção usada pela IMU e por bibliotecas como o Road Runner, que nomeia as rodas de "par" (paralela) e "perp" (perpendicular). Por isso, ele é o modo recomendado antes de migrar o código para o robô: as equações escritas nele podem ser transferidas sem inversões de sinal. O modo Sandbox foi mantido por compatibilidade com as versões anteriores. Em ambos os modos, o robô simulado mede 125 un. de lado, o equivalente a 18 polegadas, tamanho máximo do robô na FTC.

3.2 COMPENSAÇÃO DO ARCO ROTACIONAL E MATRIZ DE ROTAÇÃO

Quando o robô gira, as dead wheels, afastadas do centro de rotação, percorrem um arco e registram deslocamento mesmo sem translação. O comprimento desse arco é o produto do ângulo, em radianos, pela distância da roda ao centro. A variação de heading da IMU ($\Delta\psi$, em graus, normalizada para o intervalo de −180° a 180°) é primeiro convertida para radianos:

$$\Delta\theta = \Delta\psi \cdot \pi \;/\; 180 \tag{1}$$

Em seguida, calcula-se o arco percorrido por cada roda, em que R_x (`SENSOR_OFFSET_X`) é a distância lateral da roda paralela ao centro e R_y (`SENSOR_OFFSET_Y`) é a distância longitudinal da roda perpendicular:

$$S_x = \Delta\theta \cdot R_y \qquad S_y = \Delta\theta \cdot R_x \tag{2}$$

Os arcos são descontados das leituras brutas, isolando a translação do chassi no referencial do robô:

$$\Delta x_{local} = \Delta x_{bruto} - S_x \qquad \Delta y_{local} = \Delta y_{bruto} - S_y \tag{3}$$

Por fim, a matriz de rotação 2D transporta esse deslocamento para o referencial do campo, usando o heading atual θ em radianos. No modo Sandbox, o eixo Y já aponta para baixo e o heading é medido no sentido horário; por isso, a matriz é aplicada diretamente, sem inversão adicional de sinal em Δy:

$$\Delta X = \Delta x_{local} \cdot \cos\theta - \Delta y_{local} \cdot \sin\theta \tag{4}$$

$$\Delta Y = \Delta x_{local} \cdot \sin\theta + \Delta y_{local} \cdot \cos\theta \tag{5}$$

No modo FTC, a roda paralela mede o avanço ($\Delta f = \Delta y_{bruto} - S_y$, positivo para a frente) e a perpendicular mede o deslocamento lateral ($\Delta l = \Delta x_{bruto} - S_x$, positivo para a esquerda). Como o heading é anti-horário e 0° aponta para +X, a matriz fica:

$$\Delta X = \Delta f \cdot \cos\theta - \Delta l \cdot \sin\theta \tag{6}$$

$$\Delta Y = \Delta f \cdot \sin\theta + \Delta l \cdot \cos\theta \tag{7}$$

No simulador, as duas rodas registram arco positivo quando o robô gira no sentido positivo do modo ativo. No robô real, o sinal de cada arco depende do lado em que a roda foi montada e deve ser conferido girando o robô no lugar: com o sinal correto, a posição calculada permanece praticamente constante durante o giro.

3.3 MODO DE TRÊS RODAS: HEADING CALCULADO PELOS ENCODERS

Ao selecionar "3 (X + Y + Y2)" na aba "Odometria", o simulador passa a fornecer uma segunda roda paralela (`rawEncoders.y2` e `prevEncoderY2`), montada simetricamente à primeira, a uma distância R_x do outro lado do centro. Durante uma rotação, as duas rodas registram arcos de sinais opostos; durante uma translação, registram o mesmo valor. Assim, a variação de heading e o avanço do robô podem ser calculados apenas pelos encoders:

$$\Delta\theta = (\Delta y - \Delta y_2) \;/\; (2 \cdot R_x) \tag{8}$$

$$\Delta y_{local} = (\Delta y + \Delta y_2) \;/\; 2 \tag{9}$$

Com a opção "IMU: Desligada", `imu.heading` passa a valer NaN, o que obriga o estudante a acumular o heading calculado em `myState.heading`, como ocorre em

robôs que não usam a IMU para a odometria. O resultado dessa configuração está na Tabela 9.

3.4 LEITURAS EM TICKS E CALIBRAÇÃO DO DIÂMETRO DA RODA

Encoders reais informam ticks, e não distâncias. O simulador fornece as leituras em ticks (`rawTicks.x`, `rawTicks.y` e `rawTicks.y2`), calculadas a partir de dois parâmetros da aba "Odometria": o diâmetro real da roda *D* (em milímetros) e o número de ticks por volta (TPR). A distância percorrida é:

$$\mathrm{d} = \mathrm{ticks} \cdot (\pi \cdot \mathrm{D}) \;/\; \mathrm{TPR} \tag{10}$$

Para conversão, 1 un. da arena equivale a 3,6576 mm e 1 polegada a 25,4 mm. Os ticks são números inteiros, o que reproduz a resolução finita dos encoders. O exercício de calibração funciona assim: o estudante escreve no código o diâmetro nominal da roda (por exemplo, 32 mm). Se o diâmetro "real" do painel for alterado, simulando desgaste ou tolerância de fabricação, a odometria passa a errar em proporção à distância percorrida; um diâmetro real 3% maior gera cerca de 3% de erro em linha reta. Para corrigir, o estudante move o robô por uma distância conhecida, compara com o valor calculado e ajusta a constante, que é o mesmo procedimento usado no robô real.

3.5 PINPOINT ODOMETRY COMPUTER SIMULADO

O goBILDA Pinpoint Odometry Computer é um coprocessador que lê as dead wheels, integra uma IMU própria e entrega a pose do robô já calculada. O simulador oferece um objeto `pinpoint` com uma API simplificada, inspirada no driver oficial: `update()`, `getPosX()`, `getPosY()`, `getHeading()`, `getEncoderX()`, `getEncoderY()` e `resetPosAndIMU()`. Como no driver real, `getHeading()` retorna o heading em radianos, sem normalização (acumula voltas completas). As distâncias são retornadas na unidade do modo de coordenadas ativo, e as leituras de encoder, em ticks.

O Pinpoint simulado usa a sua própria IMU e, por isso, continua funcionando mesmo com a IMU do Control Hub desligada. Ele tem dois usos pedagógicos: servir de referência para comparar com a odometria escrita pelo estudante e permitir que a equipe aprenda a API antes de usar o hardware. Por ser uma simplificação, ele não reproduz a comunicação I²C, as velocidades nem as funções de configuração do driver real (como `setOffsets`); os offsets usados são os definidos no painel.

4. MODOS DE MOVIMENTO E CINEMÁTICA DO CHASSI

O simulador oferece quatro modos de movimento, escolhidos no seletor "Modo de Movimento" (Tabela 2). Em todos eles, as dead wheels medem o movimento real

do chassi, de modo que a odometria da Seção 3 vale para qualquer chassi. O que muda é a forma como o movimento desejado (avançar, deslocar-se de lado e girar) se transforma em movimento: a **cinemática inversa** converte o movimento desejado em potências de motor, e a **cinemática direta** converte as potências em movimento do chassi. Uma potência de 1,0 numa roda corresponde à velocidade linear máxima definida no painel.

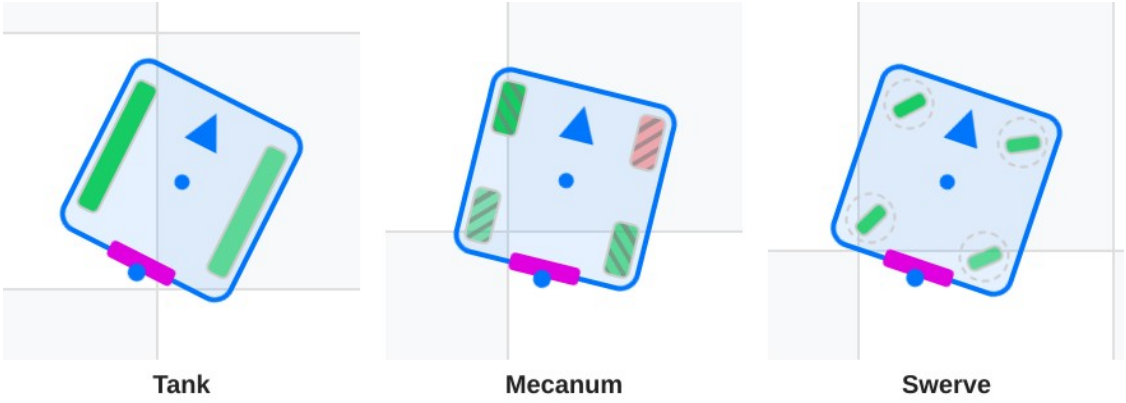
Tabela 2 – Modos de movimento do simulador

Modo	Motores	Movimentos possíveis	Efeitos simulados
Ideal	Nenhum (comando direto do chassi)	Frente, lateral e giro, exatos	Nenhum
Tank	2 (esquerda e direita)	Frente e giro; sem deslocamento lateral	Inércia dos motores
Mecanum	4 (FL, FR, BL, BR)	Frente, lateral e giro	Inércia; perda de 10% no deslocamento lateral
Swerve	4 módulos (ângulo e velocidade)	Frente, lateral e giro	Inércia; velocidade de esterçamento limitada

Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

As equações desta seção usam o referencial do robô do simulador: v_y é a velocidade para a frente, v_x é a velocidade para a direita e ω é a velocidade de giro, positiva no sentido horário (em radianos por ciclo). T é a distância entre as rodas esquerda e direita ("Distância entre rodas", 110 un. por padrão) e B é a distância entre os eixos dianteiro e traseiro (90 un.). A Figura 1 mostra como cada chassi aparece na arena: as rodas ficam verdes quando giram para a frente e vermelhas quando giram para trás, com intensidade proporcional à potência.

Figura 1 – Robô nos modos Tank, Mecanum e Swerve, avançando para a frente e para a direita enquanto gira



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

4.1 MOVIMENTO IDEAL

No modo Ideal, o chassi executa exatamente o comando recebido, sem rodas nem inércia. É o modo indicado para treinar apenas os conceitos de odometria e navegação, sem a interferência das características de um chassi. Todas as medições da Seção 8.1 foram feitas neste modo.

4.2 TRAÇÃO DIFERENCIAL (TANK)

Na tração diferencial, dois conjuntos de rodas (esquerdo e direito) são acionados de forma independente. Para avançar, os dois lados giram juntos; para virar à direita, o lado esquerdo gira mais rápido que o direito. A cinemática inversa, que calcula as velocidades das rodas esquerda (v_E) e direita (v_D), é:

$$\mathrm{v_E} = \mathrm{v_y} + \omega \cdot \mathrm{T} \;/\; 2 \qquad \mathrm{v_D} = \mathrm{v_y} - \omega \cdot \mathrm{T} \;/\; 2 \tag{11}$$

E a cinemática direta, que o simulador usa para mover o robô a partir das potências reais, é:

$$\mathrm{v_y} = (\mathrm{v_E} + \mathrm{v_D}) \;/\; 2 \qquad \omega = (\mathrm{v_E} - \mathrm{v_D}) \;/\; \mathrm{T} \tag{12}$$

Essas são as relações da classe de cinemática diferencial do WPILib, com o sinal de ω invertido, porque aqui o giro horário é positivo. Como não existe componente lateral, as teclas A e D (e o eixo horizontal do stick esquerdo) não movem o robô: para mudar de direção, o robô tank precisa girar primeiro, o que afeta a estratégia de navegação.

4.3 TRAÇÃO MECANUM

As rodas Mecanum têm roletes a 45°, que permitem deslocamentos em qualquer direção. A cinemática inversa segue as equações do Game Manual 0, com $L = T/2 + B/2$; se alguma potência ultrapassar 1, todas são divididas pelo maior valor, preservando a proporção:

$$\mathrm{FL} = \mathrm{v_y} + \mathrm{v_x} + \omega \cdot \mathrm{L} \qquad \mathrm{FR} = \mathrm{v_y} - \mathrm{v_x} - \omega \cdot \mathrm{L} \tag{13}$$

$$\mathrm{BL} = \mathrm{v_y} - \mathrm{v_x} + \omega \cdot \mathrm{L} \qquad \mathrm{BR} = \mathrm{v_y} + \mathrm{v_x} - \omega \cdot \mathrm{L} \tag{14}$$

A cinemática direta recupera o movimento do chassi a partir das velocidades das quatro rodas:

$$\mathrm{v_y} = (\mathrm{FL} + \mathrm{FR} + \mathrm{BL} + \mathrm{BR}) \;/\; 4 \qquad \mathrm{v_x} = \eta \cdot (\mathrm{FL} - \mathrm{FR} - \mathrm{BL} + \mathrm{BR}) \;/\; 4 \tag{15}$$

$$\omega = (\mathrm{FL} - \mathrm{FR} + \mathrm{BL} - \mathrm{BR}) \;/\; (4 \cdot \mathrm{L}) \tag{16}$$

O coeficiente η = 0,9 representa a perda de eficiência do deslocamento lateral causada pelo deslizamento dos roletes, efeito conhecido no robô real; o Game Manual 0 sugere compensá-lo multiplicando o comando lateral por cerca de 1,1. Como as dead wheels medem o movimento real, a odometria por dead wheels não é afetada,

mas a odometria feita com os encoders das rodas de tração superestima o deslocamento lateral (Seção 8.2).

4.4 TRAÇÃO SWERVE

No swerve, cada um dos quatro módulos tem dois motores: um que gira a roda (velocidade) e outro que a esterça (ângulo). Para um módulo na posição (x_i, y_i) do robô, a velocidade desejada é a soma da translação do chassi com a velocidade produzida pelo giro, dada pelo produto vetorial $\omega \times r_i$:

$$v_{ix} = v_x + \omega \cdot y_i \qquad v_{iy} = v_y - \omega \cdot x_i \tag{17}$$

O ângulo e a velocidade do módulo são obtidos com atan2 e com o Teorema de Pitágoras; o ângulo é medido a partir da frente do robô, no sentido horário:

$$\alpha_i = \operatorname{atan2}(v_{ix},\, v_{iy}) \qquad s_i = \sqrt{(v_{ix}^2 + v_{iy}^2)} \tag{18}$$

Se o novo ângulo exigir um giro maior que 90° em relação ao ângulo atual, é mais rápido apontar a roda para o lado oposto e invertê-la (α + 180° e velocidade negativa). Essa otimização, também usada pelo WPILib, está implementada no mixer do simulador e é um dos desafios propostos no modelo de TeleOp swerve. Na cinemática direta, cada módulo empurra o robô na direção do seu ângulo **real**, que se aproxima do ângulo desejado a uma velocidade limitada ("Vel. de esterçamento", 15° por ciclo por padrão). Assim, logo após uma mudança brusca de direção, o robô se move por alguns ciclos numa direção intermediária, como acontece no robô real. A velocidade de giro do chassi é calculada pelo método dos mínimos quadrados a partir das quatro rodas.

4.5 INÉRCIA DOS MOTORES E ENCODERS DE TRAÇÃO

Nos modos Tank, Mecanum e Swerve, a potência real de cada motor não muda instantaneamente: a cada ciclo, ela se aproxima da potência desejada em um passo máximo de $1 - 0{,}95 \cdot I$, em que I é a inércia escolhida (de 0 a 100%). Com o valor padrão de 40%, o passo é de 0,62 por ciclo. A Tabela 3 resume as capacidades de movimento medidas em cada modo, sem ruído.

Tabela 3 – Capacidade de movimento medida em cada modo (sem ruído, inércia de 40%)

Modo	Frente (un./ciclo)	Lateral (un./ciclo)	Giro (°/ciclo)	Ciclos até 95% da vel. lateral
Ideal	4	4	1,5	1
Tank	4	0	1,5	—
Mecanum	4	3,6	1,5	2
Swerve	4	4	1,5	5

Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

Os encoders das rodas de tração ficam disponíveis no código em `driveEncoders` (distância girada por cada roda e, no swerve, o ângulo real de cada módulo em `driveEncoders.angles`), com os valores do ciclo anterior em `prevDriveEncoders`. Diferentemente das dead wheels, esses encoders contam a rotação da roda mesmo quando ela desliza ou quando o robô empurra uma parede. No swerve, o tempo maior para atingir a velocidade lateral (5 ciclos, contra 2 no Mecanum) vem do tempo de esterçamento dos módulos.

5. PROGRAMAÇÃO DO ROBÔ: COMANDOS, MOTORES E GAMEPAD

O código do estudante pode controlar o robô em três níveis: pelos comandos de alto nível (`commands`), que usam o mixer pronto do simulador; pelos motores (`motors`), em que o próprio estudante escreve a cinemática inversa; e pela leitura de um controle real (`gamepad1` e `gamepad2`).

5.1 INTERFACE DE COMANDOS AUTÔNOMOS

O objeto `commands` (Tabela 4) define o movimento desejado do chassi. Os valores são aplicados no ciclo seguinte, somam-se aos comandos do teclado e do controle e são limitados pelos controles de velocidade linear e de rotação. Nos modos Tank, Mecanum e Swerve, esses comandos passam pela cinemática inversa da Seção 4, como se a equipe usasse uma biblioteca de chassi pronta; no modo Tank, o componente lateral é ignorado.

Tabela 4 – Campos do objeto commands

Movimento	Modo Sandbox	Modo FTC
Lateral	`commands.dx`: un./ciclo, positivo para a direita	`commands.left`: pol./ciclo, positivo para a esquerda
Longitudinal	`commands.dy`: un./ciclo, positivo para trás	`commands.forward`: pol./ciclo, positivo para a frente
Giro	`commands.turn`: graus/ciclo, positivo no sentido horário	`commands.turn`: graus/ciclo, positivo no sentido anti-horário

Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

5.2 CONTROLE DIRETO DOS MOTORES E MODELOS DE TELEOP

Para treinar a programação de cada chassi, o estudante pode preencher diretamente o objeto `motors` (Tabela 5). Quando ele é preenchido, tem prioridade sobre o mixer pronto do simulador. O objeto `DRIVE` informa o modo ativo, a distância entre rodas, a distância entre eixos e a posição de cada módulo, necessária para a cinemática do swerve.

Tabela 5 – Campos do objeto motors em cada modo

Modo	Campos	Valores
Tank	`motors.left`, `motors.right`	Potência de −1 a 1
Mecanum	`motors.frontLeft`, `motors.frontRight`, `motors.backLeft`, `motors.backRight`	Potência de −1 a 1
Swerve	`motors.frontLeft.angle` e `motors.frontLeft.speed` (idem para os demais módulos)	Ângulo em graus (0° = frente, no sentido do heading do modo); velocidade de −1 a 1

Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

O menu "Modelo" do editor oferece modelos iniciais de TeleOp para os três chassis. Cada modelo lê o controle, deixa um TODO com dicas para o estudante completar a cinemática inversa e já seleciona o chassi correspondente. O Código 1 mostra uma solução de referência para o TeleOp Mecanum, que aplica as equações (13) e (14) com normalização; ela foi verificada nos testes automatizados do simulador.

Código 1 – Solução de referência do TeleOp Mecanum

```
double y = -gamepad1.left_stick_y;   // frente
double x = gamepad1.left_stick_x;    // lateral (direita = +)
double rx = gamepad1.right_stick_x;  // giro (horário = +)

double d = Math.max(Math.abs(y) + Math.abs(x) + Math.abs(rx), 1);
motors.frontLeft = (y + x + rx) / d;  motors.backLeft = (y - x + rx) / d;
motors.frontRight = (y - x - rx) / d; motors.backRight = (y + x - rx) / d;
```

Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

5.3 SUPORTE A GAMEPAD

O simulador lê controles conectados ao computador por USB ou Bluetooth por meio da Gamepad API dos navegadores, consultando o estado de cada controle a cada ciclo, como recomenda a documentação da MDN. O primeiro controle é exposto ao código como `gamepad1` e o segundo como `gamepad2`, com os mesmos nomes de campo da classe Gamepad do FTC SDK (Tabela 6). Os sticks variam de −1 a 1 e os gatilhos de 0 a 1. Como no FTC, empurrar o stick para a frente produz `left_stick_y` negativo, conforme observa o Game Manual 0; por isso, os modelos usam `y = -gamepad1.left_stick_y`.

Tabela 6 – Campos do gamepad e mapeamento padrão ("standard") do navegador

Campo (FTC SDK)	Elemento do controle	Índice no navegador
`left_stick_x`, `left_stick_y`	Stick esquerdo	axes[0], axes[1]
`right_stick_x`, `right_stick_y`	Stick direito	axes[2], axes[3]
`left_trigger`, `right_trigger`	Gatilhos (0 a 1)	buttons[6], buttons[7]
`a`, `b`, `x`, `y`	Botões frontais (✕ ○ □ △ no PlayStation)	buttons[0] a buttons[3]
`left_bumper`, `right_bumper`	Botões superiores	buttons[4], buttons[5]
`back`, `start`, `guide`	Botões centrais	buttons[8], buttons[9], buttons[16]
`left_stick_button`, `right_stick_button`	Pressionar os sticks	buttons[10], buttons[11]
`dpad_up`, `dpad_down`, `dpad_left`, `dpad_right`	Direcional	buttons[12] a buttons[15]

Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

Os sticks passam por uma zona morta de 0,08, reescalonada para que o valor continue chegando a 1: sem ela, um controle em repouso enviaria pequenos valores e o robô andaria sozinho. O teclado continua funcionando e soma-se ao `gamepad1` (W/S no eixo vertical do stick esquerdo, A/D no horizontal e as setas no stick direito), com o resultado limitado ao intervalo de −1 a 1. Por privacidade, o navegador só revela o controle depois que um botão é pressionado com a página aberta; o painel "Teclado e Controle" indica a conexão e mostra a posição dos sticks em tempo real. O funcionamento foi verificado por testes automatizados que simulam controles; a validação com o controle físico da equipe deve ser registrada no Caderno de Engenharia.

6. ALGORITMOS DE NAVEGAÇÃO HOLONÔMICA

Os algoritmos desta seção usam a odometria da Seção 3 e os comandos da Seção 5.1. Eles foram escritos para chassis holonômicos (Ideal, Mecanum e Swerve); num chassi tank, o componente lateral é ignorado, e a navegação precisa alternar entre girar em direção ao alvo e avançar.

6.1 LÓGICA DE APONTAMENTO E ANGLE WRAPPING

Para alinhar o robô a um ângulo desejado, calcula-se o erro angular e normaliza-se o resultado para o intervalo de −180° a 180°, garantindo que a correção ocorra pelo menor caminho. Como o operador `%` pode retornar valores negativos em Java e em JavaScript, soma-se 360 antes da segunda operação (Código 2).

Código 2 – Normalização do erro angular (angle wrapping)

```
double diff = targetHeading - imu.heading;
diff = ((diff % 360) + 360) % 360;
if (diff > 180) {
    diff -= 360;
}
```

Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

6.2 NAVEGAÇÃO PONTO A PONTO (GO TO TARGET)

Na navegação ponto a ponto, o erro de posição no campo (e_X, e_Y) é decomposto no referencial do robô usando o heading atual θ, gerando comandos simultâneos de avanço, deslocamento lateral e giro (Tabela 7). A distância até o alvo, calculada pelo Teorema de Pitágoras, serve de critério de parada.

Tabela 7 – Comandos holonômicos e decomposição vetorial (modo FTC)

Componente	Cálculo	Ação no chassi	Aplicação
Avanço (`forward`)	$e_X \cdot \cos\theta + e_Y \cdot \sin\theta$	Movimento para frente ou para trás	Aproximação do alvo
Lateral (`left`)	$-e_X \cdot \sin\theta + e_Y \cdot \cos\theta$	Deslocamento lateral puro	Alinhamento fino
Giro (`turn`)	$k \cdot$ diff (erro angular normalizado)	Giro em torno do centro	Manutenção da orientação

Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

O Código 3 reúne a odometria do modo FTC e o Go To Target. Ele usa controle proporcional simples (ganho de 0,15 para a translação e de 0,1 para o giro) e uma margem de parada de 0,75 polegada (cerca de 5 un.). Esses valores são didáticos e podem ser alterados. Na execução de teste, no modo Ideal, o robô partiu de (0, 0) com heading de 90° e parou a 0,66 polegada do alvo (24, 36), dentro da margem, com heading final de 0,0° (Figura 2).

Código 3 – Odometria e Go To Target no modo FTC

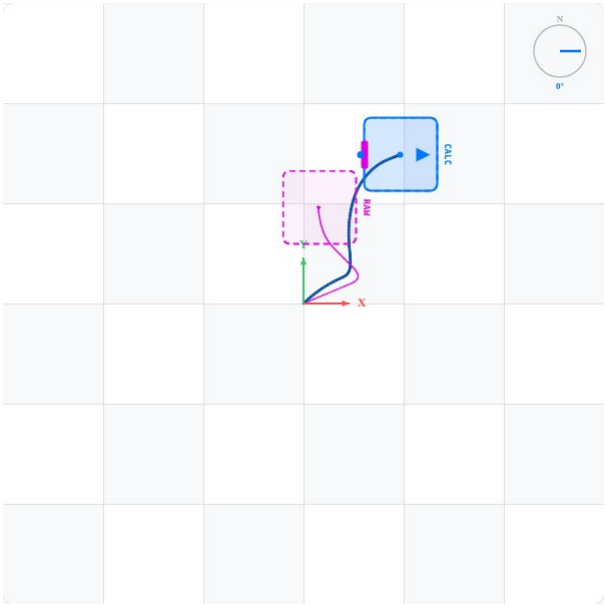
```
// --- Odometria (modo FTC) ---
double dHeading = imu.heading - prevImuHeading;
if (dHeading > 180) dHeading -= 360;
if (dHeading < -180) dHeading += 360;
double dTheta = Math.toRadians(dHeading);
double theta = Math.toRadians(imu.heading);
double left = (rawEncoders.x - prevEncoderX) - dTheta * SENSOR_OFFSET_Y;
double fwd  = (rawEncoders.y - prevEncoderY) - dTheta * SENSOR_OFFSET_X;
myState.x += fwd * Math.cos(theta) - left * Math.sin(theta);
myState.y += fwd * Math.sin(theta) + left * Math.cos(theta);

// --- Go To Target ---
double targetX = 24, targetY = 36, targetHeading = 0;
double errX = targetX - myState.x;
double errY = targetY - myState.y;
```

```
double dist = Math.sqrt(errX * errX + errY * errY);
double diff = targetHeading - imu.heading;
diff = ((diff % 360) + 360) % 360;
if (diff > 180) diff -= 360;
if (dist > 0.75) {
    commands.forward = (errX * Math.cos(theta) + errY * Math.sin(theta)) * 0.15;
    commands.left    = (-errX * Math.sin(theta) + errY * Math.cos(theta)) * 0.15;
}
commands.turn = diff * 0.1;
telemetry.addData("dist", dist);
return { x: myState.x, y: myState.y, heading: imu.heading };
```

Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

Figura 2 – Execução do Go To Target no modo FTC, com eixos e fantasmas ativados



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

7. METROLOGIA, TELEMETRIA E DIAGNÓSTICO EM AMBIENTE VIRTUAL

A interface foi organizada para que as ações mais usadas fiquem sempre à vista (Figura 3). Acima da arena, uma barra reúne os botões de pausar, reiniciar, mostrar fantasmas, eixos e rastros; abaixo dela ficam a legenda e o painel "Teclado e Controle". À direita, o seletor de modo de movimento mostra a potência de cada motor (e, no swerve, o ângulo de cada módulo), e três abas separam a telemetria, a configuração da odometria e os ajustes do chassi. O cabeçalho tem atalhos para a arena e para o editor de código.

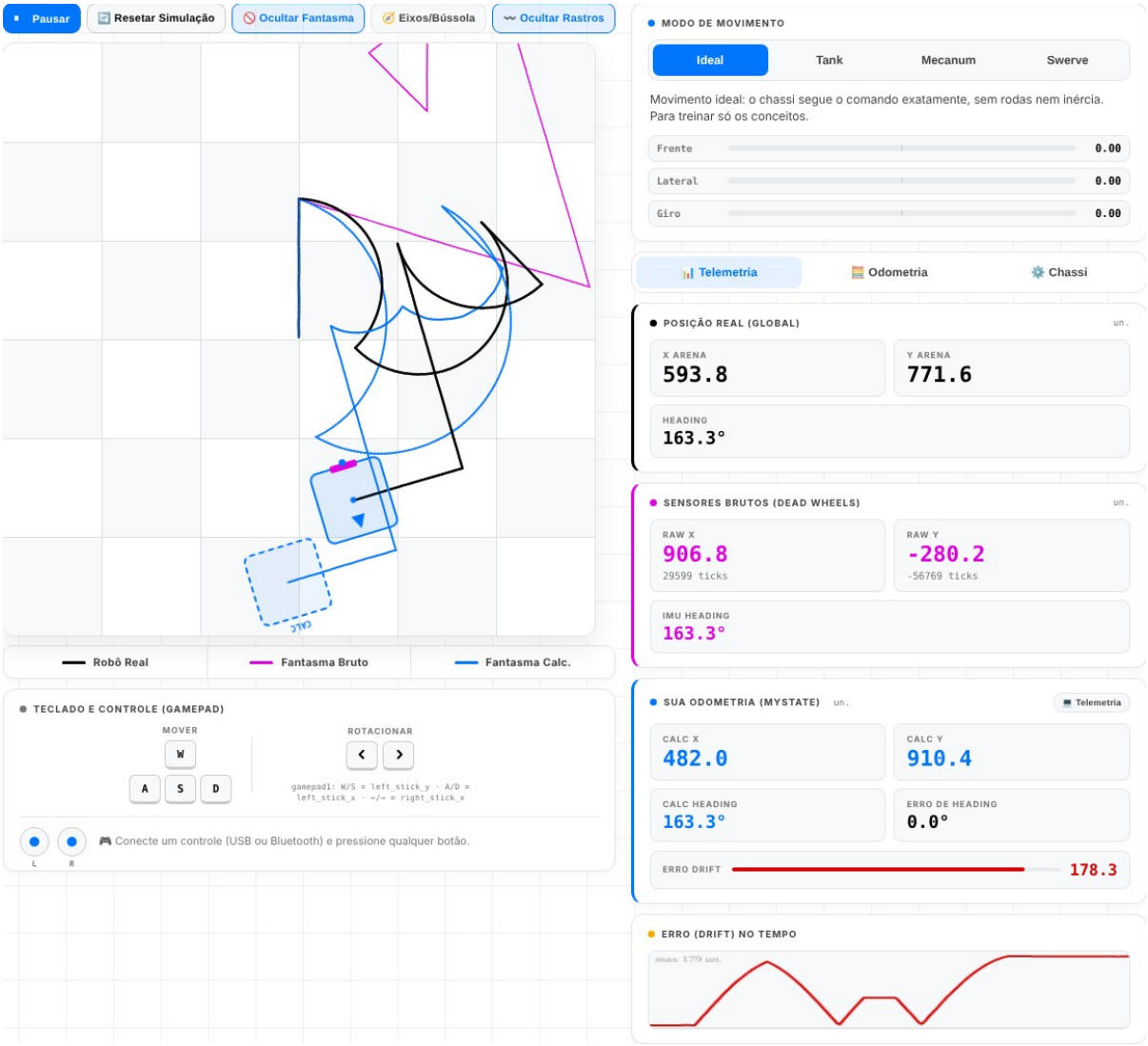
A aba "Telemetria" apresenta, lado a lado, camadas de dados independentes, para que o estudante compare o que o robô fez com o que o seu código calculou:

- **Posição Real (gabarito):** a pose verdadeira gerada pelo simulador. Ela está disponível no código em `robotState` apenas para comparação e não deve ser usada no cálculo da odometria.

- **Sensores Brutos:** as leituras acumuladas das dead wheels, em distância e em ticks, e o heading da IMU, que pode ter ruído próprio.
- **Sua Odometria:** a posição e o heading retornados pelo código do estudante, com o erro de posição (drift), o erro de heading e um gráfico do drift ao longo do tempo.

Para o diagnóstico, o simulador oferece ainda fantasmas que mostram a pose calculada e a pose bruta, rastros coloridos da trajetória real, bruta e calculada, um console de telemetria no estilo da Driver Station (`telemetry.addData`), eixos e bússola, e a indicação da linha do código em que ocorreu um erro. Se o robô se move, mas os valores de "Sua Odometria" divergem da "Posição Real", há uma falha no modelo matemático. As causas mais comuns são: sinal trocado na matriz de rotação, arco não compensado, offset incorreto, diâmetro de roda errado ou convenção de heading trocada entre os modos.

Figura 3 – Interface com a barra de ações, o seletor de movimento, as abas e o painel de controle



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

8. RESULTADOS: MEDIÇÕES NO SIMULADOR

As medições a seguir foram obtidas com rotas de teste automatizadas, repetidas com a mesma sequência pseudoaleatória em todas as execuções, para que as diferenças se devessem apenas ao algoritmo ou ao chassi. O drift é a distância entre a posição real e a calculada ao final da rota.

8.1 ETAPAS DO ALGORITMO DE ODOMETRIA

Para medir o efeito de cada etapa do algoritmo, foi executada no modo Ideal uma rota de 600 ciclos (cerca de 10 segundos a 60 ciclos por segundo), que combina avanço, ré, deslocamento lateral, giros e movimentos combinados com giro. As versões testadas foram: (a) sem correção, somando diretamente os deltas brutos; (b) apenas a matriz de rotação, sem compensação do arco; e (c) completa, com compensação do arco e matriz de rotação.

Tabela 8 – Drift final em polegadas (unidades da arena entre parênteses)

Versão do código	Ruído 0%	Ruído 50%	Ruído 100%
(a) Sem correção	162,39 (1127,7 un.)	156,97 (1090,1 un.)	151,39 (1051,3 un.)
(b) Só matriz de rotação	25,66 (178,2 un.)	25,58 (177,6 un.)	25,48 (176,9 un.)
(c) Completa (arco + matriz)	0,00 (0,0 un.)	0,24 (1,7 un.)	0,48 (3,4 un.)

Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

A versão (a) produz um erro maior que o próprio campo, porque os deltas medidos estão no referencial do robô e precisam ser rotacionados. A versão (b) erra cerca de 25,66 polegadas em todos os níveis de ruído: o arco não compensado aparece a cada giro e é praticamente independente do ruído. A trajetória desse caso está na Figura 3, em que o rastro calculado (azul) se afasta do real (preto) a cada rotação. A versão (c) tem drift nulo sem ruído, o que valida as equações (3) a (5), e o erro cresce lentamente com o ruído (0,24 e 0,48 polegada), porque apenas o ruído de leitura dos encoders se acumula.

Tabela 9 – Outras configurações (mesma rota, modo Ideal)

Configuração	Drift final	Erro de heading
Completa, modo FTC, ruído 0%	0,00 (0,0 un.)	0,00°
Completa, modo FTC, ruído 50%	0,24 (1,7 un.)	0,00°
Três rodas, IMU desligada, ruído 0%	0,00 (0,0 un.)	0,00°
Três rodas, IMU desligada, ruído 50%	0,24 (1,7 un.)	0,14°
Completa, só ruído da IMU (50%)	2,01 (14,0 un.)	1,73°
Pinpoint simulado, ruído 0%	0,00 (0,0 un.)	0,00°

Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

Os resultados do modo FTC são idênticos aos do modo Sandbox, o que confirma as equações (6) e (7) e as conversões entre os modos. O modo de três rodas também atinge drift nulo sem ruído; com ruído, o heading calculado pelos encoders acumula 0,14° de erro. O caso mais instrutivo é o do ruído apenas na IMU: um erro de heading de 1,73° gerou 2,01 polegadas de drift, mais do que o ruído máximo dos encoders. Isso mostra que a qualidade do heading é a variável mais crítica da odometria.

8.2 DEAD WHEELS E ENCODERS DE TRAÇÃO NOS DIFERENTES CHASSIS

Para comparar as fontes de odometria, a mesma rota foi percorrida nos modos Tank, Mecanum e Swerve com duas versões do código: (A) a odometria completa por dead wheels e IMU, e (B) uma odometria que usa apenas os encoders das rodas de tração (`driveEncoders`) e a IMU, sem dead wheels. Nessa versão, o deslocamento do chassi é obtido pela cinemática direta da Seção 4. No modo Tank, os trechos de deslocamento lateral da rota não movem o robô.

Tabela 10 – Drift final em polegadas: dead wheels (A) e encoders de tração (B)

Chassi	A – 0%	A – 50%	A – 100%	B – 0%	B – 50%	B – 100%
Tank	0,00	0,09	0,19	0,00	0,44	0,88
Mecanum	0,00	0,11	0,23	9,29	10,55	11,54
Swerve	0,00	0,07	0,13	0,00	1,72	3,48

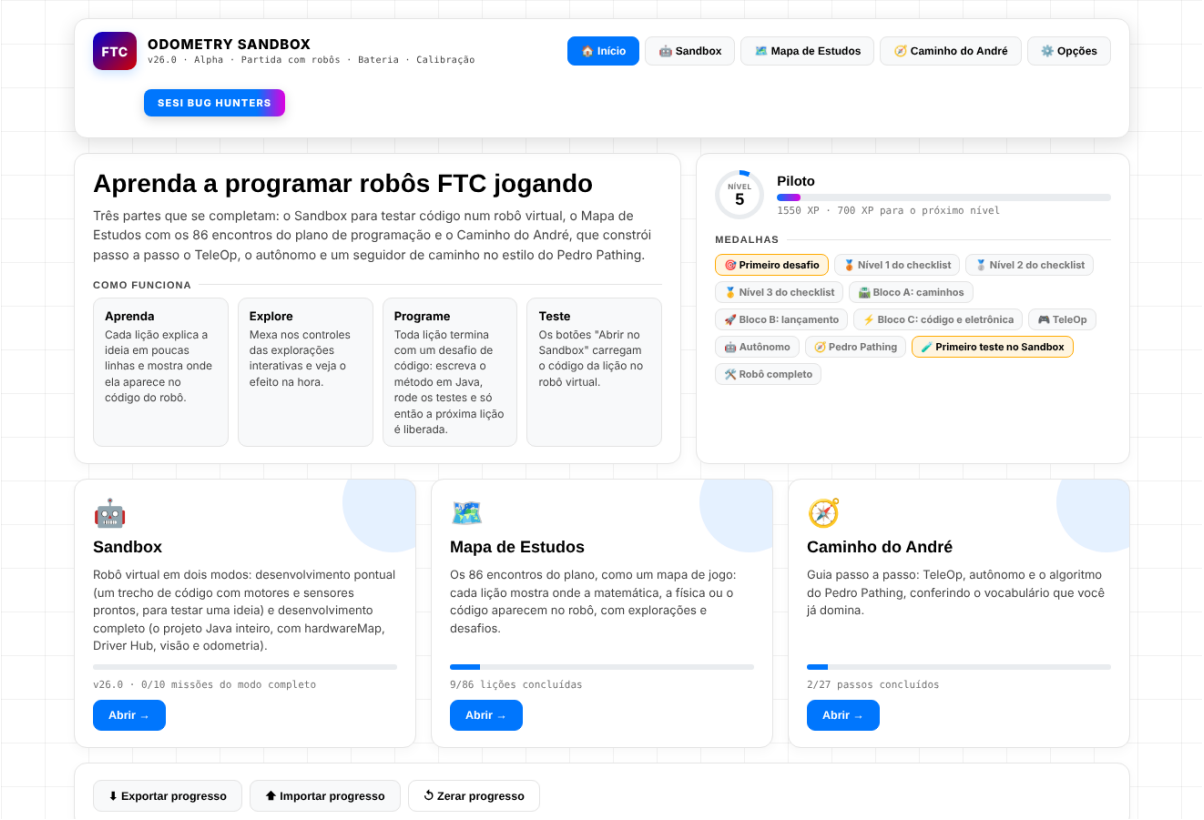
Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

As dead wheels mantiveram o drift em no máximo 0,23 polegada em todos os chassis e níveis de ruído. Os encoders de tração são exatos sem ruído nos modos Tank e Swerve, mas pioram com o deslizamento simulado (até 3,48 polegadas no Swerve), porque medem a rotação da roda, e não o movimento real do chassi. No Mecanum, o erro aparece mesmo sem ruído (9,29 polegadas): a perda de 10% no deslocamento lateral faz os encoders indicarem um strafe maior do que o real. Esse resultado justifica o uso de dead wheels (ou de um Pinpoint) em robôs com tração Mecanum, que é a configuração mais comum na FTC.

9. TELA INICIAL E SISTEMA DE PROGRESSO

A tela inicial (Figura 4) apresenta as partes disponíveis nesta versão (Início, Sandbox, Mapa de Estudos e Caminho do André), o nível e as medalhas do estudante e as ferramentas de progresso. Ela é a primeira tela exibida ao abrir o arquivo.

Figura 4 – Tela inicial do aplicativo



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

9.1 ORGANIZAÇÃO EM TELAS E NAVEGAÇÃO

O cabeçalho ganhou botões de navegação (Início, Sandbox, Mapa de Estudos e Caminho do André). Cada tela tem um endereço próprio no formato `#/tela/item` — por exemplo, `#/roadmap/35` abre o encontro 35 do Mapa de Estudos —, o que permite usar o botão Voltar do navegador e guardar atalhos. A última tela aberta fica registrada, e o botão "Continuar" da tela inicial leva o estudante de volta a ela.

Ao sair da tela do Sandbox, o laço de simulação é pausado: a função `gameLoop` verifica se a tela está visível antes de calcular, o que evita consumo de processamento em segundo plano e impede que o robô se mova enquanto o estudante lê outra parte. Os nomes das telas foram traduzidos para os 14 idiomas da interface.

Uma função de ligação, `openInSandbox`, permite que as outras telas abram o Sandbox já configurado: ela carrega um código no editor, escolhe o chassi, o modo de coordenadas e o nível de ruído, reinicia e inicia a simulação e mostra, acima da arena, de qual lição o código veio.

9.2 PROGRESSO, NÍVEIS E MEDALHAS

O sistema de pontos de experiência (XP) usa uma progressão quadrática, em que cada nível pede um pouco mais que o anterior. O XP total necessário para chegar ao nível *n* é:

$$\mathrm{XP}(\mathrm{n}) = 75 \cdot (\mathrm{n} - 1) \cdot \mathrm{n} \tag{19}$$

Assim, o nível 2 pede 150 XP, o nível 4 pede 900 XP e o nível 10 pede 6 750 XP. Cada nível tem um título, de "Novato" a "Pedro Pathfinder". O progresso fica salvo no próprio navegador (`localStorage`), sem servidor e sem cadastro; ele pode ser exportado num arquivo JSON, importado em outro computador e zerado.

As fontes de XP estão na Tabela 11. Cada conquista é registrada uma única vez, de modo que refazer um desafio já resolvido não soma pontos.

Tabela 11 – Fontes de XP

Conquista	XP
Desafio resolvido	20
Lição concluída	60
Revisão ou checkpoint concluído	150
Projeto concluído	200
Termo de vocabulário acertado (Caminho do André)	15
Passo do Caminho do André concluído	100

Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

As medalhas marcam marcos do percurso: o primeiro desafio, o primeiro teste no Sandbox, cada fase do Mapa de Estudos e cada capítulo do Caminho do André.

10. MAPA DE ESTUDOS INTERATIVO

O Mapa de Estudos transforma em jogo os dois planos de estudo do programador novo de FTC usados pela equipe: o plano de 60 encontros, que cobre os níveis 1 a 3 do checklist, e a sua extensão, o nível 4 (encontros 61 a 86). Cada encontro de três horas do plano virou uma lição interativa, em português e em inglês. As lições mantêm o plano original (aprender, praticar e aplicar no robô) e acrescentam o que o papel não permite: explorações interativas, desafios corrigidos na hora e códigos que abrem direto no Sandbox.

10.1 FASES, ÁREAS E MAPA

Os 86 encontros estão divididos em seis fases (Tabela 12). Cada encontro pertence a uma de cinco áreas, identificadas por cor e ícone no mapa: Matemática (27), Física (17), Código (23), Projeto (7) e Revisão (12, entre revisões e checkpoints).

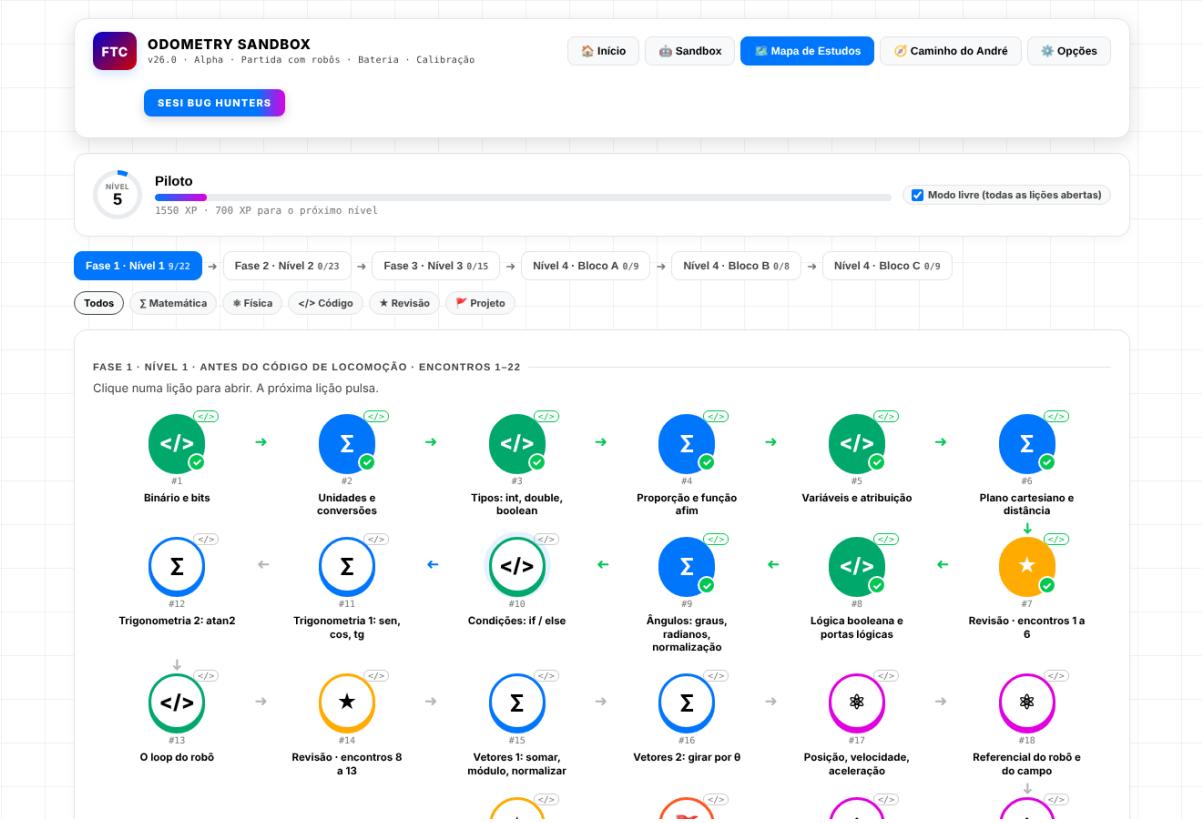
Tabela 12 – Fases do Mapa de Estudos

Fase	Encontros	Tema	Mat.	Fís.	Cód.	Proj./ Rev.
Fase 1 · Nível 1	1–22	Antes do código de locomoção	8	4	6	1 / 3
Fase 2 · Nível 2	23–45	Autônomo e odometria	8	4	7	1 / 3
Fase 3 · Nível 3	46–60	Ajuste	4	3	3	2 / 3
Nível 4 · Bloco A	61–69	Matemática para seguir caminhos	7	0	0	1 / 1
Nível 4 · Bloco B	70–77	Energia, quantidade de movimento e lançamento	0	6	0	1 / 1
Nível 4 · Bloco C	78–86	Código mais fundo e eletrônica digital	0	0	7	1 / 1

Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

O mapa de cada fase desenha os encontros como um caminho em zigue-zague, como num jogo (Figura 5). Encontros concluídos recebem um sinal de visto, encontros bloqueados aparecem esmaecidos e o próximo encontro a fazer pulsa. Setas ligam cada encontro ao seguinte e ficam verdes no trecho já percorrido; no fim de cada fase, um cartão indica o próximo destino e fica em destaque quando a fase é concluída. Ao concluir uma lição, o próprio painel mostra um botão para a lição seguinte ou, no último encontro, para a próxima fase. Filtros por área destacam, por exemplo, só as lições de física.

Figura 5 – Mapa de Estudos: Fase 1, com o jogador no nível 4



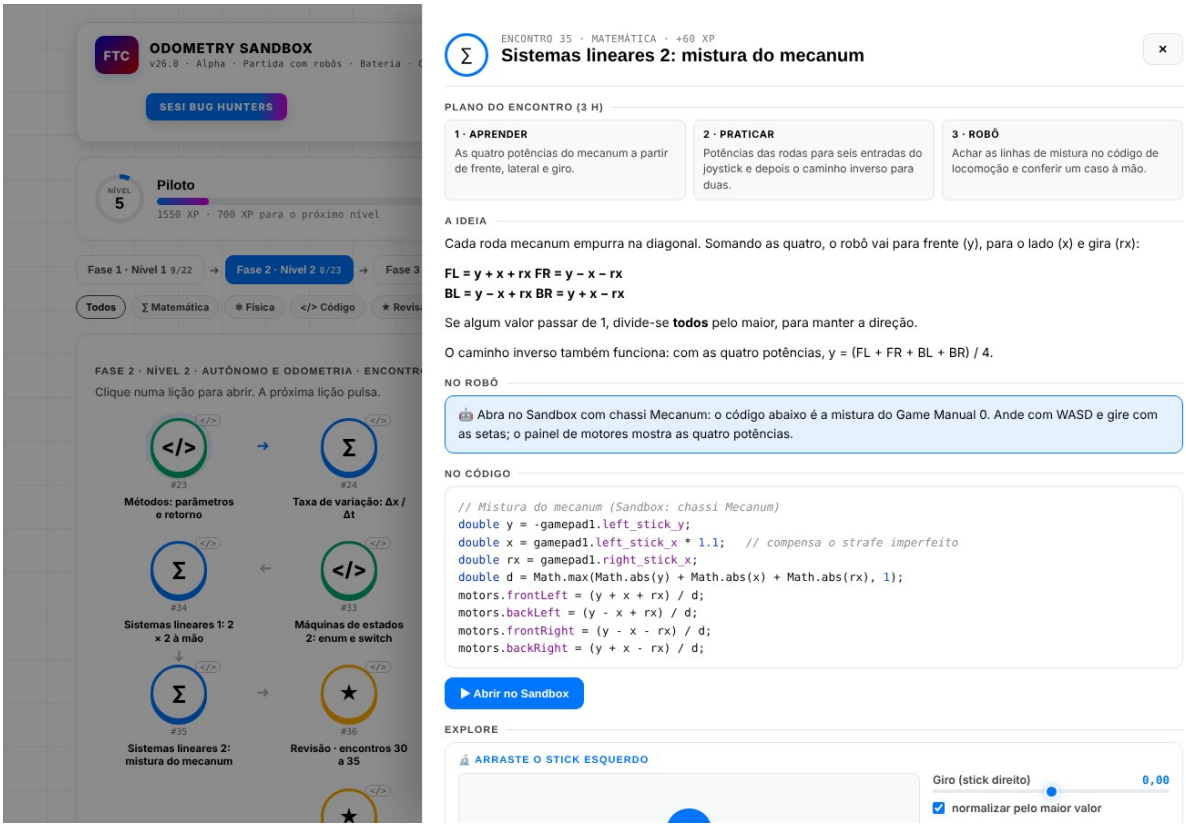
Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

10.2 ESTRUTURA DE UMA LIÇÃO

Ao clicar num encontro, abre-se um painel lateral com a lição (Figura 6), organizada sempre na mesma ordem:

- **Plano do encontro (3 h):** os três momentos do plano original — aprender, praticar e aplicar no robô.
- **A ideia:** a explicação em poucos parágrafos, com o exemplo resolvido e a equação em destaque.
- **No robô:** onde o conteúdo aparece no código ou no comportamento do robô; é a parte que justifica estudar o assunto.
- **No código:** o trecho de Java correspondente. Em 8 encontros (21, 23, 27, 33, 35, 44, 49, 68), um botão "Abrir no Sandbox" carrega o código no simulador com o chassi e o modo de coordenadas certos.
- **Explore:** uma exploração interativa (seções seguintes), em que o estudante mexe em controles e vê o efeito.
- **Desafios** e **Pronto quando:** os exercícios corrigidos automaticamente e o critério de conclusão do plano original.

Figura 6 – Painel de uma lição: encontro 35, mistura do mecanum



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

10.3 DESAFIOS E EXPLORAÇÕES INTERATIVAS

As lições têm 199 desafios próprios, de cinco tipos (Tabela 13). Depois de três tentativas erradas, aparece o botão "Mostrar resposta"; ao acertar, o desafio mostra a explicação do resultado.

Tabela 13 – Tipos de desafio do Mapa de Estudos

Tipo	Quantidade	Como é corrigido
Cálculo numérico	97	Compara o número digitado com a resposta, com tolerância definida em cada desafio (aceita vírgula ou ponto).
Múltipla escolha	70	Compara a opção escolhida com a correta; a ordem das opções é embaralhada de forma fixa por desafio.
Completar o código	14	Cada lacuna do trecho de Java é uma lista de opções; todas precisam estar certas.
Ordenar linhas	11	O estudante move as linhas com setas até a ordem correta (de um algoritmo, de um OpMode, de um ajuste).
Missão interativa	7	Uma exploração com meta: o desafio é resolvido quando a própria exploração detecta que a meta foi atingida.

Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

Há 50 explorações interativas diferentes no Mapa, desenhadas em SVG e sem bibliotecas externas: conversores (bits, ticks e centímetros), gráficos (posição–tempo, reta velocidade–torque, frenagem $v^2 = 2ad$), modelos geométricos (rotação de vetores, atan2, ponto e reta, círculo de *look-ahead*, curvas de Bézier com o algoritmo de de Casteljau) e simulações (PID de um elevador, filtro de média ponderada, lançamento oblíquo, colisões, threads disputando um contador, somador de 4 bits e encoder de quadratura). As 7 missões pedem, por exemplo, ligar os bits até formar 13, fazer o mecanum andar só para o lado, ajustar o PID do elevador sem oscilação (passar no máximo 2,5 cm do alvo, estabilizar em menos de 2 s e terminar a menos de 1 cm) e acertar um alvo com um lançamento oblíquo.

10.4 REGRAS DE PROGRESSÃO

As regras de progressão seguem o plano original:

- **Ordem:** um encontro só é liberado depois que o anterior foi concluído. O nível 4 começa depois do encontro 60, e os seus três blocos são independentes entre si, como no plano.
- **Conclusão:** uma lição é concluída quando todos os seus desafios estão resolvidos. Ao concluir todos os encontros de uma fase, o estudante ganha a medalha da fase.
- **Revisões e checkpoints:** em vez de conteúdo novo, montam uma prova com um desafio de cada encontro anterior (6 nas revisões, 8 nos checkpoints e 10 no encontro final), escolhidos com uma semente fixa, para que a mesma revisão tenha sempre os mesmos desafios. A lista dos encontros revisados mostra o critério "pronto quando" de cada um.
- **Modo livre:** uma opção libera todos os encontros, para uso do professor ou de quem já domina um assunto.

10.5 VALIDAÇÃO AUTOMATIZADA

Um teste automatizado (Playwright, com o navegador Chromium sem interface) abre os 86 encontros nos dois idiomas e resolve cada desafio pela própria interface: clica nas opções, digita os números, escolhe as lacunas e reordena as linhas. Em seguida, confere se cada lição foi concluída e se as medalhas das fases foram concedidas, e verifica que nenhuma missão interativa se resolve sozinha ao abrir. Resultado: 86 de 86 lições concluídas nos dois idiomas, todos os desafios não interativos resolvidos pela interface, as 7 missões abertas sem resolução automática e nenhum erro de página. Os códigos dos encontros com botão "Abrir no Sandbox" foram executados no simulador (Tabela 14).

Tabela 14 – Códigos das lições executados no Sandbox

Encontro	Chassi	O que o código faz	Resultado medido
21	Mecanum	Field-centric: gira o vetor do stick por −heading	Após girar o robô, W o afasta do piloto
23	Ideal	Métodos `clamp` e `distance`, distância até um alvo	Telemetria da distância e da potência sugerida
27	Ideal	Detecção de borda no botão A (Espaço)	Dois cliques longos contados como 2
33	Ideal	Autônomo de três estados com `switch`	Estado DONE em (-24,5; 24,9) pol.
35	Mecanum	Mistura do mecanum com normalização	D move o robô só para a direita
44	Mecanum	Waypoints com odometria e heading normalizado	Volta à origem com erro de 0,43 pol.
49	Ideal	Giro com um erro de sinal plantado	O robô foge do alvo e estaciona em 178,5°
68	Mecanum	Seguidor GVF completo (Seção 11)	Para a 0,05 pol. do fim

Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

11. CAMINHO DO ANDRÉ: TELEOP, AUTÔNOMO E PEDRO PATHING

O Caminho do André é um guia passo a passo para construir, com as próprias mãos, as três peças de software de um robô de competição: o TeleOp, o autônomo e um seguidor de caminho no estilo do Pedro Pathing. O capítulo do Pedro Pathing segue o artigo que descreve a biblioteca (SRIPADA; HENDERSON, 2026): em vez de usar a biblioteca como uma caixa-preta, o estudante escreve o algoritmo no Sandbox, um termo por vez, e só depois vê como cada termo aparece na API real.

11.1 CAPÍTULOS E ESTRUTURA DOS PASSOS

São 27 passos em três capítulos independentes (Tabela 15). Dentro de um capítulo, os passos são liberados em ordem.

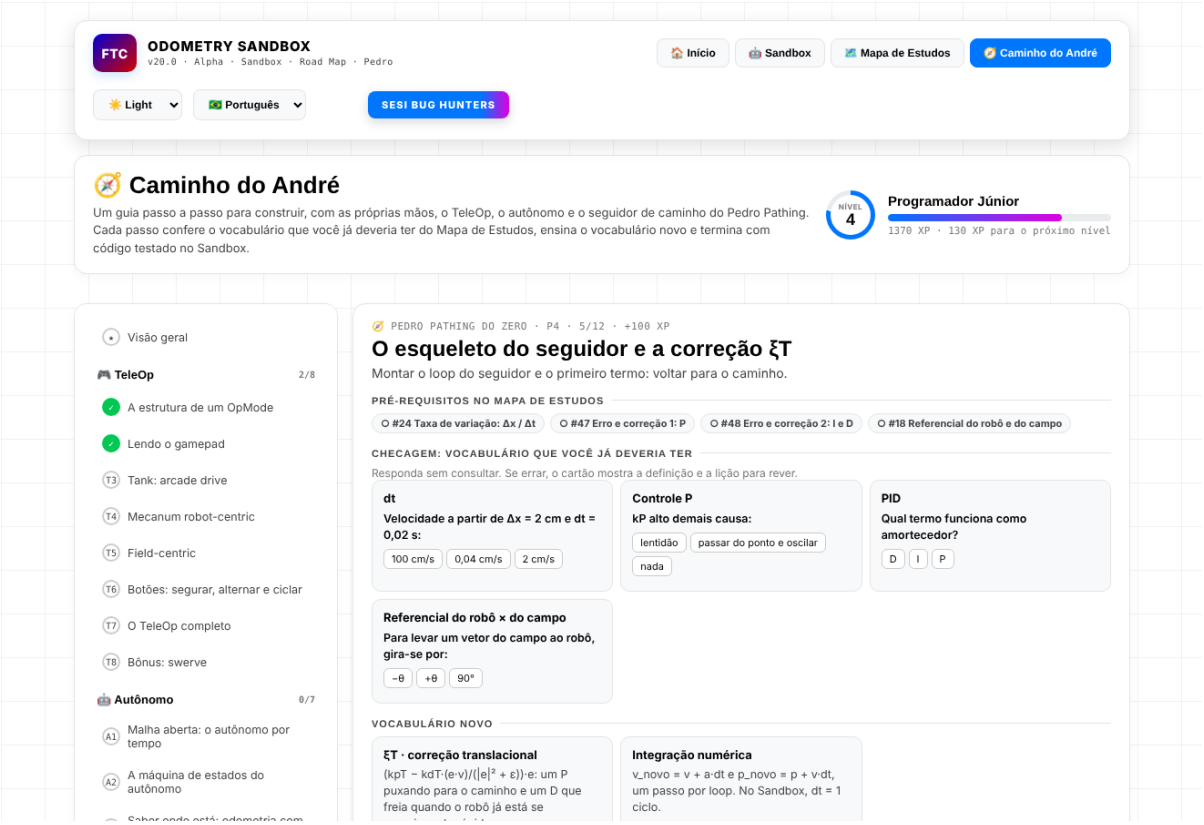
Tabela 15 – Capítulos do Caminho do André

Capítulo	Passos	Conteúdo
TeleOp	T1–T8	Estrutura do OpMode; leitura do gamepad (sinal do eixo Y, zona morta, curva quadrática); arcade drive no tank; mecanum robot-centric e field-centric com reinício do yaw; botões de segurar, alternar e ciclar; o TeleOp completo com modo lento; e, como bônus, a cinemática do swerve com a otimização de 90°.
Autônomo	A1–A7	Malha aberta contra malha fechada; máquina de estados sem `sleep`; odometria com o Pinpoint; ir até um ponto; girar até um ângulo com kS e acomodação; rota de waypoints; timeouts e o relógio de 30 s, com a versão do mesmo autônomo na API do Pedro Pathing.
Pedro Pathing do zero	P0–P11	Checagem de vocabulário; a curva de Bézier; tangente, normal e curvatura; o ponto mais próximo pelo método de Newton; os termos ξT, ξD e ξC; o heading; o seguidor completo; por que ele converge; o ajuste (tuning) na ordem do Pedro; e a API real.

Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

Cada passo tem a mesma estrutura (Figura 7): os pré-requisitos no Mapa de Estudos, com um atalho para cada encontro; a checagem do vocabulário que o estudante já deveria ter; o vocabulário novo; a ideia, com as equações; uma exploração interativa; o código para o Sandbox, testado; o código equivalente para o robô real (FTC SDK ou Pedro Pathing); os desafios; e o critério de conclusão. Um passo só é concluído quando todas as checagens de vocabulário e todos os desafios estão certos, valendo 100 XP; concluir um capítulo dá a sua medalha.

Figura 7 – Um passo do Caminho do André (P4), com pré-requisitos e checagem de vocabulário



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

11.2 CHECAGEM DE VOCABULÁRIO

O vocabulário tem 74 termos, cada um com definição e uma pergunta curta de checagem. 34 deles foram ensinados no Mapa de Estudos (por exemplo, "normalização de ângulo", encontro 9, ou "$v^2 = 2 \cdot a \cdot d$", encontro 37) e aparecem como **checagem**: a definição fica escondida até o estudante responder, e um erro mostra a definição e o atalho para a lição que ensina o termo. Os outros 40 são **vocabulário novo** (por exemplo, "zona morta", "ponto mais próximo ψ(p)", "método de Newton" e "vizinhança tubular"): a definição aparece antes da pergunta. Ao todo, os passos fazem 89 checagens de termos já vistos e 40 apresentações de termos novos, e cada acerto vale 15 XP. O passo P0 funciona como uma prova de entrada do capítulo do Pedro Pathing, com 12 termos do Mapa (vetores, rotação, referenciais, frenagem, controle P e PID, curvas paramétricas, Bézier, derivada, tangente, curvatura e dt).

11.3 TELEOP E AUTÔNOMO NO SANDBOX

Os capítulos de TeleOp e de autônomo partem do Sandbox e terminam no robô real. Cada código foi executado no simulador com um roteiro de teclas automatizado (por exemplo, girar 90° e depois empurrar W, no field-centric) e uma verificação do resultado; todos passaram nos dois idiomas. Alguns resultados ilustram as ideias dos passos:

- **A1 (malha aberta):** 0,4 pol./ciclo por 90 ciclos levam o robô a 36,0 pol. com a velocidade padrão; com o controle de velocidade reduzido, como uma bateria fraca, o mesmo código anda menos e não percebe.
- **A2 (máquina de estados):** o autônomo passa por FORWARD, TURN, STRAFE e WAIT (40 ciclos, sem `sleep`) até DONE, com a troca de estado registrada na telemetria.
- **A4 e A5 (ir até um ponto e girar):** o robô para a 0,30 pol. do alvo (24; 36), e o giro termina dentro de 1° depois de 10 ciclos seguidos dentro da tolerância.
- **A7 (timeouts):** um waypoint fora do campo prende o robô na parede; o timeout de 300 ciclos libera o estado, e o robô volta à origem. O histórico da telemetria registra "ok TIMEOUT ok".

Para esses códigos, o transpilador ganhou dois equivalentes do Java: `String.equals`, que permite máquinas de estados com `String` (o Sandbox não aceita `enum`), e `Math.signum`, usado no termo kS do giro.

11.4 O SEGUIDOR GVF DE SEGUNDA ORDEM

O artigo modela o seguidor como um campo vetorial guia (*guiding vector field*, GVF) de segunda ordem: em vez de dizer ao robô com que velocidade andar, o campo dá a **aceleração** desejada em cada ponto, o que respeita a inércia do robô. O caminho é uma curva paramétrica ϕ(t), t ∈ [0, 1] — no código, uma Bézier cúbica. Tudo começa pelo ponto do caminho mais próximo do robô na posição *p*:

$$\psi(\mathrm{p}) = \operatorname{argmin}_{\mathrm{t}} \, |\phi(\mathrm{t}) - \mathrm{p}| \tag{20}$$

No mínimo, o erro é perpendicular à tangente, isto é, a derivada de D(t) = ½|ϕ(t) − p|² se anula. O código acha esse zero com o método de Newton, começando do *t* do ciclo anterior (*warm start*) e limitando *t* ao intervalo [0, 1]:

$$\mathrm{D}'(\mathrm{t}) = (\phi(\mathrm{t}) - \mathrm{p}) \cdot \phi'(\mathrm{t}) \qquad \mathrm{D}''(\mathrm{t}) = |\phi'(\mathrm{t})|^2 + (\phi(\mathrm{t}) - \mathrm{p}) \cdot \phi''(\mathrm{t}) \tag{21}$$

$$\mathrm{t} \leftarrow \mathrm{t} - \mathrm{D}'(\mathrm{t}) \,/\, \mathrm{D}''(\mathrm{t}) \tag{22}$$

Com e = ϕ(ψ(p)) − p, a velocidade *v* do robô, o tangente unitário T, a normal N e a curvatura κ no ponto mais próximo, a parte translacional do campo é a soma de três termos:

$$\xi_{\mathrm{T}} = (\mathrm{k_{pT}} - \mathrm{k_{dT}} \cdot (\mathrm{e} \cdot \mathrm{v}) \,/\, (|\mathrm{e}|^2 + \varepsilon)) \cdot \mathrm{e} \tag{23}$$

$$\xi_{\mathrm{C}} = \mathrm{k_c} \cdot (\mathrm{v} \cdot \mathrm{T})^2 \cdot \kappa \cdot \mathrm{N} \tag{24}$$

$$\xi_{\mathrm{D}} = \mathrm{k_{pD}} \cdot (\min(\mathrm{V_{max}}, \sqrt{(2 \cdot \mathrm{a} \cdot \mathrm{d})}) - \mathrm{v} \cdot \mathrm{T}) \cdot \mathrm{T} \tag{25}$$

O termo translacional ξ_T é um controle proporcional que puxa o robô para o caminho, com um amortecimento que freia quando o robô já se aproxima depressa (o ε evita a divisão por zero sobre o caminho). O termo centrípeto ξ_C antecipa a força que a curva pede ($a_c = v^2\kappa$, encontro 67). O termo de tração ξ_D acelera ao longo do caminho até a maior velocidade que ainda permite parar no fim, em que *d* é o comprimento de arco restante — a fórmula $v^2 = 2ad$ do encontro 37, usada ao contrário. A cada ciclo, a velocidade nova é a antiga mais a soma dos termos, limitada em tamanho (saturação), e é convertida para o referencial do robô por R(−θ):

$$\mathrm{v_{novo}} = \mathrm{sat}(\mathrm{v} + \xi_{\mathrm{T}} + \xi_{\mathrm{C}} + \xi_{\mathrm{D}}, \, \mathrm{V_{max}}) \tag{26}$$

O artigo ainda define um campo de heading (um controle PD sobre o erro de rotação, mais um termo que acompanha a aceleração da interpolação de heading) e, longe do caminho, um segundo campo mais simples, misturado ao primeiro por uma função suave χ que vale 1 dentro de uma vizinhança tubular do caminho e 0 fora dela: $\xi = \chi \cdot \xi_1 + (1 - \chi) \cdot \xi_2$. A convergência é provada com uma função de Lyapunov. O código do Sandbox usa o campo interno ξ_1 e um heading apenas proporcional, simplificações discutidas na Seção 18.

O código é construído em passos cumulativos (P1 a P8): desenhar a Bézier; desenhar T, N e o círculo osculador; achar ψ(p) dirigindo o robô com o teclado; montar o esqueleto do seguidor com ξ_T; acrescentar ξ_D, ξ_C e o heading. O Código 4 mostra o núcleo do passo P8, que é o mesmo do projeto do encontro 68.

Código 4 – Núcleo do seguidor GVF (passo P8, modo FTC)

```
// 3) Correção translacional
double ex = B[0] - px, ey = B[1] - py;
double e2 = ex * ex + ey * ey;
double gainT = KPT - KDT * (ex * vx + ey * vy) / (e2 + EPS);
double aTx = gainT * ex, aTy = gainT * ey;
// 4) Força centrípeta
double kappa = (D1[0] * D2[1] - D1[1] * D2[0]) / (sp * sp * sp);
double vt = vx * Tx + vy * Ty;
double aCx = KC * vt * vt * kappa * Nx, aCy = KC * vt * vt * kappa * Ny;
// 5) Tração tangencial: v = sqrt(2 a d)
double vdes = Math.min(VMAX, Math.sqrt(2 * ADEC * Math.max(rest, 0)));
double aDx = KPD * (vdes - vt) * Tx, aDy = KPD * (vdes - vt) * Ty;
// 6) Soma, integração e saturação
double nvx = vx + aTx + aCx + aDx, nvy = vy + aTy + aCy + aDy;
double nv = Math.hypot(nvx, nvy);
if (nv > VMAX) { nvx *= VMAX / nv; nvy *= VMAX / nv; }
// 7) Do campo para o robô e heading pela tangente
commands.forward = nvx * Math.cos(theta) + nvy * Math.sin(theta);
commands.left = -nvx * Math.sin(theta) + nvy * Math.cos(theta);
```

Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

Para desenhar o caminho e os vetores na arena, o Sandbox ganhou uma API de desenho em coordenadas do modo ativo (Tabela 16), refeita a cada ciclo. O transpilador passou a aceitar também `new double[n]`, usado nos vetores de pontos.

Tabela 16 – API de desenho na arena

Chamada	Desenha
`draw.path(xs, ys, cor)`	Uma linha passando pelos pontos (xs[i], ys[i]) — o caminho
`draw.polyline(pontos, cor)`	O mesmo, com uma lista de pontos
`draw.line(x1, y1, x2, y2, cor)`	Um segmento (por exemplo, do robô ao ponto mais próximo)
`draw.arrow(x1, y1, x2, y2, cor)`	Uma seta (vetores T, N e ξ)
`draw.circle(x, y, r, cor)`	Uma circunferência (círculo osculador, alvos, garra)
`draw.point(x, y, cor)`	Um ponto
`draw.text(x, y, texto, cor)`	Um rótulo

Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

11.5 RESULTADOS DO SEGUIDOR NO SIMULADOR

Os passos P5 (ξ_T + ξ_D), P6 (+ ξ_C) e P7 (+ heading) foram executados no Sandbox, nos chassis Ideal, Mecanum e Swerve, sem ruído e com 50% de ruído, com a mesma sequência pseudoaleatória. O erro de trajetória foi medido contra a curva real, amostrada em 2 001 pontos (Tabela 17).

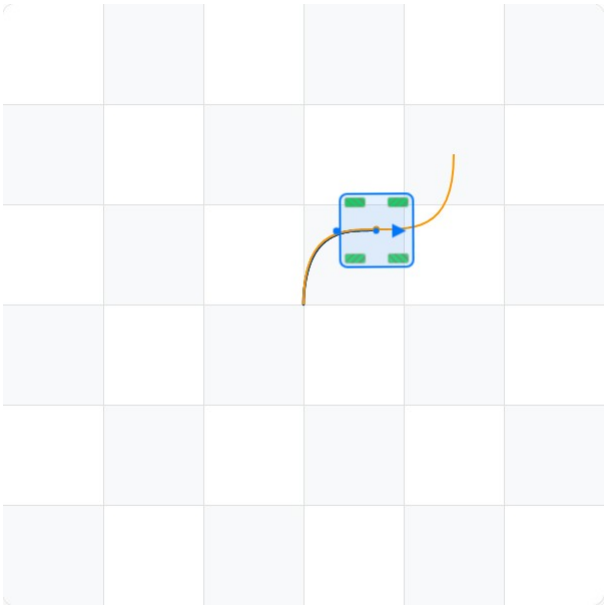
Tabela 17 – Seguidor GVF: erro máximo de trajetória e erro final, em polegadas

Passo	Ruído	Ideal máx.	Mecanum máx.	Swerve máx.	Mecanum final	Mecanum ciclos
P5 (ξ_T + ξ_D)	0%	1,35	1,71	1,35	0,11	172
P5 (ξ_T + ξ_D)	50%	1,49	1,90	1,57	0,12	191
P6 (+ ξ_C)	0%	0,05	1,03	0,05	0,09	169
P6 (+ ξ_C)	50%	0,71	1,46	0,42	0,18	189
P7 (+ heading)	0%	1,34	0,25	0,61	0,05	203
P7 (+ heading)	50%	0,97	0,28	0,74	0,20	207

Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

O termo centrípeto foi o que mais reduziu o erro: sem ruído, o erro máximo caiu de 1,35 para 0,05 pol. no chassi Ideal e de 1,71 para 1,03 pol. no Mecanum, o que confirma, no simulador, a função de antecipação descrita no artigo. O controle de heading teve efeitos opostos conforme o chassi: no Mecanum, ele reduziu o erro máximo para 0,25 pol., porque, olhando para a tangente, o robô anda quase só para a frente e escapa da perda de 10% do strafe; no Ideal e no Swerve, o erro aumentou (1,34 e 0,61 pol.), porque o robô gira durante o ciclo enquanto a conversão para o seu referencial usa o heading do início do ciclo. Em todos os casos o robô parou a menos de 0,6 pol. do fim do caminho (Figura 8).

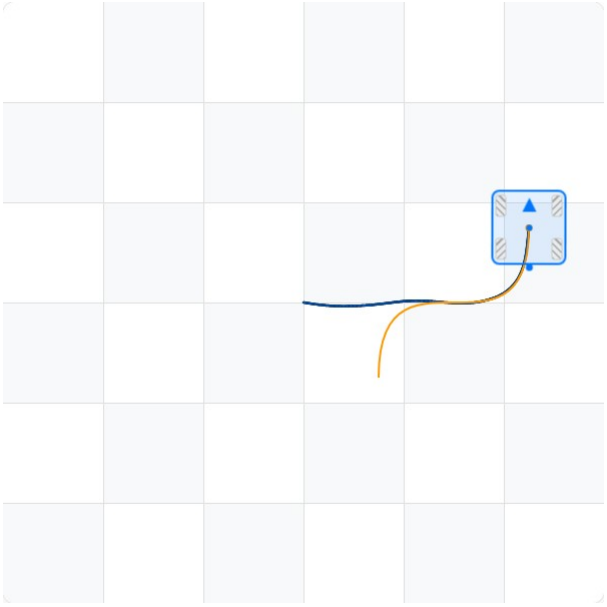
Figura 8 – Seguidor GVF no meio da curva (passo P8): caminho em laranja e trajetória real em azul



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

No passo P9, o caminho foi deslocado para que o robô comece a cerca de 25 pol. dele (Figura 9). O termo translacional domina no início e leva o robô ao caminho; depois de entrar a 1,5 pol. da curva, o erro máximo ficou entre 1,38 e 1,84 pol. nos três chassis, e o robô terminou a menos de 0,6 pol. do fim.

Figura 9 – Passo P9: o robô parte longe do caminho e converge para ele



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

11.6 DO SANDBOX À API DO PEDRO PATHING

O último passo liga cada parte do código do Sandbox à API do Pedro Pathing (Tabela 18). A diferença prática mais importante é o sistema de coordenadas: o campo

do Pedro vai de 0 a 144 polegadas, com origem num canto, enquanto o modo FTC do Sandbox tem origem no centro; os ângulos das poses do Pedro são em radianos.

Tabela 18 – Do código do Sandbox à API do Pedro Pathing

No Sandbox (Caminho do André)	No Pedro Pathing
`PX`, `PY` e `bez(t)` (P1)	`new BezierCurve(p0, p1, p2, p3)`, `new BezierLine(a, b)`
Constantes KPT, KDT, KC, KPD, ADEC, KPH, VMAX (P10)	`Constants.createFollower(hardwareMap)`, com os valores do tuning
Heading pela tangente (P7)	`setTangentHeadingInterpolation()`, `setLinearHeadingInterpolation(θ0, θ1)`, `setConstantHeadingInterpolation(θ)`
O código do ciclo inteiro: $\psi(p)$, $\xi_T + \xi_C + \xi_D$, heading, comandos	`follower.update()`, chamado uma vez por loop
Iniciar o caminho / condição de saída do estado	`follower.followPath(chain)` / `!follower.isBusy()`
Pose inicial e pose atual	`follower.setStartingPose(new Pose(x, y, h))` / `follower.getPose()`

Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

12. DESAFIOS DE CÓDIGO PARA AVANÇAR

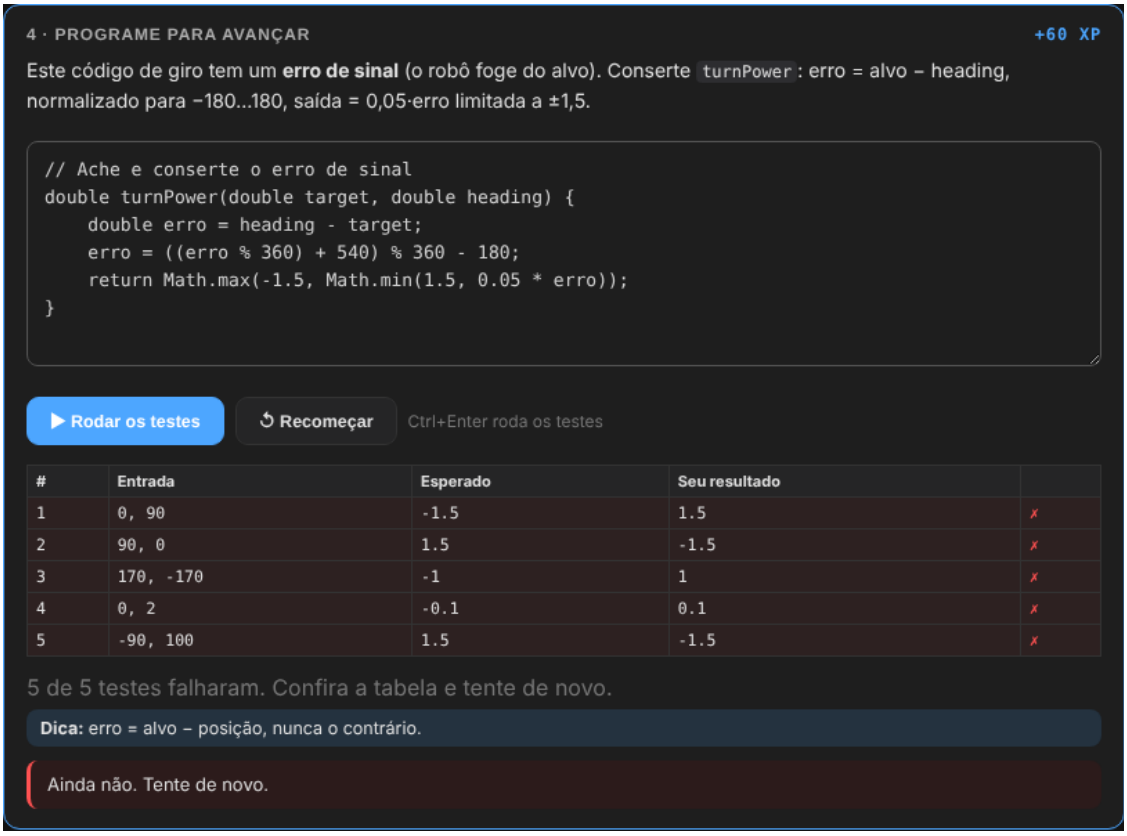
Até a versão 20, os desafios das lições conferiam números, opções e a ordem de linhas, mas o estudante podia concluir uma lição sem escrever nenhuma linha de código. A partir da versão 21, toda lição do Mapa de Estudos, toda revisão e todo passo do Caminho do André terminam com um desafio "Programe para avançar": o estudante escreve um método em Java que resolve um problema do robô usando o conteúdo da própria lição, e a lição só é concluída — e a seguinte só é liberada — quando o método passa em todos os testes. São 101 desafios próprios (74 nas lições e 27 nos passos), com 452 casos de teste, além de um desafio de código sorteado em cada uma das 12 revisões.

12.1 COMO É UM DESAFIO DE CÓDIGO

Cada desafio traz um enunciado curto, a assinatura do método e um código inicial que compila mas ainda não resolve o problema (Figura 10). Em alguns encontros, o código inicial já contém um erro realista para ser encontrado — no encontro 49, por exemplo, o erro do controle de giro está calculado ao contrário, e o robô foge do alvo. Ao rodar os testes, o aplicativo mostra uma tabela com a entrada de cada teste, o resultado esperado e o resultado do estudante; depois do primeiro erro aparece uma dica e, em alguns desafios, depois de três, uma segunda dica mais direta. Diferentemente dos outros desafios, não existe o botão "Mostrar resposta": o objetivo é que o próprio estudante chegue ao código — a ajuda mais forte são as dicas

em escada da versão 25 (Seção 12.10), que mostram as peças da solução, mas não montadas.

Figura 10 – Desafio de código do encontro 49, com o código inicial (erro de sinal) e a tabela de testes



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

Os problemas foram escritos para usar o conteúdo de cada encontro em uma situação do robô (Tabela 19).

Tabela 19 – Desafios de código por área e exemplos

Área	Desafios	Exemplos de método pedido
Matemática	27	`fromBinary` (converter um texto binário em número), `ticksToCm` (ticks do encoder em centímetros), rotação de vetores, `atan2` e distância até um alvo.
Física	17	Distância de frenagem por $v^2 = 2ad$, energia, lançamento oblíquo, torque e velocidade de um motor pela reta velocidade–torque.
Código	23	Field-centric (`fieldCentric`), máquina de estados do autônomo, normalização do mecanum, contador de pulsos de um encoder de quadratura (`quadCount`).
Projeto	7	Comandos de navegação até waypoints (`goToCommand`) e diagnóstico de um ajuste de PID a partir das medições (`diagnose`).
Caminho do André	27	Mistura do TeleOp e do módulo de swerve (T1–T8), estados do autônomo (A1–A7), Bézier, ponto mais próximo pelo método de Newton e conversão para as coordenadas do Pedro Pathing (P0–P11).

Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

No encontro 57 (PID do elevador), além dos testes de valores, o método do estudante controla uma simulação do elevador: o desafio só passa se o elevador chegar ao alvo sem ultrapassá-lo demais e sem oscilar.

12.2 CORREÇÃO AUTOMÁTICA E ISOLAMENTO

Na versão 21, a correção usava o mesmo transpilador de Java para JavaScript do Sandbox; desde a versão 24, usa o compilador de Java do modo completo (Seção 12.5). Nos dois casos, o código do estudante e a solução de referência são compilados e executados lado a lado; os valores esperados vêm da própria solução, e não de uma tabela digitada à mão, o que evita respostas esperadas erradas. Números são comparados com tolerância relativa de 0,1 % (ajustável por desafio), textos e valores lógicos são comparados exatamente, e vetores são comparados elemento a elemento.

- **Isolamento:** na versão 21, a execução acontecia num Web Worker, criado a partir do próprio arquivo HTML, que era encerrado se o código não terminasse em 2,5 s. Desde a versão 24, o código roda na própria página, compilado com um relógio: se passar de 2 s — por exemplo, num loop infinito —, a execução para com a mensagem de tempo esgotado, sem travar a página (Seção 12.5).
- **Mensagens de erro:** erros de sintaxe são mostrados antes da execução; um método com nome errado é avisado ("Não encontrei o método…"); exceções aparecem na linha do teste em que ocorreram.
- **Rascunho salvo:** o código digitado é salvo junto com o progresso, e o botão "Recomeçar" volta ao código inicial.
- **Pontuação:** cada desafio de código vale 40 XP, somados aos 20 XP de um desafio comum.

Um teste automatizado confere, nos dois idiomas, que todas as 101 soluções de referência passam nos próprios testes, que nenhum código inicial passa sem ser alterado, que os valores calculados à mão de alguns desafios conferem com a solução e que um loop infinito é interrompido (2,5 s). Outro teste conclui todas as lições e passos pela interface, escrevendo o código de cada desafio no editor e rodando os testes. Resultado: 101 de 101 desafios corretos nos dois idiomas e nenhum erro de página.

12.3 CORREÇÃO DO CÓDIGO INICIAL DO SWERVE

Na versão 20, o código inicial do chassi swerve no editor do Sandbox tinha dois defeitos: ignorava o comando de giro (o robô não girava) e, no modo de coordenadas FTC, calculava o ângulo das rodas com os eixos trocados, de modo que o strafe saía espelhado. Além disso, durante as trocas rápidas de direção, as rodas eram empurradas com força total enquanto ainda estavam esterçando, o que fazia o robô "escorregar" para direções erradas — o defeito ficava mais visível depois de algum tempo dirigindo.

O código inicial foi reescrito para cada modo de coordenadas (Código 5). A velocidade de cada módulo passa a ser a velocidade do robô somada à do giro (ω × r); quando a roda precisaria virar mais de 90°, ela vira para o outro lado e inverte o sentido; e a potência é multiplicada pelo cosseno do erro de esterçamento (compensação do cosseno), para que a roda empurre pouco enquanto ainda não está apontada para a direção certa. A mesma compensação foi acrescentada ao misturador interno e ao passo T8 do Caminho do André.

Código 5 – Módulo de swerve do código inicial (modo FTC), versão 21

```
double[] module(double fx, double ly, double cur) {
    double a = y + rx * ly / R;      // velocidade do robô + giro (ω × r)
    double b = -x - rx * fx / R;
    double speed = Math.hypot(a, b);
    if (speed < 0.05) return new double[]{cur, 0};   // parado: mantém o ângulo
    double angle = Math.toDegrees(Math.atan2(b, a));
    double diff = ((angle - cur) % 360 + 540) % 360 - 180;
    if (Math.abs(diff) > 90) {       // otimização: gira para o outro lado e inverte
        angle = angle + 180; speed = -speed;
        diff = ((angle - cur) % 360 + 540) % 360 - 180;
    }
    speed = speed * Math.cos(Math.toRadians(diff));   // compensação do cosseno
    return new double[]{angle, speed};
}
```

Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

O efeito foi medido no simulador com 8.000 ciclos de comandos aleatórios de teclado no modo FTC, contando as amostras em que o robô andava a mais de 25° da direção pedida, e com comandos isolados de 60 ciclos (Tabela 20).

Tabela 20 – Código inicial do swerve antes e depois da correção (modo FTC)

Medida	Versão 20	Versão 21
Amostras na direção errada (de 1.919)	1.064 (55 %)	0
Strafe para a direita (60 ciclos)	Para a esquerda (−224,5 pol.)	Para a direita (+227,6 pol.)
Seta para a direita (giro, 60 ciclos)	Não gira (1,4°)	Gira (≈191°)
Frente + giro	Só anda para a frente	Faz a curva (100,8°)

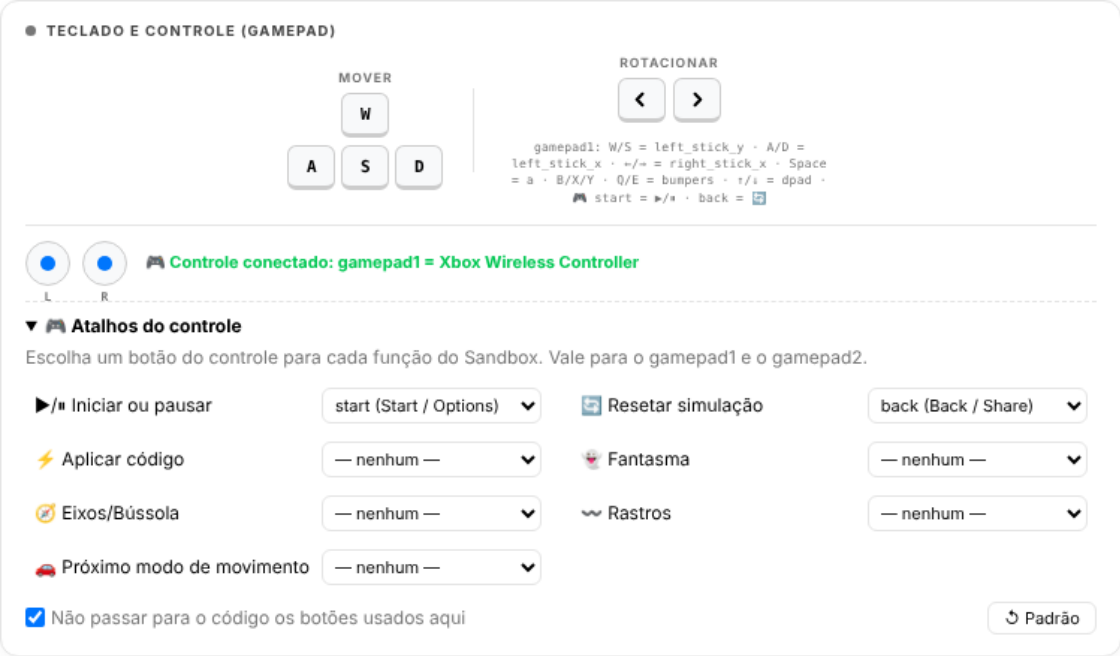
Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

12.4 BOTÕES DO CONTROLE: ATALHOS, BORDAS E VIBRAÇÃO

A revisão 21.1 ampliou o uso do controle. Até então, o código podia ler os botões (`gamepad1.a`, `gamepad1.dpad_up` etc.), mas as funções do próprio Sandbox — iniciar, pausar, resetar — só eram acionadas pelo mouse, o que obrigava o piloto a largar o controle durante os testes. Agora:

- **Atalhos do controle:** um painel na área "Teclado e Controle" (Figura 11) liga cada função do Sandbox a um botão do controle físico (gamepad1 ou gamepad2): iniciar ou pausar, resetar a simulação, aplicar o código, fantasma, eixos, rastros e próximo modo de movimento. Por padrão, `start` inicia e pausa e `back` reseta; os gatilhos também podem ser usados (acionam acima de 50 %). Cada botão tem uma só função, a escolha fica salva no navegador e o botão da função pisca quando é acionado.
- **Conflito com o código:** se o código do editor também lê um botão usado como atalho, o painel avisa. A opção "Não passar para o código os botões usados aqui" (ligada por padrão) faz o código receber `false` nesses botões quando vêm do controle físico; o teclado continua valendo.
- **Métodos de borda do FTC SDK:** o código pode usar `gamepad1.aWasPressed()`, `aWasReleased()`, `dpadUpWasPressed()`, `leftBumperWasPressed()` e os demais, com o mesmo significado do SDK: o método devolve `true` uma única vez para cada clique, mesmo que só seja chamado algumas voltas depois. Também valem os nomes do controle de PlayStation (`cross`, `circle`, `square`, `triangle`, `share`, `options`, `ps`) e `atRest()`.
- **Vibração (rumble):** `gamepad1.rumble(ms)`, `rumble(forte, fraco, ms)`, `rumbleBlips(n)`, `stopRumble()` e `isRumbling()` vibram o controle físico quando o navegador permite (Chrome e Edge) e, em qualquer caso, fazem o indicador do controle tremer na tela.

Figura 11 – Painel "Atalhos do controle", com a configuração padrão (start inicia e pausa; back reseta)



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

Um teste automatizado com um controle simulado confere que `start` alterna a simulação uma única vez por clique (mesmo segurando), que um botão já apertado ao conectar o controle não dispara a função, que `back` devolve o robô ao centro, que os atalhos podem ser trocados, inclusive para um gatilho, e que o código conta corretamente os cliques por `aWasPressed()`, recebe os nomes do PlayStation e aciona a vibração.

12.5 EDITOR, CONSOLE E MENSAGENS DE ERRO DOS DESAFIOS

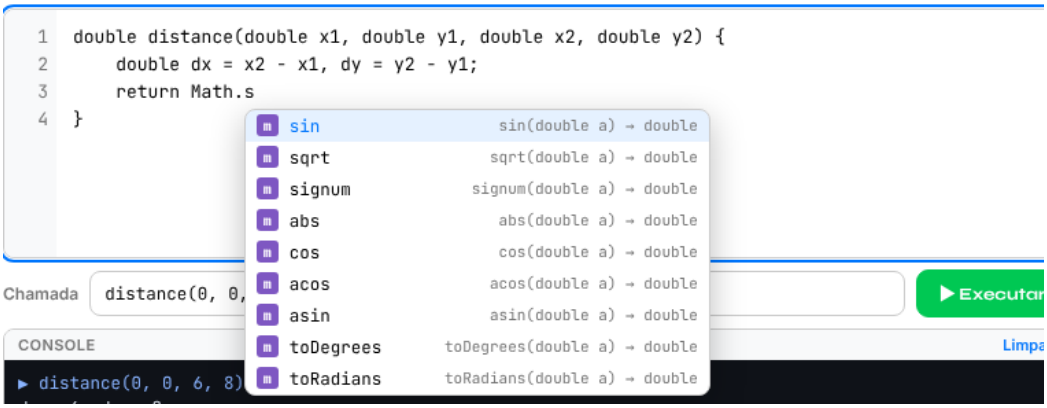
Na versão 24, o editor dos desafios de código das lições e do Caminho do André foi refeito a partir do que os estudantes relataram: não havia sugestões, a edição era pouco prática e não havia como rodar o método e imprimir valores para descobrir o erro — só o botão que rodava os testes. As mudanças:

- **Editor:** números de linha, indentação automática (Enter depois de "{" abre o bloco e já coloca a "}" correspondente), Tab e Shift+Tab indentam e desindentam várias linhas, Ctrl+/ comenta e descomenta, e a linha com erro fica marcada na margem.
- **Sugestões:** o mesmo mecanismo de sugestões do modo completo, num modo próprio para as lições, que oferece só o Java da linguagem — Math, String, Integer, Double, Character, Boolean, System.out e Arrays —, as variáveis e parâmetros do próprio método e modelos como for, if, while e System.out.println, sem as classes do FTC SDK, que não existem nesses

desafios (Figura 12). Ao aceitar um método, a sugestão já coloca os parênteses, e digitar o ")" de fechamento só passa por cima do que foi colocado.

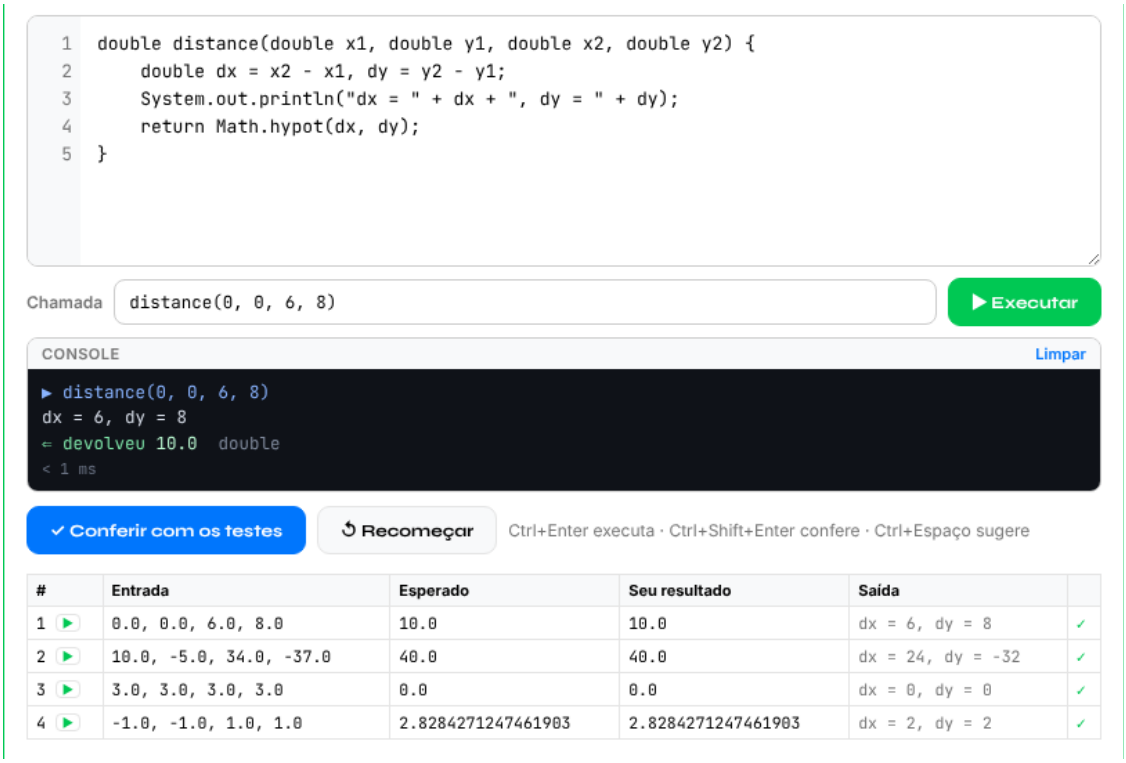
- **Executar e console:** além do botão "Conferir com os testes", há uma caixa "Chamada", que já vem preenchida com os valores do primeiro teste, e o botão "Executar" (Figura 13). Ele roda o método com essa chamada e mostra num console o que foi impresso com System.out.println, print ou printf, o valor devolvido, com o tipo de retorno do método, e o tempo de execução. A chamada pode ser trocada por outros valores, por outro método do estudante ou por comandos, como um for que imprime vários resultados.
- **Saída nos testes:** ao conferir, a tabela de testes ganha a coluna "Saída" quando o método imprime algo, mostrando o que foi impresso em cada teste, e cada linha tem um botão ▶️ que leva aqueles valores para a caixa "Chamada" e executa.

Figura 12 – Sugestões no editor das lições: só o Java da linguagem e as variáveis do próprio método



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

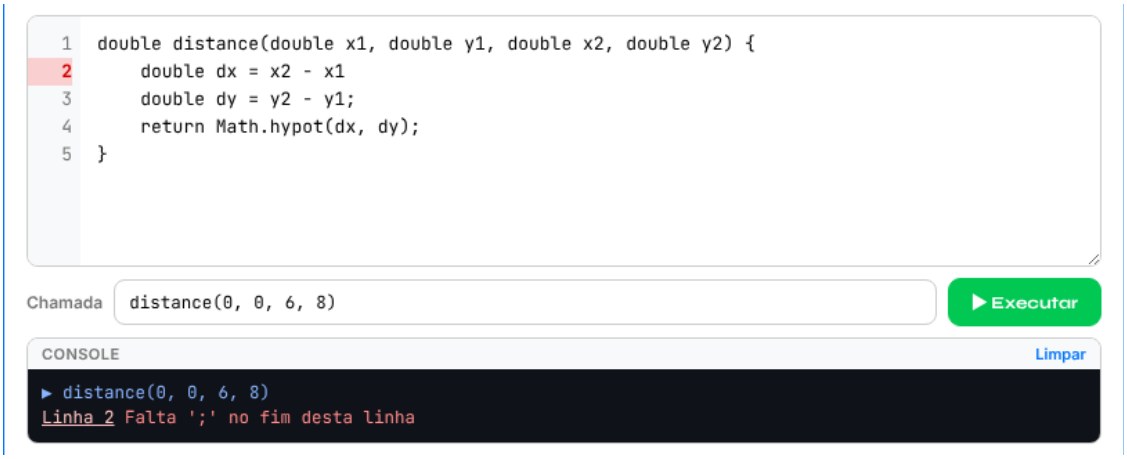
Figura 13 – Desafio de código na versão 24: editor com números de linha, Executar com console e Conferir com a coluna de saída



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

A correção também mudou por dentro. Em vez do transpilador baseado em substituições de texto, os desafios passaram a usar o compilador de Java do modo completo: o método do estudante é colocado dentro de uma classe que ele não vê, e os erros voltam com a linha certa e no estilo das mensagens do Java (Figura 14). Antes de trocar, todas as 101 soluções de referência foram compiladas e executadas pelos dois caminhos, e os resultados de todos os testes foram idênticos. Os laços continuam protegidos: se o código rodar por mais de 2 s, a execução para com a mensagem de tempo esgotado, e uma recursão sem fim vira StackOverflowError. Algumas mensagens foram escritas especialmente para os erros mais comuns de quem está começando (Tabela 21).

Figura 14 – Erro de compilação marcado na margem e explicado no console, com link para a linha



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

Tabela 21 – Erros comuns e a mensagem mostrada no desafio de código

Erro no código	Mensagem
Faltou o ";" no fim da linha	Linha 2: Falta ';' no fim desta linha
Faltou fechar a "}" do método	Linha 1: Falta fechar a chave '{' aberta nesta linha: coloque uma '}' no fim do bloco
Texto entre aspas simples ('oi')	Linha 2: Texto entre aspas simples: 'oi'. Em Java, aspas simples são só para UM caractere (char); para texto (String), use aspas duplas: "oi"
Método sem o tipo de retorno	Linha 1: Falta o tipo do valor devolvido antes de distance: por exemplo double distance(...), int, boolean, String ou void
for (int i = 0, i < 3, i++)	Linha 2: No for, as três partes são separadas por ponto e vírgula: for (int i = 0; i < n; i++)
retrun ou elseif	Linha 2: "retrun" não é uma palavra do Java: você quis dizer return? — Em Java é "else if", com espaço
Variável não declarada ou com maiúscula errada (dX)	Símbolo não encontrado: dX — não existe variável ou método com esse nome aqui (foi declarado? confira as maiúsculas) (na linha 3)
Usar um texto null	NullPointerException: tentou usar .length de um objeto null (a variável recebeu um valor?) (na linha 3)
Nome do método diferente do pedido	Não encontrei o método distance(…) — o seu código tem distancia(…); o nome precisa ser exatamente distance

Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

Os testes automatizados conferem as 101 soluções e os códigos iniciais pelo novo caminho, nos dois idiomas; digitam um método inteiro pelo teclado, usando as sugestões; executam com a chamada padrão, com comandos e com erros de compilação e de execução, conferindo a linha marcada; conferem a coluna de saída e o botão ▶️ de cada teste; e repetem a resolução de todos os desafios das 86 lições e dos 27 passos pela interface, que continuou igual à da versão 23.

12.6 TESTE DE MESA E DEPURADOR

Ainda na versão 24, todos os editores de código ganharam uma forma de ver o programa rodar passo a passo. Nos desafios das lições e do Caminho do André, que rodam uma vez e terminam, é o **teste de mesa**; nos dois modos do Sandbox, em que o programa roda sem parar dentro de um laço, é o **depurador**. Os dois mostram duas contagens diferentes: o **passo**, que só aumenta, e a **linha**, que vai e volta quando o programa está num laço.

- **Teste de mesa (lições):** o botão "📋 Teste de mesa", ao lado do "Executar", roda a chamada da caixa "Chamada" uma vez e monta uma tabela que desce do primeiro ao último passo (Figura 15): o número do passo, a linha, o código daquela linha, o valor de cada variável naquele momento — com as que acabaram de mudar em destaque — e o que foi impresso. Cada condição de if, while ou for mostra se deu true ou false, e chamadas a outros métodos do estudante aparecem com o nome do método. A última linha mostra o valor devolvido ou a exceção, e clicar numa linha da tabela leva o cursor até ela no editor.

Figura 15 – Teste de mesa do método fromBinary("1011"): a linha volta para o for a cada volta, enquanto o passo só aumenta



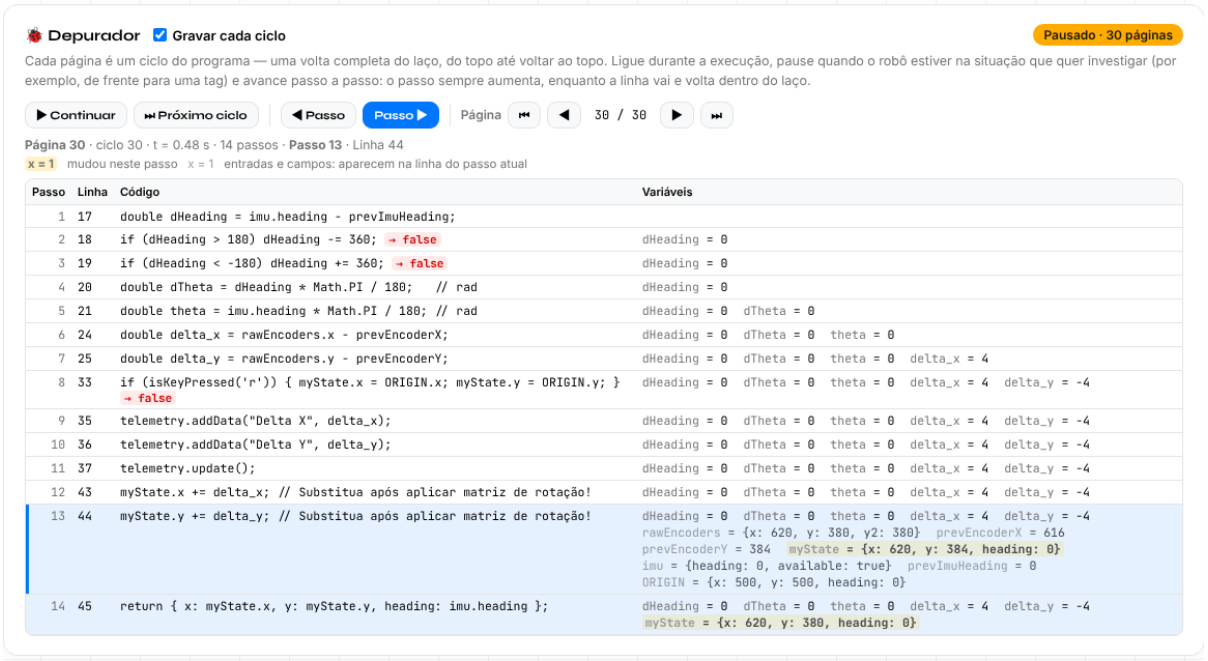
Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

- **Depurador (Sandbox, modo pontual e completo):** cada ciclo do programa — uma volta completa do laço, do topo até voltar ao topo — vira uma **página**, em

vez de os passos se acumularem numa lista só (Figura 16). No modo pontual, o ciclo é uma execução do trecho de código; no modo completo, é o que o OpMode faz entre uma espera do simulador e a seguinte (no LinearOpMode, uma volta do while (opModeIsActive()); no OpMode iterativo, uma chamada de loop()), e o que roda antes do START fica numa página de INIT.

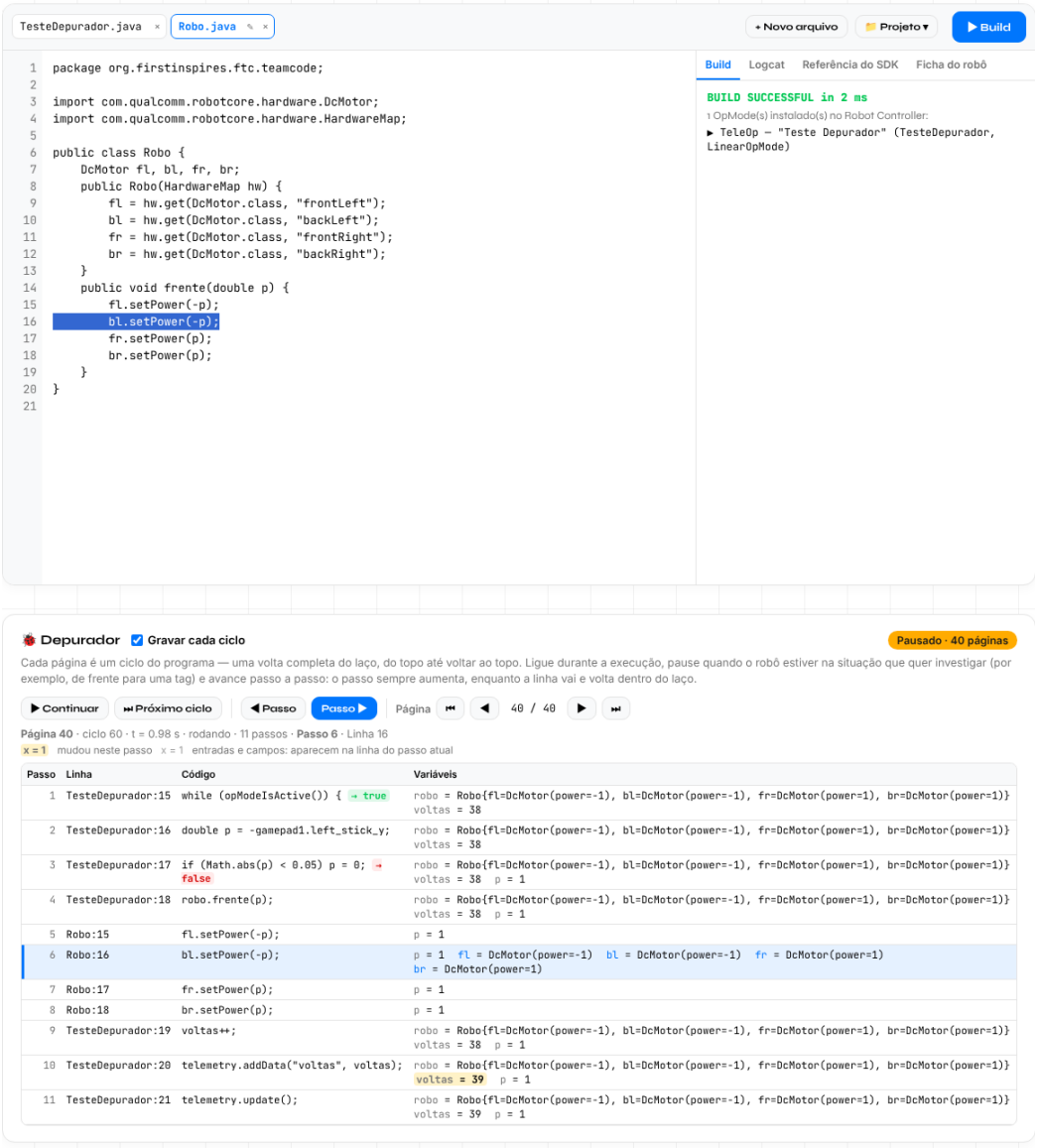
- **Ligar durante a execução:** a caixa "Gravar cada ciclo" pode ser marcada com o robô andando. Quando o robô chega à situação que se quer investigar — por exemplo, de frente para uma AprilTag que o código não está lendo direito —, "Passo ▶️" pausa a simulação na página mais recente e passa a andar um comando por vez. Dentro de uma página o robô fica parado; ao passar do último passo, o Sandbox roda exatamente mais um ciclo e abre a página seguinte. "◀️ Passo" volta, inclusive para a página anterior; "Próximo ciclo" roda um ciclo inteiro; os botões de página navegam pelas últimas 400 páginas; e "Continuar" volta a rodar normalmente.
- **Linha no editor:** o passo escolhido fica marcado no editor (no Monaco, com a linha destacada; no editor simples, com a linha selecionada). No modo completo, quando o passo está em outro arquivo do projeto — por exemplo, dentro de um método de uma classe auxiliar —, o editor troca de aba sozinho (Figura 17).
- **Variáveis compactas:** para a tabela não ficar enorme, cada linha mostra as variáveis locais e o que mudou; as entradas do modo pontual (rawEncoders, imu, myState, gamepad1…) e os campos da classe aparecem completos só na linha do passo atual.

Figura 16 – Depurador no modo pontual, pausado no passo 13 da página 30 (o modelo de odometria)



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

Figura 17 – Depurador no modo completo: o passo 6 está dentro do método frente da classe Robo, e o editor abriu Robo.java



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

Por dentro, os dois usam o mesmo instrumentador. Antes de cada comando — e dentro da condição de cada if, while e for — ele insere uma chamada que registra a linha e o valor das variáveis visíveis naquele ponto. Corpos de if, else, for e while escritos sem chaves ganham chaves, para que cada comando tenha o seu registro. A chamada começa conferindo se a gravação está ligada; desligada, não faz mais nada, e por isso os builds do modo completo já saem instrumentados e o depurador pode ser ligado a qualquer momento. No modo pontual, uma cópia instrumentada do mesmo código é preparada quando o depurador é ligado. A gravação tem limites para não pesar: 1500 passos no teste de mesa, 600 passos por página e as 400 páginas mais recentes. No caso mais pesado testado, o TeleOp de swerve, gravar custou cerca de 1,1 ms por ciclo.

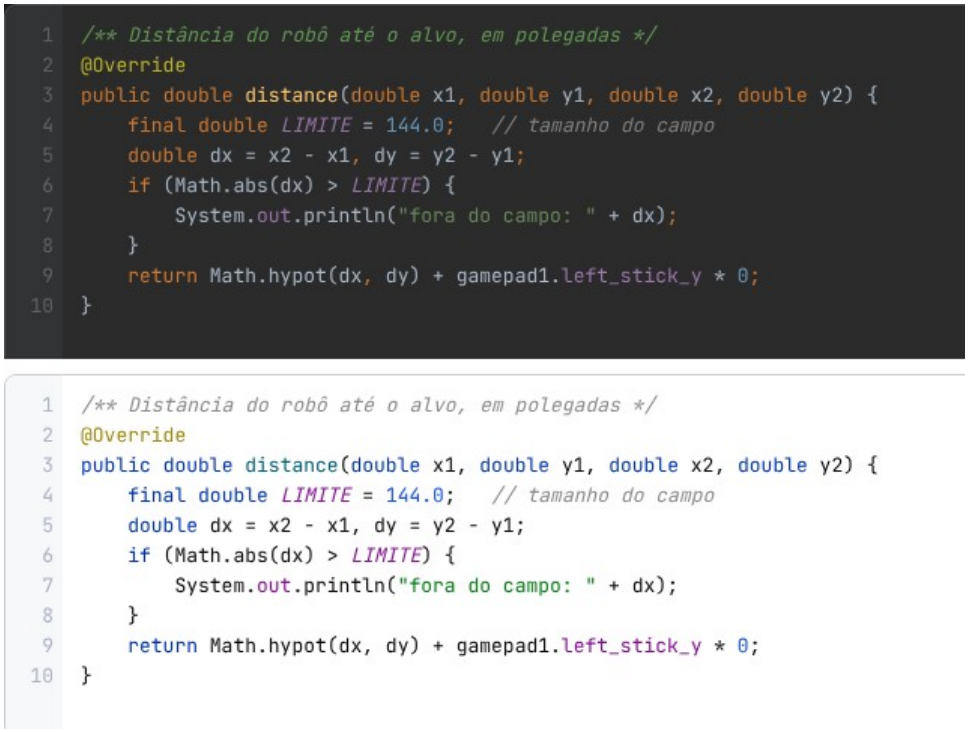
A instrumentação não pode mudar o que o programa faz, e isso foi conferido nos três lugares: as 101 soluções de referência das lições deram os mesmos resultados em todos os testes com a gravação ligada e desligada; os modelos do modo pontual (odometria, tank, mecanum e swerve, nos dois sistemas de coordenadas e nos dois idiomas, 16 combinações) levaram o robô exatamente à mesma posição depois da mesma sequência de teclas; e, no modo completo, o autônomo, o TeleOp de referência e o TeleOp de swerve terminaram na mesma posição, com até quatro casas decimais. Os testes de interface conferem a sequência de linhas e as condições de um teste de mesa, a página de INIT, as páginas que começam no while, a troca de arquivo, a marcação da linha no Monaco e no editor simples e que o robô não anda enquanto se avança dentro de uma página.

12.7 CORES DO ANDROID STUDIO E MENU DE OPÇÕES

Para que o código no Sandbox tenha a mesma aparência que terá no robô, todos os editores — os desafios das lições e dos passos, o modo pontual e o modo completo — e todos os trechos de código mostrados nas lições, nos passos, nos desafios de completar e de ordenar linhas e nas tabelas do depurador passaram a usar as cores do Android Studio (Figura 18): o esquema Darcula nos temas escuro e OLED e o IntelliJ Light no tema claro. Palavras-chave, textos, números, comentários e comentários de documentação, anotações, a declaração de cada método, os campos (como gamepad1.left_stick_y) e as constantes em maiúsculas têm cada um a sua cor, como no Android Studio, e as ligaduras da fonte foram desligadas para que == e != apareçam como são digitados.

No editor Monaco, isso exigiu uma gramática de Java própria, que separa declarações de métodos, campos e constantes, além de temas novos. Nos editores de texto simples, usados nas lições e quando não há internet, uma camada colorida fica por baixo do texto digitado, alinhada caractere por caractere e rolando junto com ele, de modo que a seleção, o cursor, as sugestões e a marcação do depurador continuam funcionando como antes.

Figura 18 – O mesmo método nos temas escuro (Darcula) e claro (IntelliJ Light)

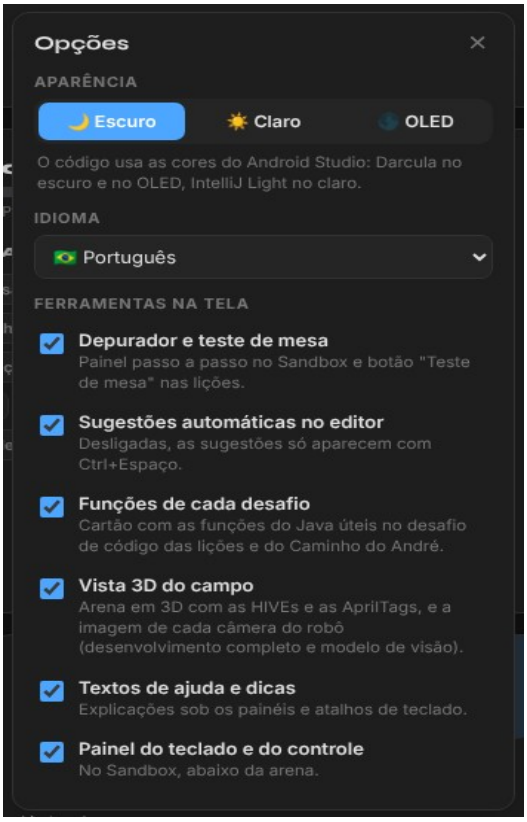


```java
/** Distância do robô até o alvo, em polegadas */
@Override
public double distance(double x1, double y1, double x2, double y2) {
    final double LIMITE = 144.0;   // tamanho do campo
    double dx = x2 - x1, dy = y2 - y1;
    if (Math.abs(dx) > LIMITE) {
        System.out.println("fora do campo: " + dx);
    }
    return Math.hypot(dx, dy) + gamepad1.left_stick_y * 0;
}
```

Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

Para a tela não ficar carregada, a escolha do tema e do idioma saiu do cabeçalho e foi para um botão "⚙️ Opções" na barra de navegação (Figura 19), que também liga e desliga partes da tela: o depurador e o teste de mesa (desligados, os painéis somem e a gravação para), as sugestões automáticas do editor (desligadas, elas só aparecem com Ctrl+Espaço), os textos de ajuda e as dicas de atalho, o cartão com as funções de cada desafio (seção 12.9) e o painel do teclado e do controle no Sandbox. As escolhas ficam guardadas no navegador e voltam na próxima visita.

Figura 19 – Menu de opções na barra de navegação



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

Os testes conferem que a camada colorida acompanha a rolagem vertical e horizontal do editor, que a gramática do Monaco classifica declarações, campos, constantes e linhas de import como o Android Studio, que cada opção esconde a parte certa da tela e é lembrada depois de recarregar a página, e que desligar as sugestões automáticas mantém o Ctrl+Espaço. Todos os testes anteriores — incluindo a resolução dos 372 desafios das lições e dos 99 do Caminho do André pela interface — continuaram passando.

12.8 O CÓDIGO ORGANIZADO EM CLASSES E PASTAS

No robô real, as equipes não escrevem tudo num arquivo só: separam o hardware numa classe, cada mecanismo (garra, elevador, intake) numa classe própria, as constantes em outra, e às vezes criam um OpMode base com o que todos os autônomos têm em comum. O modo completo já aceitava vários arquivos — uma classe de hardware usada pelos OpModes, constantes como Constantes.KP e métodos static —, mas recusava várias formas comuns de organizar o código: herança entre classes do próprio projeto, classes abstratas, interfaces, dois métodos ou construtores com o mesmo nome, pastas dentro do TeamCode e classes internas; o import static era aceito, mas ignorado. Na versão 24, tudo isso passou a funcionar como no Android Studio (Tabela 22).

Tabela 22 – Formas de organizar o código em classes aceitas no modo completo

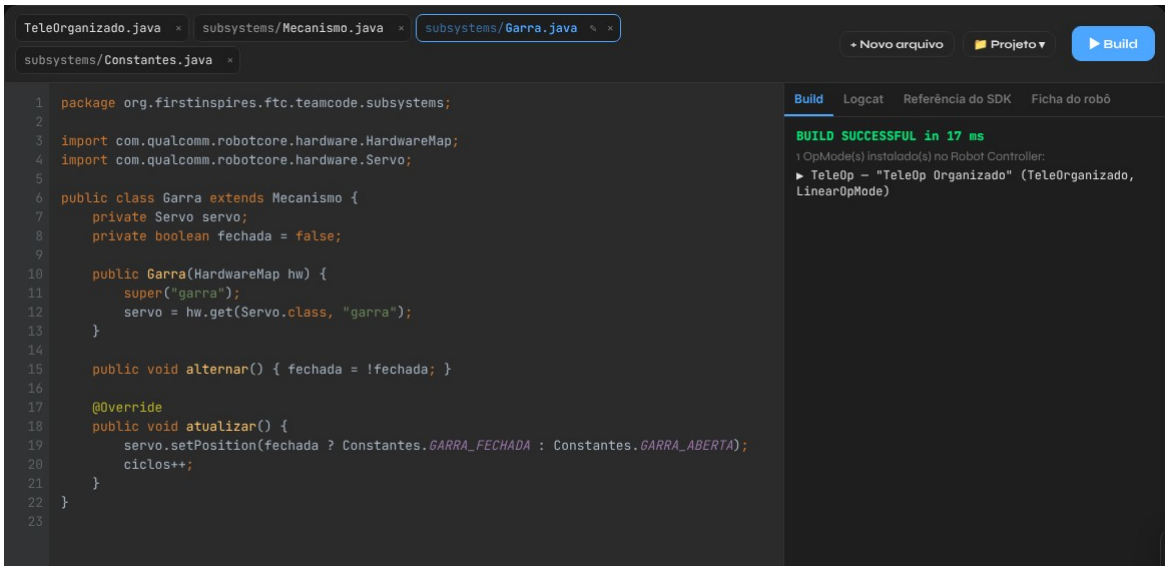
Recurso	Exemplo
Pastas (pacotes dentro do teamcode)	package org.firstinspires.ftc.teamcode.subsystems; e, no OpMode, import org.firstinspires.ftc.teamcode.subsystems.Garra;
Herança entre classes do projeto	public class Garra extends Mecanismo, com super("garra") no construtor, super.status() e campos protected
Classes e métodos abstratos	public abstract class Mecanismo { public abstract void atualizar(); }
Interfaces	class Elevador implements Atualizavel, Nomeado — com constantes, métodos default e interfaces que estendem outras
Sobrecarga	double saida(double e) e double saida(double e, double d); vários construtores, com this(...)
import static	import static org.firstinspires.ftc.teamcode.Constantes.*; e import static java.lang.Math.*;
Classes e enums internos	public static class Pose dentro de Geo, usada como new Geo.Pose(1, 2)
OpMode base	public abstract class BaseAuto extends LinearOpMode, estendida pelos autônomos
instanceof	if (m instanceof Garra) com classes e interfaces do projeto

Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

Por dentro, a herança é resolvida no build: o objeto de uma subclasse recebe os campos e os métodos de toda a cadeia de classes, e a chamada a um método sobrescrito vai sempre para a versão da classe mais específica, mesmo quando é feita dentro de um método da classe mãe — o despacho dinâmico do Java, que permite percorrer um array de Mecanismo e chamar atualizar() em cada um. A escolha entre versões sobrecarregadas é feita na hora da chamada, pela quantidade e pelo tipo dos valores passados (int, double, String, boolean ou o objeto). Os construtores seguem a ordem do Java: primeiro o construtor da classe mãe, depois a inicialização dos campos da subclasse e por fim o corpo do construtor, e os campos static de uma classe são criados depois dos de outra classe que eles usam, qualquer que seja a ordem das abas.

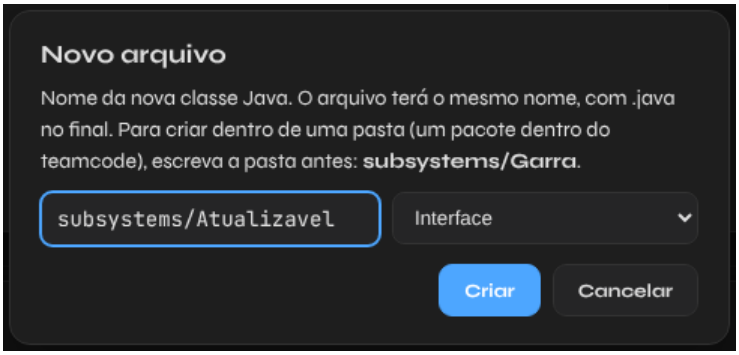
O editor acompanha a organização (Figura 20). O botão "+ Novo arquivo" aceita uma pasta antes do nome — subsystems/Garra cria o arquivo com package org.firstinspires.ftc.teamcode.subsystems — e um tipo, que escolhe o modelo inicial: classe, interface, classe abstrata, enum, TeleOp ou autônomo (Figura 21). As abas mostram a pasta de cada arquivo, as sugestões passam a conhecer as classes do projeto — inclusive os métodos herdados e o import automático quando a classe está em outra pasta —, o projeto salvo em .zip guarda cada arquivo na pasta correspondente do Android Studio e, ao importar um projeto do Android Studio, as pastas dentro do TeamCode são mantidas como estão. A Referência do SDK ganhou uma seção sobre organizar o código em classes.

Figura 20 – Projeto organizado em pastas: Garra estende a classe abstrata Mecanismo e usa as constantes de outra classe



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

Figura 21 – Novo arquivo numa pasta, com o tipo escolhido



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

Como no resto do compilador, cada regra do Java que o simulador passa a aceitar vem com uma mensagem para quando ela é quebrada (Tabela 23). Recursos que continuam fora do Sandbox — classes anônimas, lambdas, classes genéricas próprias e records — têm mensagens que explicam o que fazer no lugar.

Tabela 23 – Novas mensagens do build sobre classes

Situação	Mensagem
Subclasse sem o método abstrato	N precisa implementar o método up() de M (ou ser declarada abstract)
new de classe abstrata	M é abstract e não pode ser criada com new: crie um objeto de uma subclasse de M
Método de interface sem public	f() precisa ser public: ele implementa um método de interface, e todo método de interface é public
Campo repetido na subclasse	O campo x já existe em M, de quem N herda: use o campo herdado (sem declarar de novo) ou outro nome
super(...) fora do lugar	super(...) precisa ser o primeiro comando do construtor

Situação	Mensagem
Classe mãe sem construtor vazio	M não tem construtor sem parâmetros: o construtor de N precisa começar com super(...) passando os valores de M(int)
Classe de outra pasta sem import	Não encontrei o símbolo D — a classe D está no pacote org.firstinspires.ftc.teamcode.subsystems: falta import ...
extends com interface	I é uma interface: use implements I (extends é para classes)
Classe anônima	Classes anônimas (new I() { ... }) não são suportadas no Sandbox: crie uma classe com nome que estende ou implementa I

Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

Um teste novo roda dez programas com várias classes — polimorfismo com classe abstrata e super, interfaces com métodos default e constantes, sobrecarga de métodos e de construtores, a ordem de inicialização, import static, pastas, classes internas, um OpMode base com sleep e telemetria, a ordem dos campos static e toString/equals próprios — e confere a saída de cada um com e sem a instrumentação do depurador, além de 18 casos de erro. Outro teste opera a interface: abas com pastas, build e execução do TeleOp organizado, sugestões com métodos herdados, criação de arquivos com pasta e tipo e o .zip com as pastas do Android Studio. Os testes anteriores do modo completo deram exatamente os mesmos resultados.

12.9 FUNÇÕES DE CADA DESAFIO

Para que o estudante saiba quais ferramentas do Java um desafio pede antes de travar nele, cada desafio de código — das lições, das revisões e dos passos do Caminho do André — ganhou um cartão "🧰 Funções para este desafio", acima do editor (Figura 22). Cada função do cartão mostra a assinatura com o tipo devolvido, para que serve, como funciona, as entradas, a saída e um exemplo com o resultado. As que aparecem pela primeira vez no percurso — as lições na ordem do mapa e depois os passos do Caminho do André — ganham o selo NOVA, e o cartão já abre sozinho; quando tudo já foi visto, ele fica fechado, mostrando só os nomes, e cada função indica em que lição apareceu pela primeira vez.

Figura 22 – Cartão de funções da lição 11 (trigonometria): três funções novas e uma já usada na lição 5



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

A lista de cada desafio não foi escrita à mão: ela sai da solução de referência do próprio desafio, e por isso continua certa se um desafio mudar. São 31 verbetes nos dois idiomas: 15 do Math (abs, max, min, round, hypot, sqrt, pow, sin, cos, tan, atan2, toRadians, toDegrees, signum e PI), três de String (length, charAt e equals), três de arrays (length, new double[]{…} e new double[n]), cinco ferramentas da linguagem (o cast (int), o resto %, o operador ternário, o laço for e os operadores lógicos) e cinco métodos auxiliares que a solução sugere escrever, como lerp na curva de Bézier. Uma alternativa útil também pode entrar — no desafio da distância, Math.sqrt aparece ao lado de Math.hypot. Os 12 desafios que só pedem contas e if mostram uma frase dizendo isso, e todos os cartões terminam com a dica de usar System.out.println e o teste de mesa. O cartão pode ser escondido pelo menu de opções.

O teste automatizado monta o cartão de todos os 101 desafios nos dois idiomas, confere que toda ferramenta encontrada tem verbete e que cada uma ganha o selo NOVA uma única vez no percurso, e abre lições e passos para conferir o cartão aberto ou fechado, os selos, a alternativa, os métodos auxiliares, a opção de esconder e a largura de celular.

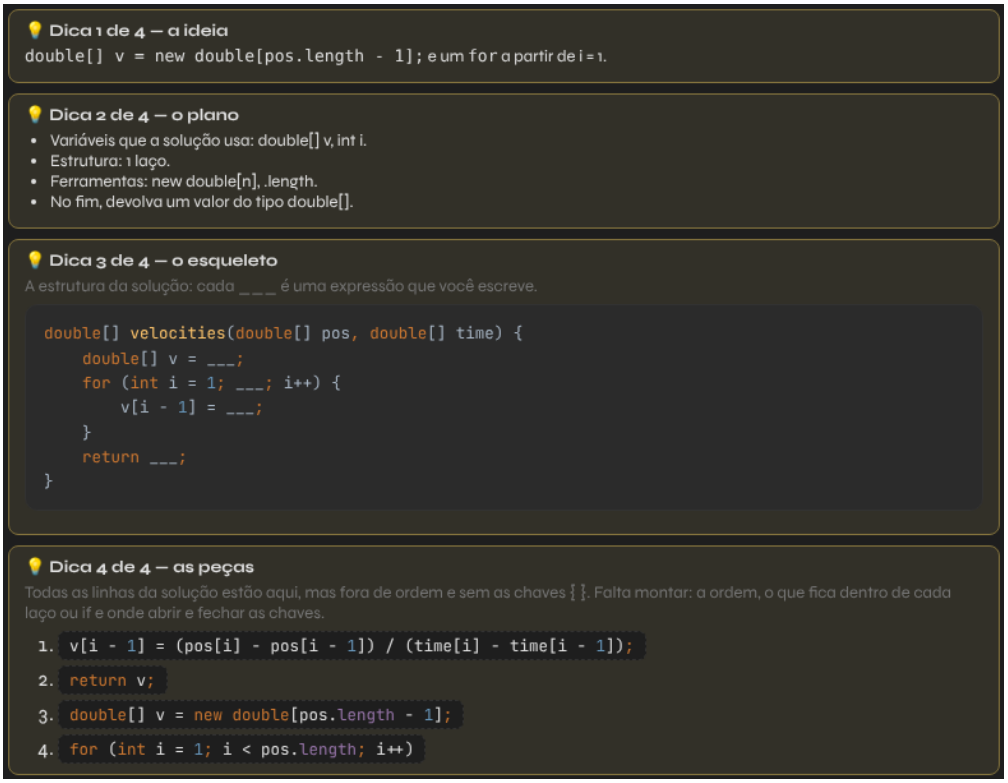
12.10 DICAS EM ESCADA

Até a versão 24, quem travava num desafio de código tinha uma dica depois do primeiro erro e, em alguns desafios, uma segunda depois do terceiro. A versão 25

acrescenta o botão "💡 Dica", que abre uma dica por vez, cada uma mais perto do código (Figura 23): (1) a ideia, que é o texto de dica do próprio desafio; (2) o plano, com as variáveis que a solução cria, quantos laços e condições ela tem, os métodos auxiliares, as funções do Java e o que o método devolve; (3) o esqueleto, que é a estrutura da solução com cada expressão trocada por ___; e (4) as peças, que são todas as linhas da solução, mas embaralhadas e sem as chaves.

Na quarta dica o código inteiro está na tela, mas não montado: o estudante ainda precisa decidir a ordem, o que fica dentro de cada laço ou if e onde abrir e fechar as chaves — é o mesmo raciocínio dos desafios de ordenar linhas do Mapa de Estudos, agora com o código do próprio desafio. As dicas abertas ficam guardadas com o progresso, e a ordem das peças é sempre a mesma para cada desafio, mas nunca a ordem original.

Figura 23 – As quatro dicas do desafio da lição 25 (velocidade em cada intervalo de um vetor de posições)



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

As dicas não foram escritas à mão: elas são geradas a partir da solução de referência de cada desafio, com um leitor de código que separa comandos, cabeçalhos de laços e de if, inicializadores de vetores e métodos auxiliares. O teste automatizado gera as dicas dos 101 desafios e confere que as peças, juntas, contêm exatamente o código da solução (menos as chaves e a assinatura do método, que já vem no código inicial), que todo esqueleto tem lacunas e não entrega nenhuma conta,

que as peças nunca saem na ordem original e que todo plano tem conteúdo; depois abre a lição 1, revela as quatro dicas, confere a ordem, as cores do código e que elas continuam abertas ao reabrir a lição.

13. ROBÔ COMPLETO E DRIVER HUB

Nas versões anteriores, o código do estudante era um trecho executado a cada ciclo do simulador, com os motores e sensores já entregues em variáveis prontas. No robô real, porém, o programador escreve arquivos Java completos e depois opera o robô por um aplicativo, o Driver Station (no Driver Hub). A versão 22 acrescenta a tela "Robô Completo", em que nada vem pronto: o estudante escreve o package, os imports, as anotações, a classe do OpMode, o mapeamento do hardware, a visão computacional e o autônomo, compila o projeto e opera o robô por um Driver Hub simulado, que exige configurar o robô e apertar INIT e START como numa partida.

13.1 DO PROJETO JAVA AO OPMODE NO SIMULADOR

O projeto do estudante tem vários arquivos .java, como na pasta TeamCode do Android Studio. O botão "Build" passa os arquivos por um compilador escrito para o Sandbox, que confere as mesmas regras que o compilador Java confere no projeto real e devolve as mensagens com o arquivo e a linha (Tabela 24).

Tabela 24 – Verificações feitas no build e exemplos de mensagens

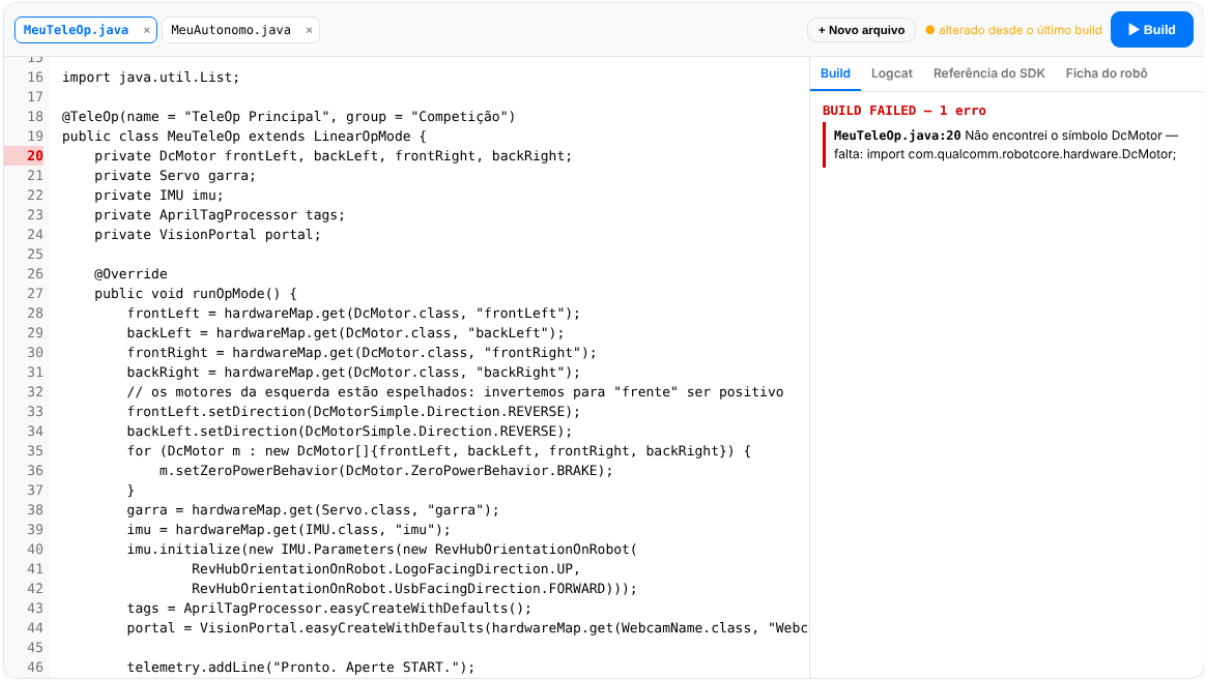
Verificação	Exemplo de mensagem
Pacote	O pacote deve ser org.firstinspires.ftc.teamcode (a pasta TeamCode do projeto)
Imports	Não encontrei o símbolo DcMotor — falta: import com.qualcomm.robotcore.hardware.DcMotor;
Import no pacote errado	Não encontrei a classe com.qualcomm.hardware.IMU — o certo é import com.qualcomm.robotcore.hardware.IMU;
Classe pública e nome do arquivo	A classe MeuTeleop é pública e deve ficar no arquivo MeuTeleop.java
Métodos obrigatórios e @Override	MeuTeleOp estende LinearOpMode, mas não tem o método public void runOpMode()
Ponto e vírgula e chaves	Falta ';' no fim desta linha
Anotações	MeuTeleOp não aparece no Driver Hub: falta @TeleOp ou @Autonomous (aviso)
Classes reais não simuladas	DistanceSensor existe no FTC SDK, mas não é simulado no Sandbox

Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

Depois das verificações, o código é traduzido para JavaScript. O ponto central da tradução é que todo método vira um gerador (function*): quando o código chama waitForStart(), sleep(ms) ou opModeIsActive(), o gerador "pausa" e devolve o controle

ao simulador, que avança a física um ciclo (1/60 s) e depois retoma o código exatamente onde ele parou. Assim um LinearOpMode escrito como no robô real — com laços while que esperam — funciona sem travar a página. Cada linha gerada guarda o arquivo e a linha de origem, para que um erro durante a execução apareça no lugar certo do código do estudante. Também são aceitos OpModes iterativos (OpMode, com init, init_loop, start, loop e stop), classes auxiliares em outros arquivos (por exemplo, uma classe RobotHardware que recebe o OpMode no construtor), enums, arrays e o laço for-each.

Figura 24 – Build com erro: a linha 20 marcada no editor e a mensagem com o import que falta



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

13.2 O FTC SDK SIMULADO

O Sandbox implementa as classes do FTC SDK que um robô de competição usa, com os mesmos nomes de classes e métodos e com o comportamento observável do robô real:

- **hardwareMap:** procura o dispositivo pelo nome e pelo tipo na configuração ativa do Driver Hub. Se o nome não existir, a mensagem é a mesma do robô real: Unable to find a hardware device with name "frontLeft" and type DcMotor.
- **DcMotor e DcMotorEx:** setPower, setDirection, setZeroPowerBehavior (BRAKE ou FLOAT), os modos RUN_WITHOUT_ENCODER, RUN_USING_ENCODER, STOP_AND_RESET_ENCODER e RUN_TO_POSITION, getCurrentPosition, setTargetPosition, isBusy e setVelocity. Os ticks do encoder vêm da distância que cada roda realmente

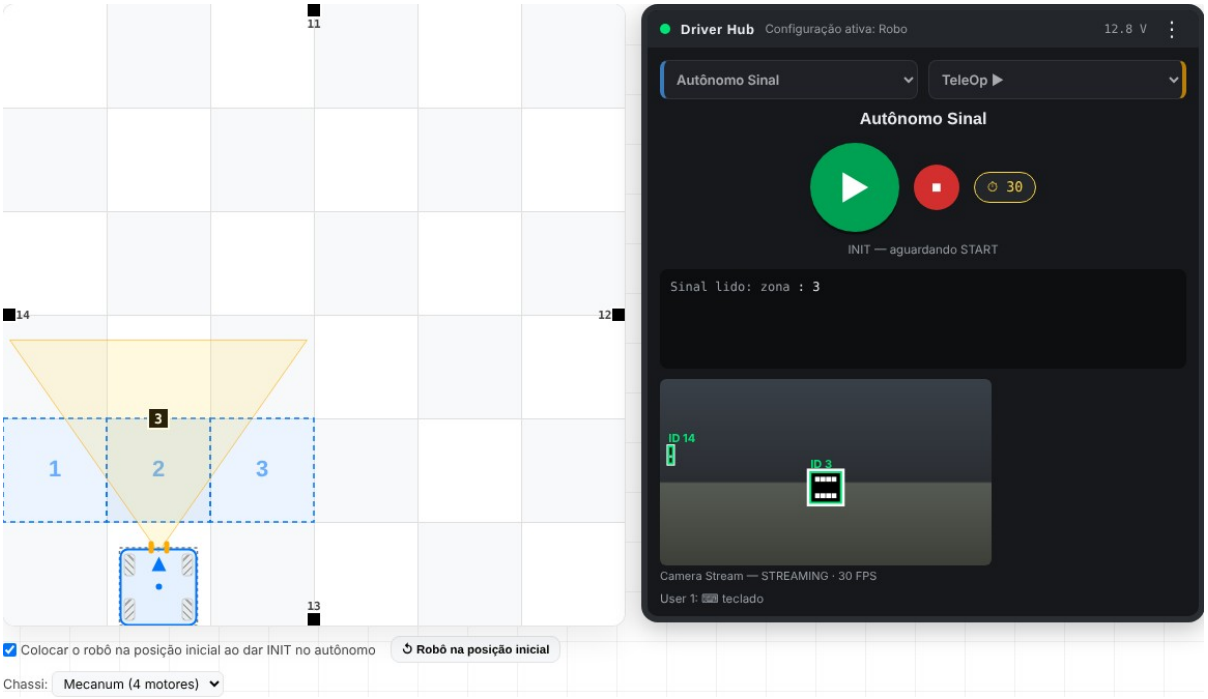
andou na física (motor goBILDA 5203 de 537,7 ticks por volta e roda de 104 mm, cerca de 41,8 ticks por polegada). Mudar para RUN_TO_POSITION sem um alvo gera o mesmo erro do SDK.

- **Montagem espelhada:** como no robô real, os motores dos dois lados estão montados espelhados; sem setDirection(REVERSE) no lado esquerdo, o comando "para a frente" faz o robô girar.
- **Servo:** setPosition, getPosition, setDirection e scaleRange; a garra aparece na frente do robô, abrindo e fechando.
- **IMU:** initialize com IMU.Parameters e RevHubOrientationOnRobot, resetYaw e getRobotYawPitchRollAngles. O Control Hub está montado com o logo para cima e a USB para a frente; se a orientação declarada for outra, o yaw sai errado (com o logo declarado na horizontal, ele não muda quando o robô gira), e uma orientação impossível (logo e USB paralelos) gera erro.
- **Telemetria:** addData (inclusive com formato, como "%.2f"), addLine, update, log e setAutoClear. Num LinearOpMode, nada aparece no Driver Hub sem telemetry.update(); num OpMode iterativo, a atualização é automática, como no SDK.
- **Utilidades:** ElapsedTime e getRuntime() usam o relógio da simulação; Range.clip, List e ArrayList, String.format e System.out.println, que aparece numa aba "Logcat".

13.3 VISÃO COMPUTACIONAL COM APRILTAGS

A câmera do robô é uma webcam que, por padrão, fica na frente do chassi (desde a versão 25, a montagem pode ser mudada em seis graus de liberdade; Seção 16.5), com campo de visão de 70°. O estudante cria um AprilTagProcessor e um VisionPortal (pelos métodos easyCreateWithDefaults ou pelos Builders do SDK) e lê as detecções com getDetections(). Cada detecção tem o id, o metadata da biblioteca de tags e o ftcPose, com x, y, range (distância), bearing (ângulo até a tag) e yaw (giro da tag em relação à câmera), em polegadas e graus, com um pequeno ruído que cresce com a distância. A câmera leva cerca de 0,75 s para abrir, como no robô real. O campo de treino, que é o padrão do desenvolvimento completo, tem quatro tags nas paredes (11 a 14) e um sinal com a tag 1, 2 ou 3, sorteada a cada INIT do autônomo; o cone amarelo na arena mostra o que a câmera vê, e o Driver Hub mostra o "Camera Stream", com contorno verde nas tags detectadas (Figura 25).

Figura 25 – Autônomo em INIT: a câmera lê o sinal (tag 3) e a telemetria mostra a zona escolhida



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

A visão é usada nas duas partes do jogo. No autônomo, o código lê o sinal no laço de INIT (while (opModeInInit())) e, depois do START, estaciona na zona correspondente usando os encoders (Código 6). No TeleOp, um botão alinha o robô com a tag 11, usando range, bearing e yaw como erros de três controles proporcionais.

Código 6 – Leitura do sinal durante o INIT (trecho do autônomo de referência)

```
AprilTagProcessor tags = AprilTagProcessor.easyCreateWithDefaults();
VisionPortal portal = VisionPortal.easyCreateWithDefaults(
        hardwareMap.get(WebcamName.class, "Webcam 1"), tags);
int zona = 2;
while (opModeInInit()) {
    for (AprilTagDetection d : tags.getDetections()) {
        if (d.id >= 1 && d.id <= 3) zona = d.id;
    }
    telemetry.addData("Sinal lido: zona", zona);
    telemetry.update();
}
portal.close();
andar(27, 0.5);
if (zona == 1) lateral(-24, 0.5);
if (zona == 3) lateral(24, 0.5);
```

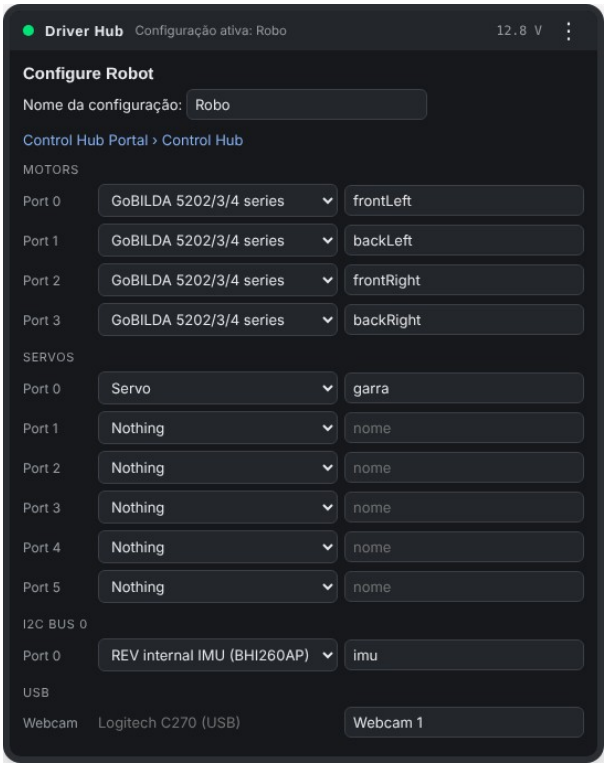
Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

13.4 O DRIVER HUB: CONFIGURAÇÃO, INIT E START

O Driver Hub simulado segue o fluxo do aplicativo real. Antes de qualquer OpMode, o estudante abre o menu "Configure Robot" e cria a configuração do robô: o tipo e o nome de cada motor nas portas 0 a 3 do Control Hub, o servo da garra, a IMU

no barramento I2C 0 e a webcam (Figura 26). A "Ficha do robô" informa como o robô foi montado — por exemplo, que a porta 0 é a roda dianteira esquerda —, e é a configuração que liga esses fios aos nomes usados no hardwareMap.get do código; um nome trocado ou uma porta errada aparece na hora como erro ou como um robô que anda torto.

Figura 26 – Tela "Configure Robot" do Driver Hub simulado



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

Com o build feito, os OpModes aparecem nas duas listas do Driver Hub, Autonomous e TeleOp, agrupados pelo group da anotação. O estudante escolhe um, aperta INIT (o código roda até o waitForStart e a câmera já pode ler o sinal) e depois START. No autônomo, o timer de 30 s encerra o OpMode sozinho e, se a anotação tiver preselectTeleOp, o TeleOp indicado já fica escolhido para a próxima etapa. Ao apertar STOP, o código tem 1,5 s para sair do laço; se não sair (um laço sem opModeIsActive()), o OpMode é encerrado à força e o Logcat explica o motivo. Uma exceção não tratada mostra a mensagem em estilo Java com o arquivo e a linha e, como no robô real, só permite um novo INIT depois de "Restart Robot". O teclado ou um controle físico funcionam como o gamepad1.

13.5 MISSÕES E VALIDAÇÃO

A tela tem dez missões, de 100 XP cada, verificadas pelo próprio simulador (Tabela 25); quem completa todas ganha a medalha "Robô completo".

Tabela 25 – Missões do Robô Completo e como são verificadas

Missão	Como o simulador confere
Configurar o robô	Configuração salva com os quatro motores nas portas do chassi, o servo da garra na porta 0, a IMU e a webcam, todos com nome.
Build sem erros	Compilação sem erros com pelo menos um TeleOp e um autônomo.
Dirigir no TeleOp	Num mesmo TeleOp, o robô anda 24 pol. para a frente, 24 pol. de lado e gira 180° (impossível com os motores na direção errada).
Garra	O servo da porta 0 vai a uma posição ≤ 0,2 e a outra ≥ 0,8.
IMU	O yaw é lido de uma IMU inicializada com a orientação real do Control Hub.
Visão	getDetections() devolve pelo menos uma tag.
Alinhar com a tag	No TeleOp, com a tag 11 detectada, o robô para a 24 ± 2 pol. dela, com bearing e yaw até ±5°.
Autônomo	Estaciona na zona do sinal sorteado em até 30 s, com dois sinais diferentes.
Odometria (v23)	Com o Pinpoint ou os pods, o robô anda 96 pol. ou mais e volta ao ponto de partida com erro de até 1,5 pol., lendo a posição da odometria.
Limelight 3A (v23)	A pose MegaTag (getBotpose ou getBotpose_MT2) fica a até 2 pol. da posição real.

Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

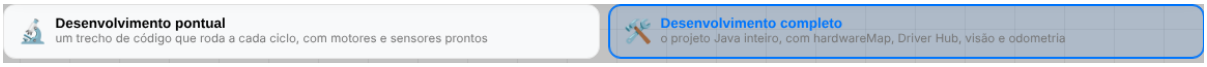
Os testes automatizados da versão 22 escrevem o projeto de referência no editor, configuram o robô pela própria tela "Configure Robot", fazem o build e operam o Driver Hub: o autônomo estacionou na zona certa com os três sinais (cerca de 1,7 s para a zona 2 e 3,3 s para as zonas laterais, bem dentro dos 30 s), o TeleOp completou as missões de direção, garra, IMU, visão e alinhamento (parando a 24,0 pol. da tag 11, com bearing de 0,0°), e as oito missões foram concluídas. Outros testes conferem as mensagens de 16 casos de compilação e de 9 situações de erro durante a execução — nome de dispositivo que não existe na configuração, NullPointerException com a linha certa, RUN_TO_POSITION sem alvo, hardwareMap usado num campo da classe, orientação impossível da IMU, laço que não respeita o STOP e telemetria sem update() —, o projeto com vários arquivos (classe RobotHardware e OpMode iterativo) e o editor com e sem o Monaco. Nenhum erro de página foi registrado, e os testes das versões anteriores continuam passando.

13.6 O SANDBOX EM DOIS MODOS

Na versão 23, o Robô Completo deixou de ser uma tela separada e passou a ser um dos dois modos do próprio Sandbox, já que os dois usam o mesmo robô e a mesma física. No topo do Sandbox, o estudante escolhe entre o **desenvolvimento pontual** — o trecho de código que roda a cada ciclo, com motores e sensores já

entregues em variáveis, para testar uma ideia como a odometria ou o mixer — e o **desenvolvimento completo** — o projeto Java inteiro com o Driver Hub (Figura 27). O botão Sandbox do menu abre o último modo usado, o endereço #/sandbox/completo leva direto ao modo completo, e o "Abrir no Sandbox" das lições continua abrindo o modo pontual.

Figura 27 – Seletor de modo no topo do Sandbox



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

As ferramentas do modo pontual também passaram a valer no modo completo (Figura 28): pausar e continuar (o que congela a física, o relógio e o OpMode juntos), resetar a simulação, mostrar ou ocultar o rastro, os eixos e a bússola, as barras de potência das rodas, os controles de velocidade, distância entre rodas, inércia e ruído, e o painel do teclado e do controle. O botão Fantasma ganha outro uso nesse modo: mostra na arena onde o Pinpoint e a Limelight acham que o robô está, e uma tabela compara as duas estimativas com a posição real. A barra de ferramentas, a legenda dos rastros e o painel do teclado passaram a ter a mesma largura da arena nos dois modos: com a fonte do aplicativo, a barra ficava mais larga que o campo e desalinhava a coluna; agora os cinco botões cabem numa linha e, em telas estreitas, quebram em linhas iguais.

Figura 28 – Ferramentas do Sandbox no modo completo, com a comparação entre a posição real e a do Pinpoint



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

13.7 SUGESTÕES NO EDITOR DE CÓDIGO

Também na versão 23, o editor de código passou a sugerir, enquanto o estudante digita, tudo o que existe nas classes do FtcSDK que o simulador reproduz. O modelo usado pelas sugestões descreve 81 classes, enums e anotações — LinearOpMode e OpMode, Gamepad, Telemetry, HardwareMap, DcMotor e DcMotorEx, Servo e CRServo, AnalogInput, IMU, YawPitchRollAngles, ElapsedTime, Range, VisionPortal, AprilTagProcessor e AprilTagDetection, GoBildaPinpointDriver, Pose2D, Limelight3A, LLResult, entre outras, além das classes do Java usadas nos

desafios (Math, String, Arrays…) e, na versão 25, Pose3D, Position e a biblioteca de tags do jogo —, mais as variáveis do modo pontual, num total de 634 campos e métodos, cada um com a assinatura, o tipo de retorno e uma explicação curta em português e em inglês.

As sugestões seguem o tipo de cada expressão, mesmo em cadeias: depois de "imu.getRobotYawPitchRollAngles()." aparecem getYaw, getPitch e getRoll (Figura 29), e depois de "gamepad1." aparecem todos os eixos, botões e métodos do controle (Figura 30). O editor reconhece o tipo de variáveis locais, campos da classe, parâmetros, itens de laços for-each, elementos de listas e o retorno de hardwareMap.get(Classe.class, ...). Também sugere:

- os valores de enums, como DcMotor.RunMode.RUN_TO_POSITION ou AngleUnit.DEGREES;
- nomes de classes, acrescentando sozinho o import que falta no topo do arquivo;
- dentro das aspas de hardwareMap.get, os nomes dos dispositivos daquele tipo cadastrados no Configure Robot do Driver Hub;
- anotações (@TeleOp, @Autonomous, @Override, @Disabled), palavras-chave e modelos prontos, como um LinearOpMode inteiro, um laço while (opModeIsActive()) ou a montagem do VisionPortal.

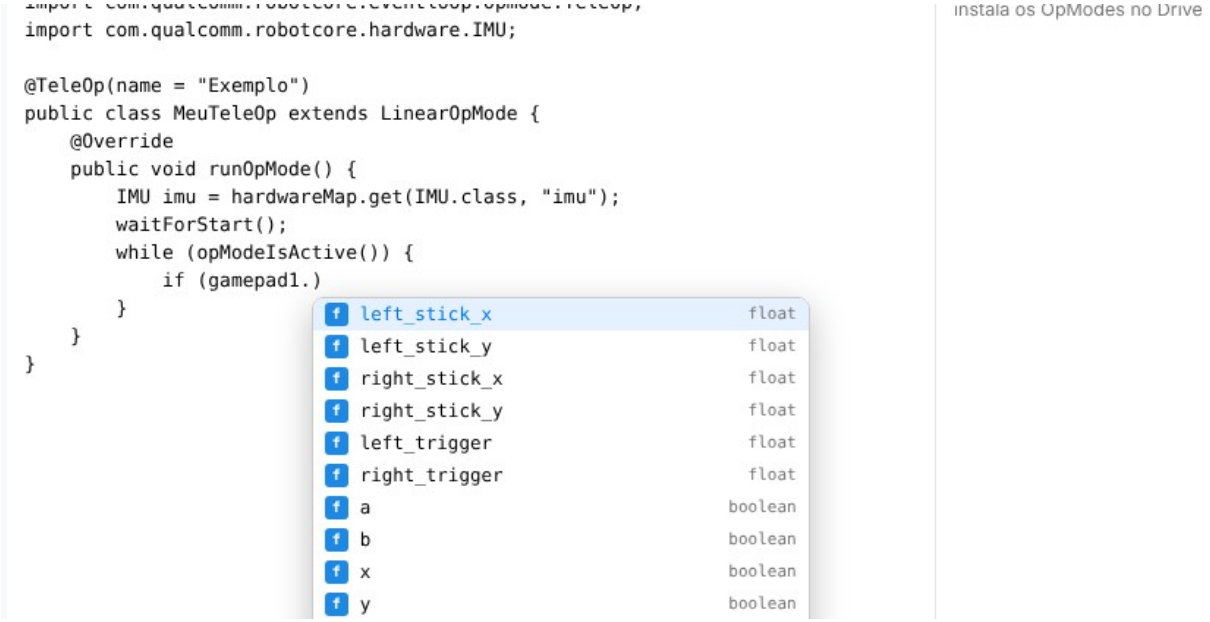
Ao abrir os parênteses de um método, uma caixa mostra os parâmetros e destaca o que está sendo digitado; no Monaco, passar o mouse sobre um nome mostra a sua explicação. As sugestões aparecem sozinhas ao digitar ".", "@" ou aspas, e Ctrl+Espaço as abre a qualquer momento; as setas escolhem, Enter ou Tab aceitam e Esc fecha. Tudo funciona nos dois modos do Sandbox e tanto no editor Monaco quanto no editor simples usado sem internet.

Figura 29 – Sugestões após uma cadeia de chamadas da IMU, com assinatura e explicação



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

Figura 30 – Campos e métodos do gamepad sugeridos ao digitar "gamepad1."



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

Os testes conferem as sugestões para o gamepad, a telemetria, a IMU, os motores, a câmera, o Pinpoint e a Limelight, os enums, as variáveis do modo pontual, o import automático, os nomes da configuração, a ajuda de assinatura e o aceite pelo teclado, nos dois editores; nenhum erro de página foi registrado.

13.8 SALVAR, ABRIR E IMPORTAR PROJETOS

O modo completo também ganhou o menu **Projeto**, na barra do editor (Figura 31), para levar o trabalho para fora do navegador e trazê-lo de volta:

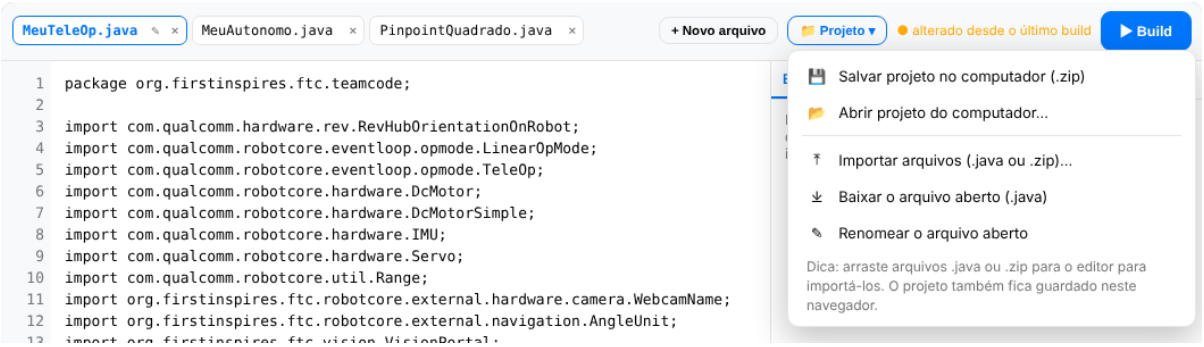
- **Salvar projeto no computador:** baixa um arquivo .zip com todos os arquivos .java, organizados nas mesmas pastas do TeamCode do Android Studio (TeamCode/src/main/java/org/firstinspires/ftc/teamcode), mais um arquivo com a configuração do Driver Hub e a montagem do robô. Os mesmos arquivos podem ser copiados para o projeto do robô real.
- **Abrir projeto do computador:** lê esse .zip e substitui o projeto atual — arquivos, configuração e montagem —, depois de uma confirmação que oferece salvar o projeto atual antes (Figura 32); em seguida, o build é feito sozinho e os OpModes já aparecem no Driver Hub.
- **Importar arquivos:** junta ao projeto um ou mais arquivos .java, ou os arquivos .java de um .zip — inclusive o .zip de um projeto inteiro do Android Studio, do qual só a pasta teamcode é aproveitada. Arquivos com nome repetido podem ser substituídos ou pulados, arquivos em pastas dentro do TeamCode (como teamcode.subsystems) ficam na pasta deles (Seção 12.8), e

só os que estão fora do pacote teamcode são oferecidos para mudar de pasta, com a linha package e os imports ajustados. Também é possível arrastar os arquivos direto para o editor.

- **Baixar o arquivo aberto:** baixa só o arquivo .java que está na tela.

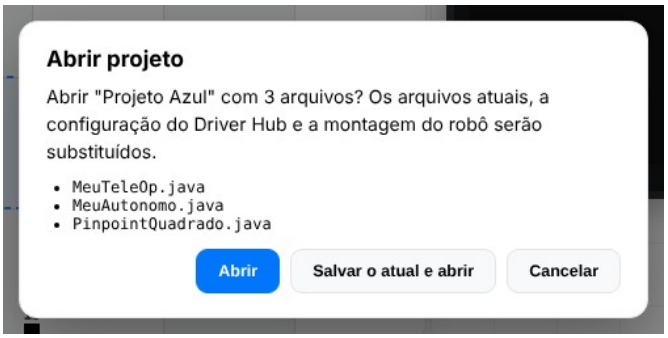
Para renomear um arquivo, basta clicar duas vezes no nome da aba (ou tocar duas vezes, no celular) ou usar o lápis ao lado do nome: o nome vira um campo de texto, Enter confirma e Esc cancela. Como no Android Studio, se a classe do arquivo tinha o nome antigo, a classe, os construtores e os usos nos outros arquivos também são renomeados, sem mudar os textos entre aspas — assim, o nome do OpMode no Driver Hub continua o mesmo. Criar e apagar arquivos passou a usar caixas de diálogo da própria página, que funcionam mesmo quando o navegador bloqueia as janelas de aviso.

Figura 31 – Menu Projeto na barra do editor do modo completo



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

Figura 32 – Confirmação antes de abrir um projeto salvo



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

Os testes salvam um projeto com a configuração do Driver Hub, trocam tudo, abrem o .zip salvo e conferem os arquivos, a configuração, a montagem e o build; importam um .zip comprimido no formato do Android Studio, com subpacote e exemplos do SDK; importam um .java com nome repetido; arrastam um arquivo para o editor; e renomeiam abas com duplo clique, com dois toques numa tela de celular e com o lápis, nos dois editores. O .zip gerado também foi aberto e conferido por programas comuns de compactação.

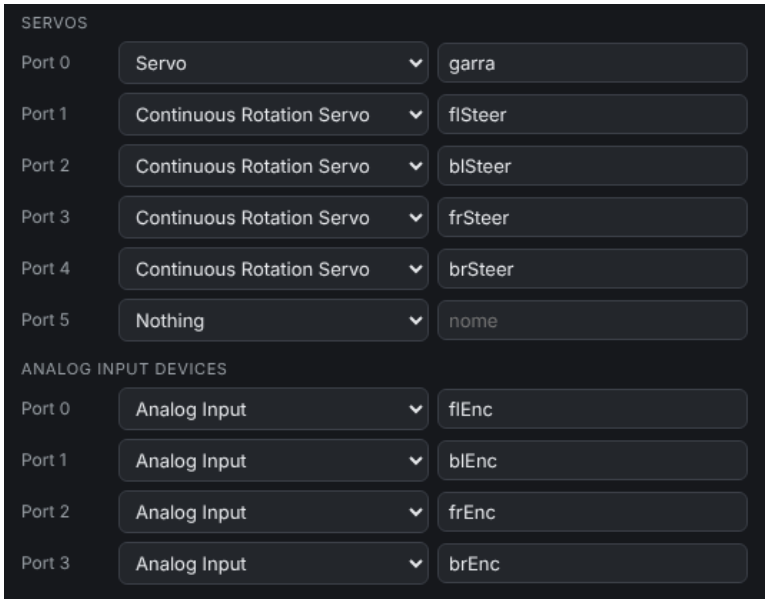
13.9 SWERVE NO DESENVOLVIMENTO COMPLETO

O swerve, que já existia no modo pontual, passou a existir também no desenvolvimento completo. Ao escolher "Swerve" no seletor de chassi, o robô ganha quatro módulos, cada um com três peças que o código precisa usar, como num robô real de FTC:

- **motor de tração** nas portas de motor 0 a 3 do Control Hub (módulos dianteiro esquerdo, traseiro esquerdo, dianteiro direito e traseiro direito). Os quatro módulos são iguais: com setPower(+1), a roda empurra para onde está apontando;
- **servo de esterçamento** (um Axon) nas portas de servo 1 a 4, na mesma ordem. Ele pode ser configurado como servo de rotação contínua (CRServo), em que setPower gira o módulo a até cerca de 600°/s, ou como servo de posição (Servo), em que setPosition de 0 a 1 leva o módulo de 0° a 355°. A porta 0 continua sendo a da garra;
- **encoder absoluto** do módulo nas entradas analógicas 0 a 3 (AnalogInput): 0 V com a roda apontando para a frente do robô, subindo até 3,3 V numa volta no sentido anti-horário.

O Driver Hub ganhou a seção Analog Input Devices na tela Configure Robot (Figura 33), na mesma posição em que ela aparece no Driver Hub real, e o Sandbox passou a simular as classes CRServo e AnalogInput, que também entraram na Referência do SDK e nas sugestões do editor. O ângulo dos módulos não se corrige sozinho: o código lê o encoder, calcula o erro até o ângulo desejado e controla o servo — normalmente com um controlador proporcional —, e também faz a cinemática inversa, que transforma o comando do controle (andar, andar de lado e girar) na velocidade e no ângulo de cada módulo. Como no robô real, quando o módulo precisaria virar mais de 90°, é mais rápido girar o ângulo complementar e inverter a roda.

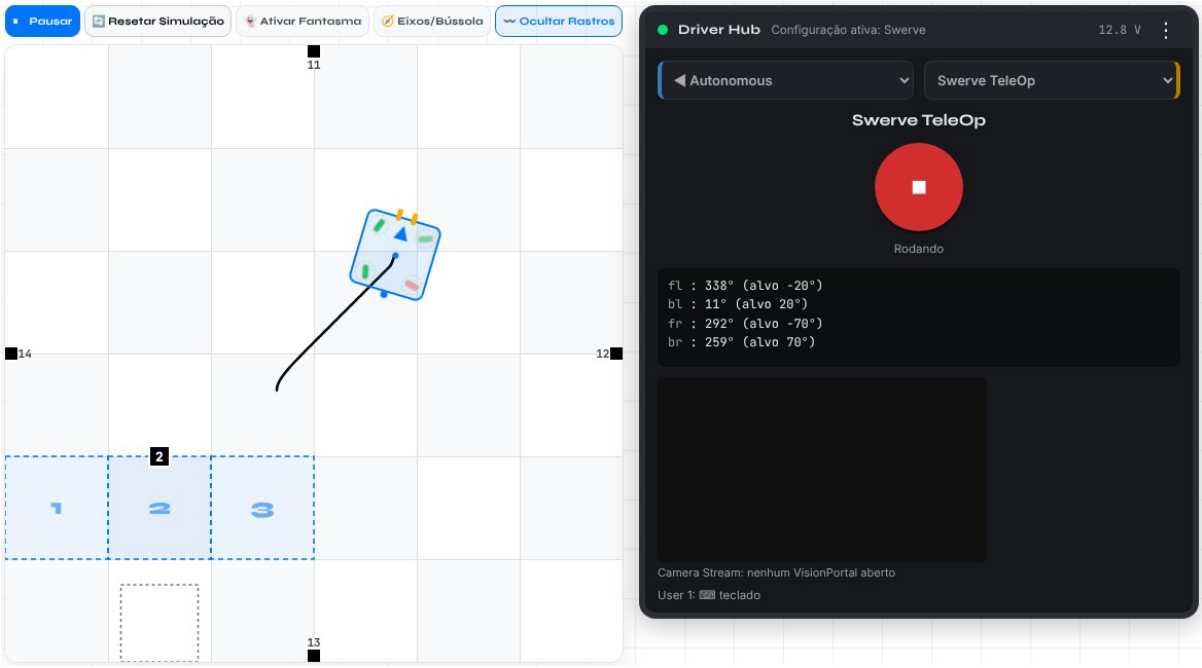
Figura 33 – Configuração do Driver Hub para o swerve: servos de esterçamento e encoders absolutos



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

O servo de esterçamento leva alguns ciclos para chegar à velocidade, e por isso um controle proporcional com ganho alto passa um pouco do alvo antes de parar, como acontece de verdade. Para o teste, foi escrito um TeleOp de swerve de referência, com cinemática inversa, inversão da roda e controle proporcional do ângulo de cada módulo (Figura 34). Configurado pela própria tela do Driver Hub, ele andou 40 pol. para a frente com os módulos a 0°; para andar de lado, os módulos viraram para 270° (chegando a 276° antes de assentar) e o robô andou 45 pol. para a direita sem girar; e, ao girar, os módulos ficaram tangentes ao círculo. A missão de direção (andar para a frente, de lado e girar 180°) foi concluída com o swerve. Com o esterçamento configurado como Servo de posição, setPosition(0,25) levou o módulo a 88,8°, a no máximo 10° por ciclo. Os testes também conferem a mensagem de erro quando o código pede Servo.class para um dispositivo configurado como CRServo, as sugestões do editor para as novas classes e a volta ao chassi do modo pontual ao trocar de modo.

Figura 34 – TeleOp de swerve em execução: os módulos viram para a direção pedida e a telemetria mostra o ângulo de cada um



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

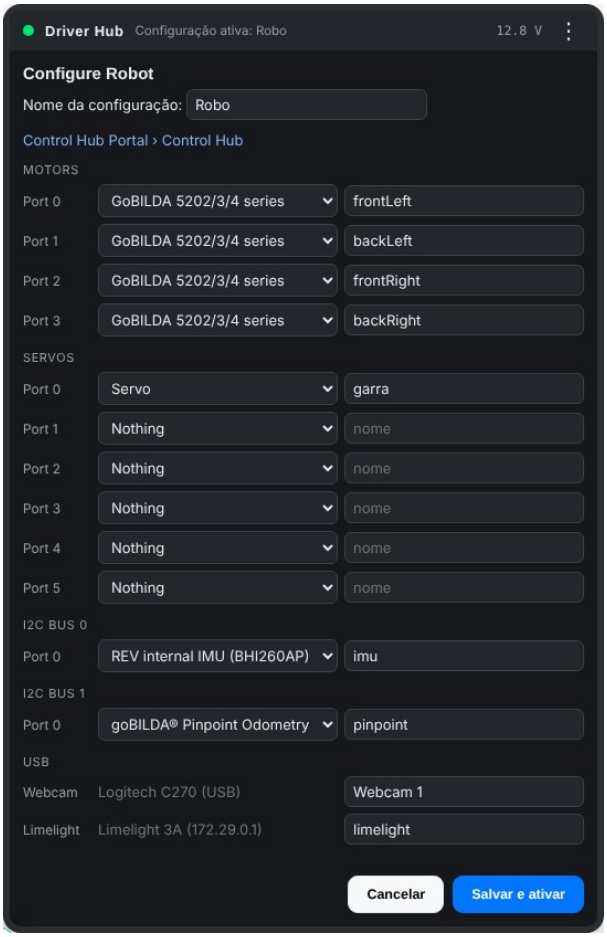
14. SENSORES DE LOCALIZAÇÃO: PINPOINT V2, PODS E LIMELIGHT 3A

A versão 23 acrescenta ao desenvolvimento completo (o antigo Robô Completo) os três sensores de localização mais usados pelas equipes: o computador de odometria goBILDA Pinpoint V2, pods de odometria individuais ligados às portas de encoder do Expansion Hub e a câmera inteligente Limelight 3A, que pode ser usada no lugar da Logitech C270 ou junto com ela. Todos usam as mesmas classes e métodos do FTC SDK e dos drivers dos fabricantes, e os erros mais comuns de montagem e de configuração aparecem no simulador como apareceriam no robô.

14.1 MONTAGEM DO ROBÔ E CONFIGURAÇÃO

Na configuração do robô, embaixo da vista 3D (Seção 17.8), escolhe-se o que está instalado: sem odometria, Pinpoint V2 com dois pods, ou três pods no Expansion Hub; o tipo de pod (goBILDA 4-Bar, com 19,894 ticks/mm, ou Swingarm, com 13,263 ticks/mm); e a câmera (C270, Limelight 3A ou as duas, escolhida ao lado da vista 3D, junto com a montagem de cada uma). A "Ficha do robô" descreve a montagem escolhida — por exemplo, que o pod X do Pinpoint fica 84 mm à direita do centro e que o pod Y foi montado invertido —, e a tela Configure Robot do Driver Hub ganha as entradas correspondentes: I2C Bus 1 para o Pinpoint (o barramento 0 é da IMU), o Expansion Hub 2 com as portas dos pods e a Limelight como dispositivo USB (Figura 35). Um dispositivo que está na configuração mas não está instalado gera o erro "Unable to find a hardware device…", com a explicação.

Figura 35 – Configure Robot com o Pinpoint no I2C Bus 1 e as duas câmeras USB



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

14.2 GOBILDA PINPOINT V2

O Pinpoint é simulado como no aparelho real: ele integra a posição sozinho, a partir dos dois pods e da sua própria IMU, e guarda essa posição entre um OpMode e outro enquanto o robô estiver ligado — o que permite levar a pose do autônomo para o TeleOp. O código usa a classe GoBildaPinpointDriver (com.qualcomm.hardware.gobilda) e os métodos setOffsets, setEncoderResolution, setEncoderDirections, resetPosAndIMU, update, getPosition (Pose2D), getPosX, getPosY, getHeading, getVelX, getEncoderX e getDeviceStatus. Os valores só mudam depois de update(), como no SDK, e o resetPosAndIMU deixa o aparelho em CALIBRATING por cerca de 0,25 s; nesse tempo o LED do Pinpoint, desenhado no robô, fica vermelho.

Os três parâmetros de configuração têm efeito físico (Tabela 26): a resolução converte ticks em milímetros; a direção de cada pod precisa fazer X crescer para a frente e Y para a esquerda; e os offsets dizem onde estão os pods, para que o giro do robô não seja confundido com deslocamento.

Tabela 26 – Efeito da configuração do Pinpoint, medido no simulador

Situação	Configuração certa	Configuração errada
Percurso de 120 pol. voltando ao início (erro final)	0,17 pol.	40,3 pol. (pod Y sem REVERSED)
Giro de 165° no lugar (posição que o Pinpoint informa)	(0,00; 0,00) pol.	(27,7; 9,6) pol. com os offsets de sinal trocado; (13,9; 4,8) pol. com offsets 0
Robô andando reto, sem girar	Posição correta	Offsets errados não aparecem: o erro só surge quando o robô gira

Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

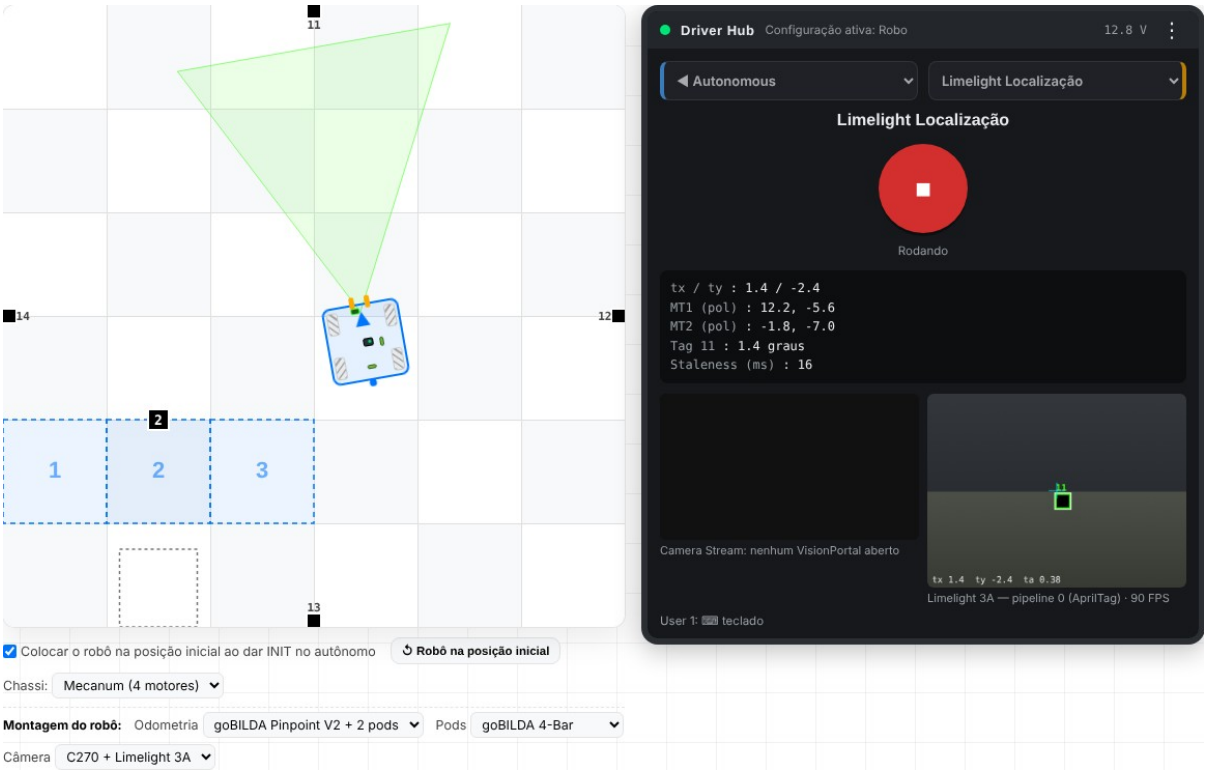
14.3 PODS DE ODOMETRIA INDIVIDUAIS

Na montagem com pods individuais, três pods ficam ligados às portas de encoder do Expansion Hub, sem motor: dois paralelos (par0, 152,4 mm à esquerda, e par1, 152,4 mm à direita, montado espelhado) e um perpendicular (perp, 139,7 mm atrás do centro). Cada um é lido como um "motor" configurado na porta, com hardwareMap.get(DcMotorEx.class, ...) e getCurrentPosition(); setDirection(REVERSE) inverte a contagem e STOP_AND_RESET_ENCODER a zera. Aqui o próprio estudante escreve a odometria de três rodas: a variação do heading vem da diferença entre os paralelos, o avanço da média deles e o deslocamento lateral do perpendicular, descontado o arco causado pelo giro. Como as rodas de odometria não são acionadas, elas não escorregam como as rodas mecanum, cujo strafe perde cerca de 10%; por isso os pods medem o movimento real do robô.

14.4 LIMELIGHT 3A: APRILTAGS, MEGATAG1 E MEGATAG2

A Limelight 3A é uma câmera com processamento próprio, ligada por USB. O código usa Limelight3A, LLResult, LLResultTypes e LLStatus (com.qualcomm.hardware.limelightvision): setPollRateHz, pipelineSwitch(0) e start(); depois, a cada volta, getLatestResult(), isValid(), getTx(), getTy() e getTa() (graus e porcentagem da imagem, com tx positivo à direita), getFiducialResults() para cada tag e getStaleness(). O pipeline 0 do Sandbox é o de AprilTags, com o mapa das tags do campo escolhido (11 a 14 no campo de treino; 30 a 45, na posição do começo da partida, no BIOBUZZ), o que habilita as duas formas de localização da Limelight: o MegaTag1 (getBotpose), que calcula a pose do robô só a partir das tags, e o MegaTag2 (getBotpose_MT2), que usa o yaw informado pelo código com updateRobotOrientation. As poses vêm em Pose3D, em metros, com origem no centro do campo. O Driver Hub mostra a imagem da Limelight com a mira, os contornos das tags e os valores de tx, ty e ta (Figura 36).

Figura 36 – Limelight 3A: MegaTag1 correto e MegaTag2 deslocado por um yaw informado com 10° de erro



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

A figura mostra um erro típico de MegaTag2. O robô foi iniciado virado 10° para a direita do norte, mas o código informou o yaw como "IMU + 90°", supondo que ele começa virado exatamente para o norte. O MegaTag1, que não depende do yaw informado, deu (12,2; −5,6) pol. para uma posição real de (12; −6). O MegaTag2 deu (−1,8; −7,0) pol.: cerca de 13 pol. de erro para 10° de yaw errado com a tag a uns 75 pol. O MegaTag2 é mais estável que o MegaTag1, mas só funciona com o yaw do robô no referencial do campo.

14.5 VALIDAÇÃO

O teste automatizado da versão 23 escolhe cada montagem pela própria tela, configura o robô e roda programas de referência. Com o Pinpoint, um percurso de 120 pol. que volta ao início terminou a 0,17 pol. do ponto de partida. A mesma odometria com o pod Y na direção errada terminou a 40 pol. Os offsets errados só apareceram quando o robô girou no lugar. Com três pods e odometria escrita pelo estudante, o percurso também terminou a 0,17 pol. A Limelight deu pose MegaTag1 a menos de 0,5 pol. da real, com a tag 11 a 65 pol., e a missão correspondente foi concluída. Um Pinpoint configurado num robô que não o tem gerou a mensagem esperada. Os testes das versões anteriores (Sandbox, Mapa de Estudos, Caminho do André, desafios de código, Robô Completo e Monaco) continuam passando, sem erros de página.

15. VISÃO EM 3D, CAMPO BIOBUZZ E MODO PONTUAL DE VISÃO

Na versão 25, a visão computacional do Sandbox ganhou três peças. A primeira é o campo do jogo da temporada 2026-2027, BIOBUZZ, com a estrutura das HIVEs e as AprilTags nas posições do Manual de Competição. A segunda é uma vista 3D do campo, desenhada com o motor gráfico do próprio autor — a reta de Bresenham e a função rotatePoint —, em que se pode andar em volta da arena ou ver o que cada câmera do robô captura. A terceira é um modelo novo no desenvolvimento pontual, para treinar a localização por visão com a Limelight 3A, com a Logitech C270 ou com as duas, com a odometria pronta ou escrita pelo estudante. A mesma versão trouxe as peças do jogo, os mecanismos do robô e a física, descritos na Seção 16.

15.1 O CAMPO BIOBUZZ E AS APRILTAGS DAS HIVES

No BIOBUZZ, as AprilTags não ficam nas paredes: ficam embaixo das CELLs das duas HIVEs, no centro do campo, viradas para o chão. Segundo a seção 9 do manual, a estrutura das HIVEs tem 49,46 pol. de largura e 38,95 pol. de profundidade na base, com os pivôs a 43,95 pol. do chão; cada HIVE (uma vermelha e uma azul) tem duas CELLs, a cerca de 18,8 pol. uma da outra, com abertura de cerca de 20 × 14 pol. e 12 pol. de fundo, e é biestável: ela tomba para um lado ou para o outro, sempre com uma CELL virada para cima. No início da partida, a HIVE vermelha fica com a CELL do lado da plateia para cima e a azul com a do lado oposto. As tags são da família 36h11, com 3,25 pol., em grupos de quatro num mesmo adesivo, com a borda de baixo virada para o centro do campo, em dois pares de 6,5 pol. separados por um vão de 7,0 pol. (Figura 9-15; a leitura dessas cotas é discutida na Seção 16.1) (Tabela 27).

Tabela 27 – AprilTags do campo BIOBUZZ (Manual de Competição 2026-2027, seção 9.9 e Figura 9-17)

IDs (da esquerda para a direita, vistos de cima)	HIVE e CELL	Altura no Sandbox (início da partida)
33, 32, 31, 30	Vermelha, lado oposto à plateia	39,7 pol. (CELL de baixo)
34, 35, 36, 37	Vermelha, lado da plateia	49,4 pol. (CELL de cima)
38, 39, 40, 41	Azul, lado da plateia	39,7 pol. (CELL de baixo)
45, 44, 43, 42	Azul, lado oposto à plateia	49,4 pol. (CELL de cima)

Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

Na extração do texto do manual, os dígitos de vários IDs se perdem (aparecem só "0" e "7" nos grupos vermelhos); os quatro grupos foram conferidos com a transcrição da Figura 9-17 feita pelo projeto aberto ViDAR, que cita o manual e o CAD oficial do campo. O desenho de cada tag não é um quadrado genérico: ele vem da

tabela de códigos da família 36h11 da biblioteca AprilTag (a mesma usada pelo FTC SDK), de modo que cada tag na tela tem o padrão de bits da tag impressa. As FLOWERs ficam a 23,39 pol. do meio de cada parede (distância do CAD), com a abertura de cima a 21,5 pol. do chão; o lado de cada uma foi conferido com a figura das posições iniciais do manual (Figura 11-1), que também mostra as GARDENs e as LOADING ZONEs (Seção 16.2).

A forma das HIVEs segue um modelo 3D feito pelo autor a partir do CAD do campo, com as medidas acima: cada CELL é uma casinha transparente nas pontas de uma barra reta que gira no pivô (Seção 16.1). Os botões da vista 3D escolhem o estado de cada HIVE no começo da partida; durante o jogo ela vira pela física, e as tags andam junto. O campo de treino das versões anteriores (tags 11 a 14 nas paredes e o sinal do autônomo) continua disponível e é o padrão do desenvolvimento completo, cujas missões dependem dele; o campo é escolhido no alto da vista 3D.

15.2 O MOTOR 3D: A RETA DE BRESENHAM E O ROTATEPOINT

O motor 3D foi escrito sem WebGL, sobre as duas funções do autor: a reta de Bresenham (calculateLine), que acha os pixels de uma reta só com somas e comparações de inteiros, e a rotatePoint, que gira um ponto em torno de um centro, primeiro no eixo vertical (ângulo a) e depois no horizontal (ângulo b). As duas foram adaptadas, sem mudar o algoritmo: a reta passou a chamar uma função para cada pixel em vez de montar uma lista, e informa a fração já percorrida, usada para a profundidade; e a rotatePoint passou a receber o centro e os senos e cossenos já calculados, uma vez por quadro (Código 7).

Código 7 – A reta de Bresenham e o rotatePoint adaptados (trecho de view3d.js)

```
function bresenham(x0, y0, x1, y1, plot) {
    const steep = Math.abs(y1 - y0) > Math.abs(x1 - x0);
    if (steep) { /* troca x e y */ }
    if (x0 > x1) { /* troca as pontas */ }
    const dx = x1 - x0; let dy = y1 - y0;
    const dir = dy < 0 ? -1 : 1; dy *= dir;
    let y = y0, p = 2 * dy - dx;
    for (let i = 0; i <= dx; i++) {
        if (steep) plot(y, x0 + i, i / dx); else plot(x0 + i, y, i / dx);
        if (p >= 0) { y += dir; p -= 2 * dx; }
        p += 2 * dy;
    }
}
function rotatePoint(p, c, R) {   // R = senos e cossenos de a e b
    const xt = p.x - c.x, yt = p.y - c.y, zt = p.z - c.z;
    return { x: c.x + (xt * R.ca + zt * R.sa),
             y: c.y + (xt * R.sb * R.sa + yt * R.cb - zt * R.sb * R.ca),
             z: c.z + (-xt * R.cb * R.sa + yt * R.sb + zt * R.cb * R.ca) };
}
```

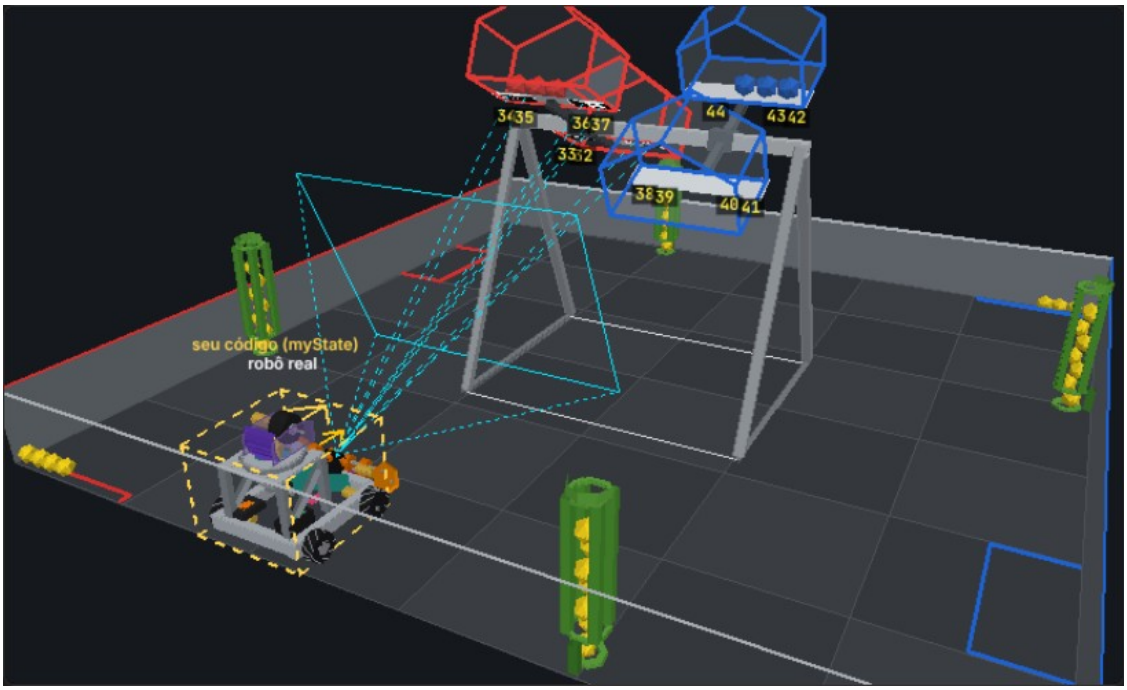
Cada ponto do campo (x, y no chão, z para cima) passa para o espaço do motor (y para cima), é girado pelo rotatePoint e projetado em perspectiva: x e y na tela são divididos pela profundidade. Antes de desenhar, cada polígono é recortado pelo plano de perto (para não desenhar o que está atrás do olho) e pela borda da imagem. O preenchimento usa a própria reta de Bresenham: as bordas do polígono são traçadas com ela e, em cada linha da tela, os pixels entre o menor e o maior x são pintados. Para que o que está na frente esconda o que está atrás, cada pixel guarda 1/z num z-buffer, e um pixel só é pintado se estiver mais perto que o já desenhado (1/z varia em linha reta na tela, então pode ser interpolado ao longo das bordas e das linhas). As retas (estrutura, FLOWERs, raios e contornos) também são traçadas com Bresenham e testadas no z-buffer, e as tags são desenhadas como a grade 10 × 10 do código 36h11, em faixas de células pretas.

As duas vistas usam a mesma função. Na arena, o centro do giro é o ponto para onde se olha, e o olho fica a uma distância D dele. Na primeira pessoa, o centro é a lente da câmera, o giro a é o heading do robô mais o yaw da câmera menos 90° (para que a frente vire a profundidade), o giro b é o pitch da câmera e o roll é aplicado como um giro da imagem em torno do centro. O vidro das CELLs é desenhado por último, da face mais longe para a mais perto, misturando 16% da cor dele com a do pixel e sem escrever no z-buffer. Com o campo BIOBUZZ completo — o robô, as 44 peças e o vidro —, um quadro de 760 × 460 pixels leva em torno de 7 ms na arena e 4 ms na primeira pessoa (medido no computador dos testes), e a vista é redesenhada até 30 vezes por segundo, só quando está na tela.

15.3 A VISTA 3D: ARENA, CÂMERAS E ROBÔ ESTIMADO

A vista 3D aparece no desenvolvimento completo, entre a arena e o editor, e no desenvolvimento pontual quando o modelo é o de visão (Figura 37). No modo Arena, arrastar gira a vista, a roda do mouse ou a pinça aproxima, Shift + arrastar (ou dois dedos) move o ponto de interesse, e "Seguir o robô" mantém o robô no centro. O robô é um modelo de poucos polígonos, com a frente marcada por uma seta amarela (Seção 16.3); as câmeras instaladas aparecem no robô, com o campo de visão desenhado, e um raio tracejado liga a câmera a cada tag detectada. Os números das tags aparecem quando elas estão visíveis.

Figura 37 – Vista 3D no modelo de visão: o robô real, o robô estimado pelo código (tracejado), o campo de visão da C270 e os raios até as tags das HIVEs



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

Com "Seguir o robô", a vista vira uma câmera em terceira pessoa: o ponto de interesse acompanha o robô e, com a opção "girar junto com o robô", o ângulo da câmera passa a ser medido a partir de trás dele, de modo que a vista gira quando o robô gira. A câmera pode ser girada, inclinada e aproximada sem o mouse: no teclado, J e L giram, I e K sobem e descem e U e O aproximam e afastam; no controle, o analógico direito do gamepad2 gira e inclina e LB e RB aproximam e afastam; e, com a opção "câmera no gamepad1" ligada, o D-pad ◀️ ▶️ do gamepad1 gira e o analógico direito ↕ inclina (com R3 apertado, aproxima). As teclas escolhidas não são usadas pelo Sandbox nem pelo exemplo; os botões do gamepad1 podem ser desligados se o OpMode do estudante precisar deles.

O robô tracejado é onde o código acha que está. No modelo de visão do modo pontual, é o myState devolvido pelo código. No desenvolvimento completo, são as estimativas dos sensores — o Pinpoint, a Limelight (MegaTag) e o robotPose da C270 — e a do próprio OpMode, quando ele manda a pose na telemetria com as legendas x, y e heading. Os modos C270 e Limelight mostram a imagem de cada câmera, com o contorno verde e o ID de cada tag detectada (Figura 38); a mesma imagem passou a ser usada no Camera Stream e na tela da Limelight do Driver Hub. Na arena 2D, o campo BIOBUZZ aparece visto de cima, com as CELLs, as tags, as FLOWERs, as GARDENs, as áreas de PARK e as peças.

Figura 38 – O que a Limelight 3A captura, inclinada 25°



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

15.4 A INCLINAÇÃO DAS CÂMERAS E O QUE ELAS VEEM

Cada câmera tem a montagem completa no robô, em seis graus de liberdade (Seção 16.5); por padrão, a C270 fica a 8 pol. à frente do centro e 6 pol. do chão, e a Limelight a 7 pol. à frente e 9 pol. do chão, ambas inclinadas 25° para cima no campo BIOBUZZ. Uma tag é detectada quando o centro dela cai dentro da imagem, quando ela está de frente para a câmera (até 72° entre a normal da tag e a direção da câmera), dentro do alcance (110 pol. na C270, 130 na Limelight), com pelo menos 9 pixels de lado na imagem e sem nenhuma peça opaca do campo entre a lente e a tag — o vidro das CELLs não esconde nada.

No campo BIOBUZZ, a inclinação decide o que a câmera vê (Figura 39). No começo da partida, as tags estão a 39,7 pol. (CELLs de baixo) e 49,4 pol. (CELLs de cima) do chão, embaixo do piso das CELLs; com a câmera reta, a 4 pés do centro, nenhuma tag entra na imagem. Inclinada 25°, a C270 vê as tags das duas CELLs de baixo (30 a 33 e 38 a 41); para alcançar as da CELL de cima, logo acima do robô, é preciso inclinar mais (com 55°, as tags 34 a 37 também entram). A detecção devolve as medidas do espaço da câmera, já com a montagem: no ftcPose, x para a direita, y para a frente da lente e z para cima da imagem, além de range, bearing, elevation e yaw, pitch e roll da tag; na Limelight, tx, ty e ta. A biblioteca getCurrentGameTagLibrary() passa a ter as 16 tags com nome ("Red Audience 34") e posição no campo, e o setCameraPose do construtor do AprilTagProcessor liga o robotPose de cada detecção, como no SDK.

Figura 39 – A mesma posição com a C270 reta (à esquerda) e inclinada 25° (à direita)



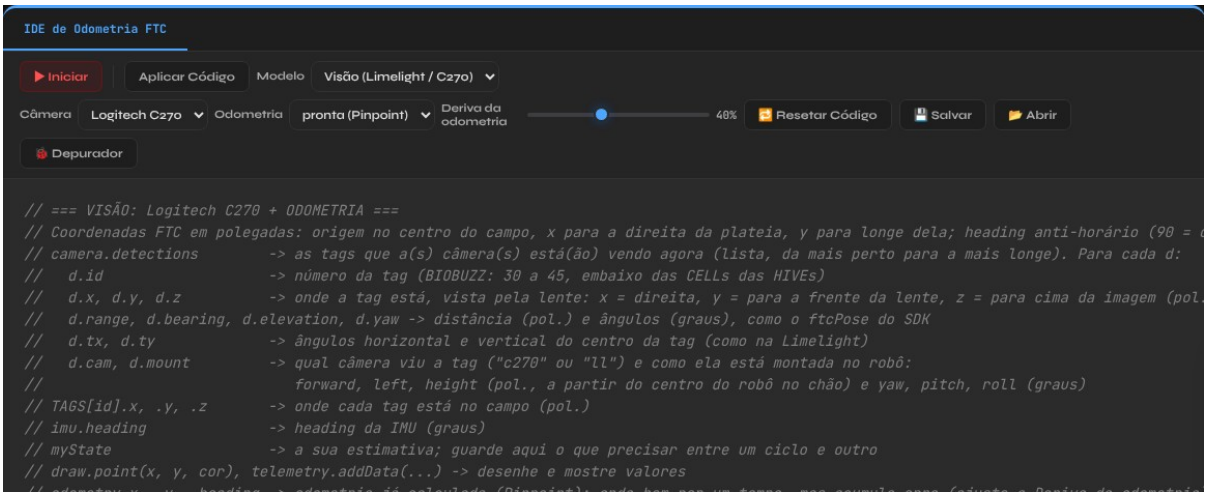
Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

No campo de treino, a C270 fica na altura das tags das paredes e, com inclinação 0°, as medidas são as mesmas das versões anteriores: os testes do desenvolvimento completo (autônomo com o sinal, TeleOp com alinhamento à tag 11, Limelight com MegaTag1 e MegaTag2) deram os mesmos números da versão 24.

15.5 MODO PONTUAL DE VISÃO: LIMELIGHT OU C270

O modelo "Visão (Limelight / C270)" do desenvolvimento pontual coloca o robô na posição inicial escolhida (por padrão, a 48 pol. do centro, de frente para as HIVEs), em coordenadas FTC, e entrega ao código, a cada ciclo: camera.detections, com as tags que as câmeras escolhidas veem (id, x, y, z, range, bearing, elevation, yaw, tx, ty, ta, a câmera que viu e a montagem dela), TAGS[id] com a posição de cada tag no campo e, na opção "odometria pronta", odometry com a pose do Pinpoint. Na opção "eu escrevo", a odometria também é do estudante, a partir das rodas de odometria. Um controle de deriva faz a odometria pronta acumular erro com a distância percorrida, como rodas que escorregam, para que a visão tenha o que corrigir (Figura 40).

Figura 40 – O modelo de visão no desenvolvimento pontual: câmera, tipo de odometria e deriva



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

O código-modelo já tem a estrutura: odometria, cálculo da pose do robô a partir da tag mais próxima e mistura das duas. Ele desfaz o roll e o yaw da câmera e deixa para o estudante justamente a inclinação (pitch): d.y é medido ao longo da lente, que está inclinada, e precisa virar a distância no chão antes de ser girado pelo heading (Código 8).

Código 8 – Pose do robô a partir de uma tag, no modelo de visão (com a correção do pitch)

```
var m = d.mount;                                    // forward, left, height, yaw,
pitch, roll
double r = m.roll * Math.PI / 180;                  // a) desfaz o roll
double cx = d.x * Math.cos(r) + d.z * Math.sin(r);
double cz = -d.x * Math.sin(r) + d.z * Math.cos(r);
double p = m.pitch * Math.PI / 180;                 // b) pitch: distância no chão
double ahead = d.y * Math.cos(p) - cz * Math.sin(p);
double yw = m.yaw * Math.PI / 180;                  // c) desfaz o yaw
double fwdR = ahead * Math.cos(yw) + cx * Math.sin(yw);
double leftR = ahead * Math.sin(yw) - cx * Math.cos(yw);
double fwd = m.forward + fwdR, left = m.left + leftR;
double visX = t.x - (fwd * Math.cos(hr) - left * Math.sin(hr));
double visY = t.y - (fwd * Math.sin(hr) + left * Math.cos(hr));
myState.corrX += k * (visX - x);                    // mistura com a odometria
myState.corrY += k * (visY - y);
```

15.6 VALIDAÇÃO

Um teste automatizado dirige o robô no modelo de visão por um percurso fixo de 10 segundos, com a deriva da odometria em 60%. Só com a odometria, o erro médio foi de 3,5 pol. e terminou em 7,6 pol.; com a solução do Código 8 e mistura de 0,1, o erro médio caiu para 0,3 pol. (no máximo 1,5 pol.). O código-modelo sem a correção do pitch errou em média 12,8 pol. — a diferença mostra por que a inclinação importa. As combinações de câmera (C270, Limelight ou as duas) e de odometria (pronta ou escrita) compilam e rodam nos idiomas testados, e o botpose da Limelight ficou a menos de 1 pol. da pose real.

No desenvolvimento completo, com o campo BIOBUZZ, um TeleOp de referência com a C270 (inclinada 25°) e a Limelight viu as tags 30 a 33 e 38 a 41, recebeu os nomes do manual e a altura da tag no ftcPose; o robotPose da C270 e o MegaTag1 da Limelight ficaram a menos de 1 pol. da pose real, e as quatro estimativas apareceram na vista 3D. Com a câmera reta, nenhuma tag entrou na imagem; com 55°, entraram também as da CELL de cima. Outros testes conferem os controles da vista (girar, aproximar, mover, voltar), a troca de câmera, a montagem guardada, o tombamento das HIVEs, a opção que esconde a vista, a troca de campo bloqueada com o OpMode rodando e a largura de celular. Todos os testes anteriores — Sandbox, Mapa de Estudos (365 de 372 desafios resolvidos pela interface, como

antes), Caminho do André, desafios de código, depurador e desenvolvimento completo — continuam passando, sem erros de página.

16. PEÇAS DO JOGO, MECANISMOS DO ROBÔ E FÍSICA

Além da visão, a versão 25 tornou o campo BIOBUZZ jogável. As HIVEs seguem a forma do jogo; entraram as peças; o robô ganhou intake, midtake, lançador (cinco tipos, em três montagens) e braço; e uma física simples, mas coerente, liga tudo isso. As câmeras têm a montagem completa, em seis graus de liberdade, e é possível usar só a C270, só a Limelight ou as duas. O robô pode começar em qualquer das posições iniciais permitidas pelas regras ou numa posição de treino, e um TeleOp de exemplo deixa testar tudo pelo teclado.

16.1 AS HIVES: CELLS EM FORMA DE CASINHA NUMA BARRA RETA

O texto do manual dá as medidas da HIVE, mas não a forma. Para chegar a ela, o autor montou um modelo 3D a partir do CAD do campo e de fotos da estrutura, e o Sandbox o segue. Cada CELL é uma "casinha": um prisma de seção pentagonal — piso, duas paredes e um telhado de duas águas — deitado ao longo de uma barra reta e alinhado com ela. A ponta de fora é a entrada, aberta; a ponta de dentro, virada para o pivô, é o fundo (a Back Skin); piso, paredes, telhado e fundo são de policarbonato transparente, com a armação na cor da aliança. As duas CELLs da mesma cor ficam nas pontas da barra, que passa pelo pivô: quando uma desce, a outra sobe. Com as medidas da seção 9.6, a seção da CELL tem 20 pol. de largura, paredes de 8,5 pol. e a cumeeira a 14 pol. do piso, com 12 pol. de fundo; o fundo de cada CELL fica a 9,4 pol. do pivô, de modo que as duas ficam a 18,8 pol. uma da outra (Tabela 28).

Tabela 28 – A HIVE no manual e no Sandbox

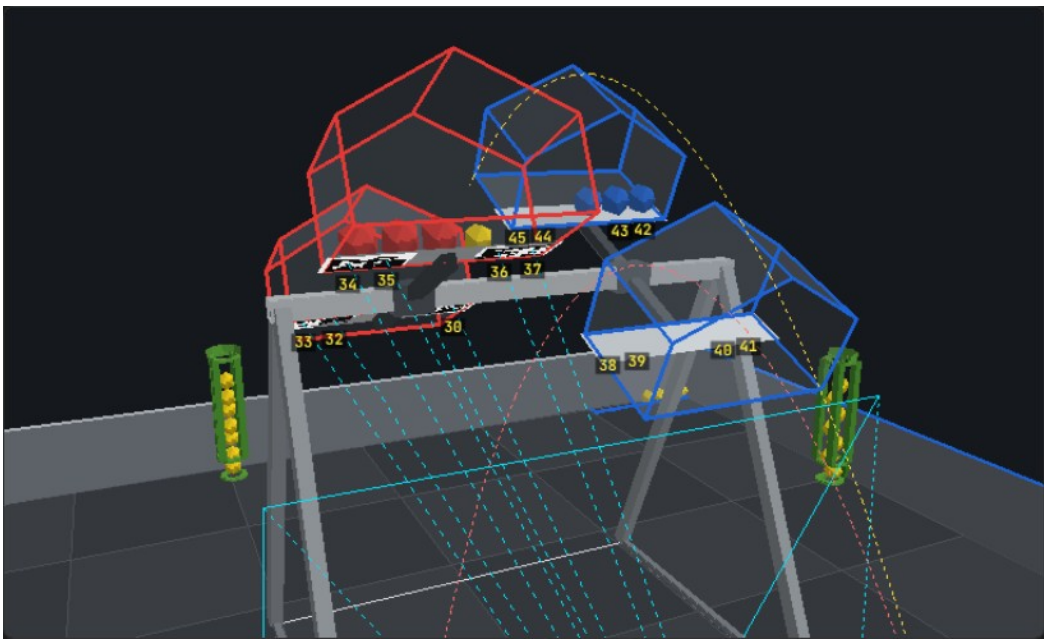
Medida	Manual (seção 9.6)	Sandbox
Base da estrutura	49,46 × 38,95 pol.	igual; cavaletes em A em x = ±24,73 pol.
Altura dos pivôs	43,95 pol.	igual
Abertura da CELL	cerca de 20 × 14 pol., 12 pol. de fundo	seção pentagonal: 20 pol. de largura, paredes de 8,5 e cumeeira a 14 pol.; 12 pol. de fundo
Distância entre as CELLs	cerca de 18,8 pol.	18,8 pol. (fundo de cada uma a 9,4 pol. do pivô)
Estados	biestável, uma CELL para cima	barra a ±25°; vira pela física (Seção 16.4)
Altura no começo da partida	—	CELL de cima de 48,6 a 66,4 pol.; CELL de baixo de 35,6 a 53,4 pol.
Tags de cada CELL	4 tags 36h11 de 3,25 pol. embaixo da CELL	embaixo do piso, viradas para o chão; 2 pares encostados com 7,0 pol. de vão

Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

Nos dois estados estáveis, a barra está girada 25° para um lado. A CELL de cima fica com a entrada virada para fora e para cima, e o piso inclinado para dentro: uma peça que entra escorrega até o fundo, que é onde o guia de montagem manda encostar os 3 NECTAR do começo. A CELL de baixo fica com a entrada virada para fora e para baixo; por isso, quando a HIVE vira, a CELL que desce despeja no campo tudo o que tinha. O comprimento e a inclinação da barra não estão no manual e foram estimados a partir do modelo do autor.

As tags ficam embaixo do piso, viradas para o chão, numa placa cinza, com a borda de baixo para o centro do campo, e a linha delas fica a 9,938 pol. da borda da entrada (a cota da linha dos furos de referência). Lidas como dois pares de tags encostadas (6,5 pol. por par, ou duas tags de 3,25 pol.) separados por 7,0 pol., as cotas da Figura 9-15 dão um grupo de 20 pol., a mesma largura da CELL; lidos os 6,5 pol. como a distância entre os centros, o grupo teria 23,25 pol. e passaria da largura do piso. Como as HIVEs giram, as tags andam com elas (Figura 41).

Figura 41 – As HIVEs: CELLs em forma de casinha, transparentes, com a armação na cor da aliança, ligadas por uma barra reta que passa pelo pivô; os 3 NECTAR no fundo de cada CELL de cima e as tags embaixo do piso



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

16.2 POLLEN, NECTAR, FLOWERS, GARDENS E PARK

POLLEN e NECTAR são bolas de polietileno de cerca de 2,8 pol. (amarelas) e 3,6 pol. (vermelhas e azuis). O manual prevê 40 POLLEN e 8 NECTAR de cada cor por partida e não informa a massa: o Sandbox usa 30 g e 50 g, estimados. Para não pesar no desenho, cada peça é um icosaedro, com 20 faces triangulares, no diâmetro oficial; de longe, quando ocuparia menos de 2 pixels de raio, vira um quadradinho.

A montagem segue o Event Field Setup Guide e a figura das posições iniciais do manual (Figura 11-1). Cada CELL que começa para cima tem 3 NECTAR da cor da HIVE, encostados no fundo e na parede do lado da parede do campo. Cada GARDEN, uma fita de 23 × 2 pol. em cantos opostos (a vermelha no canto da plateia à esquerda, a azul no canto oposto à direita), tem 4 POLLEN em linha, encostados nas paredes. As FLOWERs têm POLLEN empilhado em zigue-zague dentro dos quatro tubos: o guia manda 4 POLLEN por FLOWER, mas a pedido do autor o padrão do Sandbox é a FLOWER cheia (7), com a montagem oficial como opção; o robô começa com 2 POLLEN carregados (de 0 a 3, também como opção). Cada FLOWER é uma peça sólida, com anéis vazados: o de baixo tem o furo de 2,79 pol. onde o POLLEN assenta, e o do meio e o de cima, a 21,5 pol., deixam a abertura de 4 pol. livre por dentro; o anel de cima tem o batente de 1,25 pol. do lado da parede, e os quatro tubos verdes de 1,05 pol. formam um quadrado de 3,45 pol. A abertura de retirada, de 3,55 pol., fica entre o anel de baixo e o do meio, virada para o campo.

O PARK vale pontos no autônomo e no fim da partida quando o robô está pelo menos em parte dentro da LOADING ZONE da sua aliança (seção 10.5.4). Cada LOADING ZONE é um retângulo de 23 × 11 pol. encostado na parede da aliança, dentro de um tile — a vermelha na metade de cima da parede vermelha, a azul na metade de baixo da parede azul, como na Figura 11-1 —, marcado com fita na cor da aliança na arena 2D e na vista 3D. O painel do jogo informa se o robô está em PARK. A posição inicial do robô é escolhida no mesmo painel: há três posições para cada aliança, encostadas na parede do lado da aliança, fora da LOADING ZONE e longe das FLOWERs (regra G304), e duas de treino, no centro do campo e de frente para as HIVEs (Figura 42).

Figura 42 – O campo visto de cima: HIVEs, FLOWERs nas quatro paredes, GARDENs nos cantos opostos, as áreas de PARK (LOADING ZONEs) e as peças



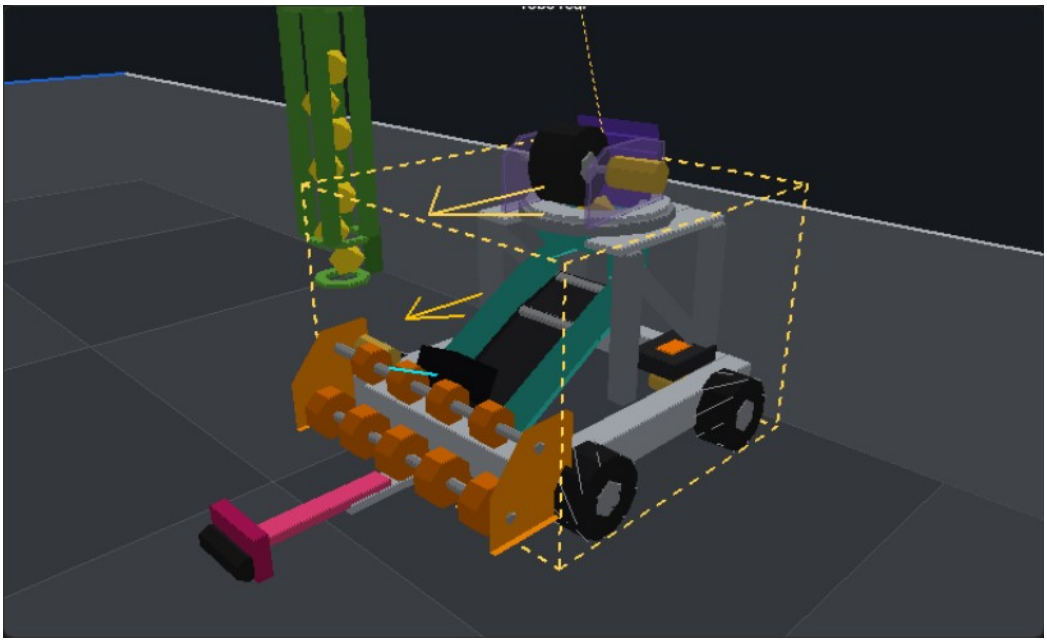
Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

16.3 O ROBÔ: INTAKE, MIDTAKE, SHOOTER E BRAÇO

O robô é um modelo de poucos polígonos (cerca de 700 faces), com 18 × 18 pol. e no máximo 18 pol. de altura, desenhado como um robô de FTC de verdade: chassi de perfis de alumínio, quatro rodas mecanum de 104 mm com os motores goBILDA por dentro dos perfis, Control Hub e bateria, e uma torre de colunas com reforços diagonais que segura a torreta. As peças que seriam impressas em 3D têm a cor do mecanismo (Figura 43):

- **Intake (laranja):** dois eixos com rodas flexíveis entre placas laranja, na borda da frente, que puxam as peças que encostam neles.
- **Midtake (verde-água):** uma correia em rampa entre roletes, com guias verde-água, que leva as peças do intake até a torre, e, no fim, uma subida vertical pelo furo da torreta, em que duas rodas flexíveis empurram a peça para cima; cabem até quatro peças no robô (POLLEN, NECTAR ou misturadas). A subida é vertical para que a peça chegue ao lançador do mesmo jeito em qualquer ângulo da torreta.
- **Shooter (roxo):** em cinco tipos e três montagens, à escolha no painel do jogo (Seção 16.4): capô fixo, capô móvel com servo, dois volantes sem capô e dois lançadores em linha — dois discos ou uma roda em cima —, numa torreta que gira 270° (±135°), fixo apontando para a frente ou, nos lançadores em linha, fixo atrás. O lançador de dois discos é sempre fixo atrás: ele atira por cima da traseira, seguindo a rampa. As placas laterais são de policarbonato transparente, como em muitos robôs de FTC, e deixam ver a peça passando por dentro.
- **Braço (rosa):** um trilho linear de três estágios que sai por baixo do intake até 13 pol. à frente do robô, rente ao chão, com uma garra e um rolo na ponta.

Figura 43 – O robô: intake (laranja), midtake (verde-água), shooter na torreta (roxo) e braço estendido (rosa)



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

No desenvolvimento completo, cada mecanismo está ligado a uma porta do Expansion Hub e é comandado pelo OpMode, como no robô real (Tabela 29). A tela de configuração do Driver Hub mostra as portas de motor e de servo do Expansion Hub

quando o campo é o BIOBUZZ, e a aba Robô e a Referência do SDK descrevem a fiação. No modelo de visão do pontual, os mecanismos são comandados pelo teclado ou pelo controle (Espaço = intake, X = alimentar o lançador, Y = liga e desliga o volante, ↑ e ↓ = força do volante, Q e E = torreta, Z e C = capô móvel, B = braço).

Tabela 29 – Fiação dos mecanismos no Expansion Hub 2

Porta	Mecanismo	Comando
Motor 0	Intake	setPower(+1) puxa; −1 cospe a peça que está na boca
Motor 1	Midtake	setPower(+1) leva as peças ao lançador (é ela que "atira"); −1 devolve
Motor 2	Shooter 2, nos lançadores de duas rodas (goBILDA 6000 rpm)	o mesmo do motor 3; montado de frente para ele: precisa de setDirection(REVERSE)
Motor 3	Shooter (goBILDA 6000 rpm, 28 ticks por volta)	setPower ou setVelocity (ticks/s, até 2800); getVelocity
Servo 0	Torreta	setPosition(0,5) = frente; 0,0 e 1,0 = 135° para a direita e para a esquerda
Servo 1	Braço	setPosition(0) recolhido; setPosition(1) estendido 13 pol.
Servo 2	Rolo do braço (Continuous Rotation Servo)	setPower(+1) segura a peça que a ponta encosta; −1 solta
Servo 3	Capô móvel	setPosition(0) = saída a 40°; setPosition(1) = 75°

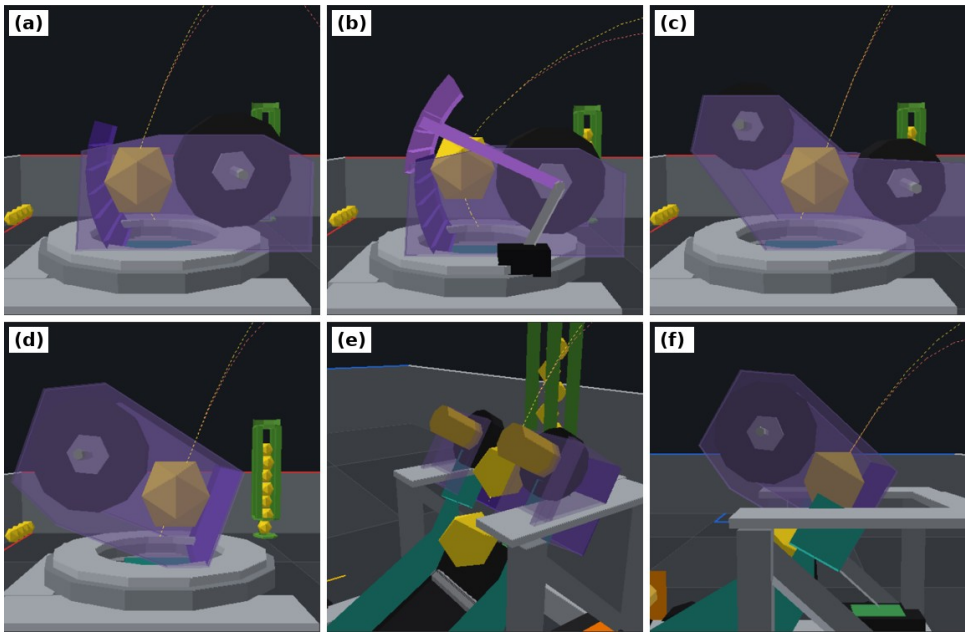
Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

Os cinco lançadores levam a peça por caminhos diferentes (Figura 44). Nos dois com capô, a folga entre a roda (raio de 2 pol.) e o capô (raio de 4,6 pol.) é de 2,6 pol., menor que a peça (POLLEN de 2,8 pol. e NECTAR de 3,6 pol.): a peça é apertada e rola entre a borda da roda, que se move, e o capô, que está parado, e o centro dela anda num arco de raio 3,3 pol. em volta do eixo, a partir de 215°. Ela sai pela ponta do capô, na direção da tangente ao arco, e por isso é o ponto em que o capô termina que define o ângulo de saída: no capô fixo, a ponta fica em 160°, e a peça sai a 160° − 90° = 70°. No capô móvel, a parte fixa termina em 170° e um segundo trecho curvo, por fora, gira em volta do eixo do volante, levado por um servo; ele põe a ponta entre 130° e 165°, e a saída fica entre 40° e 75°. Com o capô mais baixo, a peça dá mais volta na roda e sai mais deitada, o que alcança mais longe com a mesma rotação até 45°.

Sem capô, dois volantes de 3 pol., um em cima e outro embaixo da peça, apertam-na pelos dois lados; cada um tem o seu motor, e ela sobe reta entre eles, a 70°. Nos lançadores em linha, a peça continua na direção em que vem e passa reta pelas rodas: dois discos de 3 pol., um de cada lado, cada um com o seu motor, como numa máquina de arremesso de beisebol; ou uma roda de 4 pol. em cima, que a aperta

contra uma placa embaixo. O de dois discos é sempre fixo atrás; o de uma roda em cima pode ir na torreta, fixo na frente ou fixo atrás. Na torreta ou fixa na frente, a roda em cima recebe a peça da subida vertical, com uma curva de 20° antes da roda, e atira a 70°. Fixos atrás, a esteira não tem a subida: a rampa continua reta até as rodas, já inclinada a 60°, e a peça sai por cima da traseira do robô a 60°. Em todos, a trajetória prevista que a vista 3D desenha começa dentro do lançador e segue o mesmo caminho da peça.

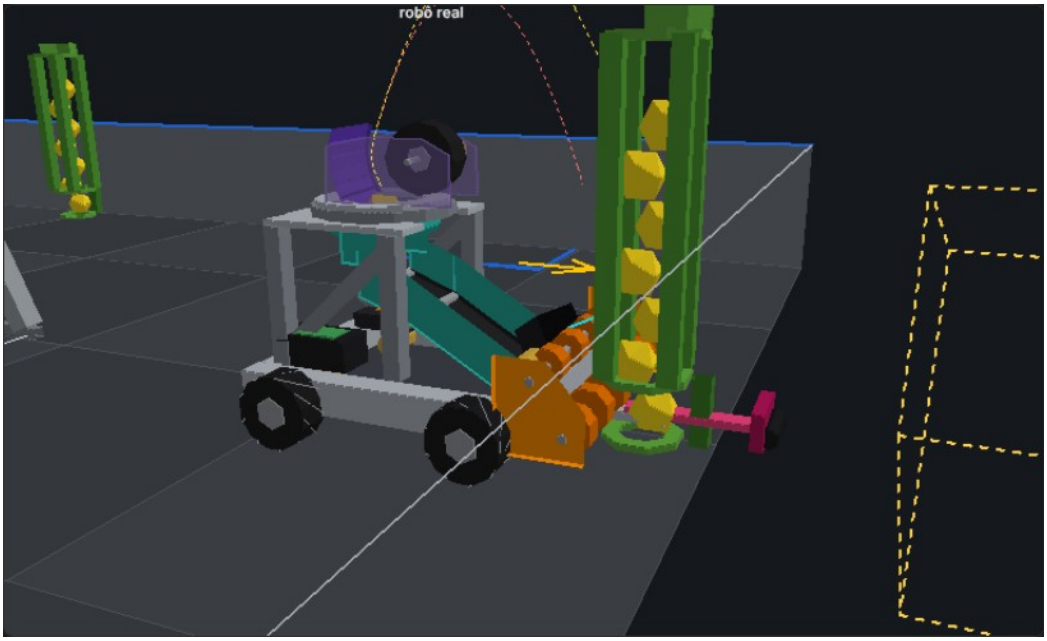
Figura 44 – Os lançadores com as laterais transparentes e uma peça passando por dentro: (a) capô fixo; (b) capô móvel, abaixado para 45°; (c) dois volantes; (d) em linha, roda em cima, na torreta; (e) em linha, dois discos, fixo atrás; (f) em linha, roda em cima, fixo atrás



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

O volante é um motor goBILDA de 6000 rpm (1:1, 28 ticks por volta): getVelocity devolve ticks por segundo, e setVelocity em RUN_USING_ENCODER faz o volante voltar à velocidade pedida mais depressa depois de cada lançamento (constante de tempo de 0,12 s, contra 0,35 s com setPower). As portas de motor 0 e 1 compartilham o encoder com os pods de odometria quando eles estão instalados, como acontece em robôs reais; a porta 2 fica livre para o pod perpendicular. Para pegar POLLEN de uma FLOWER, o robô para de frente para ela, estende o braço com o rolo puxando e recolhe com o intake ligado: o POLLEN de baixo sai pela abertura de retirada, os de cima descem, e a peça passa do braço para o robô (Figura 45).

Figura 45 – O braço tirando o POLLEN de baixo de uma FLOWER pela abertura de retirada



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

Para testar sem escrever nada, cada um dos 10 robôs possíveis — quatro lançadores na torreta, os mesmos quatro fixos na frente e os dois em linha fixos atrás — tem o seu TeleOp de exemplo, com nome próprio (por exemplo, ExemploTorretaCapoFixo.java, que aparece no Driver Hub como "Exemplo: torreta + capô fixo", ou ExemploAtrasDoisDiscos.java). O botão "Carregar o exemplo" do painel do jogo acrescenta ao projeto o exemplo do robô escolhido, ativa a configuração do Driver Hub com os nomes que ele usa (frontLeft, backLeft, frontRight, backRight, intake, midtake, shooter, arm e armRoller, mais turret na torreta, shooter2 nos lançadores de duas rodas e hood no capô móvel) e já faz o Build. Os exemplos de outros robôs que ainda estão como foram gerados saem do projeto, para que o Driver Hub não mostre um OpMode que pede peças que este robô não tem; os que o estudante mudou ficam. Cada exemplo também liga a câmera que o robô tem na montagem: com a C270, abre um VisionPortal com o AprilTagProcessor da biblioteca do jogo e o setCameraPose calculado da montagem da vista 3D, e mostra na telemetria quantas tags vê e o robotPose; com a Limelight, liga o pipeline 0 e mostra o botpose. Assim o Camera Stream e a tela da Limelight do Driver Hub passam a mostrar a imagem — sem um VisionPortal aberto, como no robô real, o Camera Stream fica vazio. Cada exemplo traz só o que o seu robô tem: com duas rodas, liga o shooter2 com setDirection(REVERSE) e só atira quando as duas chegaram na velocidade; com o capô móvel, Z e C (← e → do D-pad) mexem o servo do capô; sem torreta, explica que a mira é feita girando o robô — de frente para a HIVE, ou de costas, no lançador fixo atrás. No exemplo, o chassi mecanum anda com W A S D e gira com as setas; Espaço liga o intake, Y liga e desliga o volante (ou as rodas, nos lançadores

em linha), com setVelocity, as setas para cima e para baixo mudam a velocidade, X atira só quando a velocidade foi alcançada, Q e E giram a torreta e B estende e recolhe o braço. O código foi escrito para ser lido: cada mecanismo é um bloco curto, com a borda dos botões tratada como no FTC SDK.

16.4 A FÍSICA DAS PEÇAS, DO LANÇAMENTO E DAS HIVES

A física foi pensada para ser entendida, não para ser perfeita. As unidades são polegadas e segundos, e a gravidade é g = 386,1 pol/s². As peças soltas caem, quicam no chão (coeficiente de restituição 0,45), rolam com atrito e batem nas paredes (0,5), umas nas outras (esferas, 0,5), nas FLOWERs e em todos os tubos da estrutura das HIVEs — as pernas dos cavaletes, a barra do topo, a barra reta de cada HIVE, que gira com ela, o cubo do pivô e os trilhos do chão —, tratados como cápsulas (um segmento com raio; restituição 0,4). As paredes do campo só seguram a peça até a altura delas (12,5 pol.): acima disso, ela passa por cima, sai do campo e fica onde cair. O robô empurra as peças com a velocidade do ponto de contato, e as que estão acima de 18 pol. passam por cima dele. O intake só puxa uma peça que encosta no rolo, se houver lugar no robô. O robô não atravessa as pernas da estrutura em A nem as FLOWERs (teste de separação por eixos entre o retângulo do robô e cada obstáculo), e o braço para quando a ponta encosta na parede do campo ou na coluna de trás de uma FLOWER.

No lançamento, a velocidade de saída vem da rotação das rodas: $v = \eta \cdot \omega \cdot r$, com ω a rotação (rad/s), r o raio da roda e η a fração da velocidade da borda que passa para a peça. Quando a peça rola entre uma roda e uma superfície parada (os dois capôs e a roda em cima, com r = 2 pol.), ela andaria à metade da velocidade da borda (η = 0,5), e a compressão e o escorregamento reduzem isso para η = 0,255. Quando duas rodas empurram as duas faces (dois volantes e dois discos, com r = 1,5 pol.), ela anda mais perto da velocidade da borda: η = 0,34. Os valores de η são estimados e foram escolhidos para que todos os lançadores tenham o mesmo alcance com o mesmo motor e o mesmo ângulo: a 6000 rpm, a peça sai a 320 pol/s; a 3000 rpm, a 160 pol/s; o NECTAR, mais pesado e mais apertado, sai com 88% disso com uma roda e 92% com duas. Nos lançadores de duas rodas, ω é a média das duas: se um motor estiver desligado, a peça sai à metade da velocidade, e, se o shooter2 não tiver o REVERSE, as rodas giram uma contra a outra e a peça não sai, embora o getVelocity dos dois motores mostre a velocidade certa — o encoder fica no motor, que gira para o lado que mandaram.

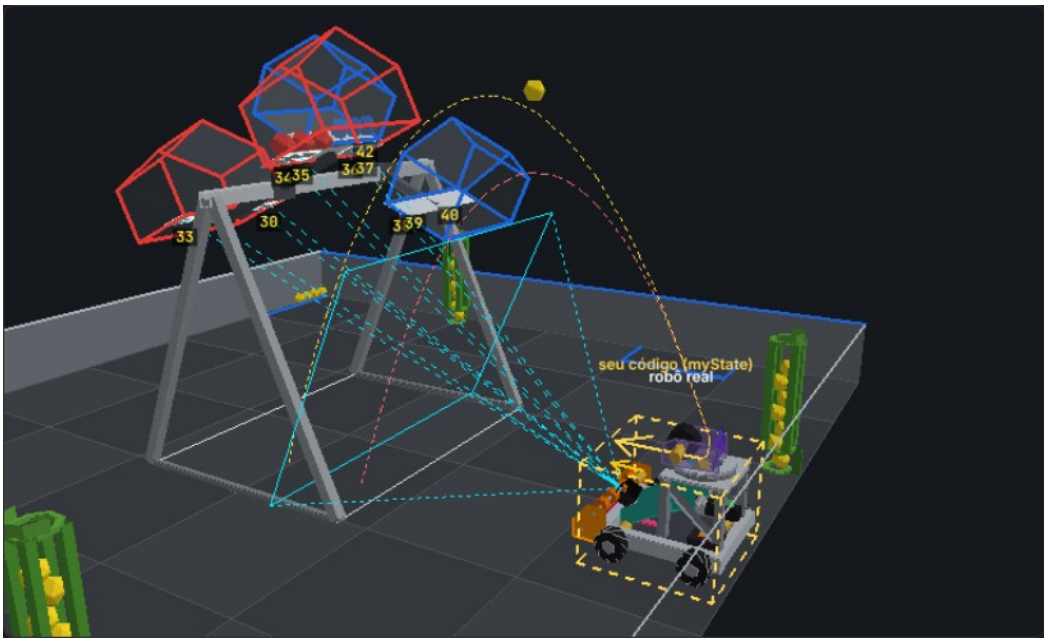
A velocidade de saída fica definida no instante em que a peça encosta nas rodas. Dentro do lançador, ela acelera de forma uniforme, da velocidade da subida (25 pol/s) até a de saída, ao longo do caminho de comprimento L (3,2 pol. no capô fixo, de

2,9 a 4,9 pol. no capô móvel, e de 2,9 a 3,3 pol. nos outros), e leva t = 2L / (v_entrada + v_saída) para passar: cerca de 18 ms a 6000 rpm, pouco mais de um quadro, e uns 50 ms a 1800 rpm. O ângulo do capô móvel também fica guardado nesse instante, para que a peça que já está passando não mude de caminho se o servo se mexer.

A peça sai na direção da tangente ao fim do caminho (70°, 60° atrás ou o ângulo do capô móvel), para onde o lançador aponta, e a essa velocidade soma-se a do próprio ponto de saída no instante em que ela sai, que tem três partes: o robô andando, o robô girando (ω × r, com r do centro do robô até a boca) e a torreta girando (com r do centro da torreta até a boca) — por isso um robô que gira ou anda enquanto atira desvia a peça, como no robô real. Não há arrasto do ar, então a trajetória é uma parábola; a física integra a queda livre de forma exata, e a trajetória prevista que a vista 3D desenha usa a mesma função, então a peça passa exatamente por cima dela (Figura 46). Cada arco termina onde a peça vai encostar em algo — o chão, uma parede, um tubo da estrutura, uma FLOWER ou uma CELL; se ela entra numa CELL de cima, o arco termina na entrada. São dois arcos, que se ligam e se desligam separadamente na barra da vista: o do POLLEN, em amarelo, e o do NECTAR, em vermelho, que sai mais devagar (88% ou 92% da velocidade) e por isso cai mais perto — com a mesma rotação, o robô precisa de um ajuste diferente para cada peça. Para cair num ponto a uma distância horizontal d e a uma altura Δz acima da boca, com a saída a α, a velocidade de saída é a da Equação 27, e o volante deve girar a v / (η · r) rad/s. Cada lançamento rouba parte da rotação das rodas, porque a energia cinética da peça sai delas: 18% (POLLEN) e 25% (NECTAR) com uma roda, 14% e 20% em cada uma quando são duas; o próximo só acerta depois que a rotação voltar.

$$v = \sqrt{(\ g \cdot d^2\ /\ (2 \cdot \cos^2\alpha \cdot (d \cdot \tan\alpha - \Delta z))\ )} \quad (27)$$

Figura 46 – Lançamento: os arcos previstos do POLLEN (amarelo) e do NECTAR (vermelho) e uma peça a caminho da CELL vermelha de cima



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

Nas CELLs, a peça é testada em coordenadas da própria CELL (u na largura, v para cima a partir do piso, w do fundo para a entrada). Se o centro dela passa pela entrada dentro da seção pentagonal, a CELL de cima a guarda numa vaga (45 vagas em camadas, ocupadas da mais baixa para a mais alta: primeiro no fundo, sobre o piso) e a peça passa a girar com a HIVE; se encosta numa parede de policarbonato ou na borda da entrada, quica, com restituição 0,4. Na CELL de baixo, que tem a entrada virada para baixo, a peça não fica: as paredes a seguram por dentro, e ela escorrega pelo piso e cai.

A HIVE é uma gangorra com trava. Nos batentes de ±25°, a trava segura a HIVE até que as peças da CELL de cima bastem para virá-la. Desde a versão 26, isso segue a regra de contagem do jogo: 8 POLLEN viram a HIVE, ou 5 NECTAR; com as duas peças misturadas, cada POLLEN vale 1/8 da virada e cada NECTAR, 1/5, e a HIVE vira quando a soma chega a 1 (Equação 28). Com os 3 NECTAR do começo (3/5 = 0,6), faltam 0,4: 4 POLLEN (0,5) ou 2 NECTAR (0,4). Até a versão 25, a trava comparava o torque das peças com um torque fixo, e o número de peças que virava a HIVE dependia da vaga em que cada uma parava; a contagem é a do manual e é fácil de conferir com a equipe. O painel do jogo mostra quanto falta para cada HIVE virar.

$$P/8 + N/5 \geq 1 \quad (28)$$

Quando a trava solta, a HIVE gira sob o torque das peças guardadas em torno do pivô, $\tau = \Sigma\ m \cdot g \cdot y$, com y a distância horizontal de cada peça ao pivô, contra uma

mola fraca "sobre o centro" que, depois do meio, ajuda a terminar o giro (momento de inércia de 0,55 kg·m² mais o das peças, amortecimento de 0,35 N·m·s); no outro batente, ela quica um pouco e é presa de novo (Código 9). Quando a CELL que estava em cima passa da horizontal, ela despeja as peças, que escorregam pela entrada e caem no campo. Com os 3 NECTAR do começo, o quarto POLLEN lançado na CELL de cima vira a HIVE, e os 7 objetos voltam para o chão.

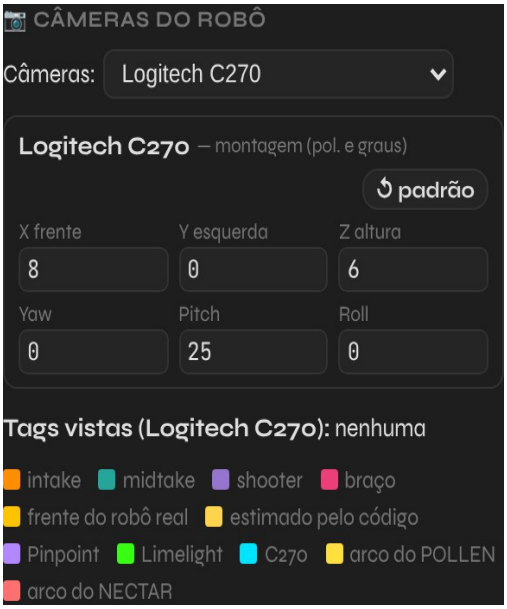
Código 9 – A trava da HIVE (trecho de bb_game.js, simplificado)

```
const u = bbTipUnits(side);                    // P/8 + N/5 das peças na CELL de cima
if (!H.free) {                                  // presa no batente
    if (u >= 1) H.free = true;                  // peças suficientes: a trava solta
    else continue;
}
const tb = bbHiveTorque(side);                 // Σ m·g·y das peças nas CELLs (N·m)
const tm = 0.25 * tauTrava * Math.sin(th / beta * Math.PI / 2);  // mola sobre o
centro
const net = tb + tm - amortecimento * w;
w += net / I * dt;  th += w * dt;
if (Math.abs(th) > beta) { th = Math.sign(th) * beta; w = -0.2 * w; if (Math.abs(w) <
0.15) { w = 0; H.free = false; } }
// a CELL com a entrada virada para baixo despeja o que tinha
for (const s of [-1, 1]) if (s * Math.sin(th) < 0.05) despejar(side, s);
```

16.5 CÂMERAS EM SEIS GRAUS DE LIBERDADE

Cada câmera tem a montagem completa: X (para a frente), Y (para a esquerda) e Z (altura da lente), em polegadas a partir do centro do robô no chão, e yaw (giro para a esquerda), pitch (para cima) e roll (topo da imagem para a direita), em graus, aplicados nessa ordem. A montagem é guardada para cada campo e editada no painel da vista 3D, ao lado da imagem, pelos seis números (a inclinação é o pitch; até a versão 25 ela também tinha um controle deslizante e um campo na montagem do robô, e cada opção agora fica num lugar só), onde também se escolhe usar só a C270, só a Limelight ou as duas (Figura 47). No desenvolvimento completo, essa escolha é a própria montagem do robô; no modelo de visão, ela é a opção "Câmera" da barra do código, porque muda o código-modelo.

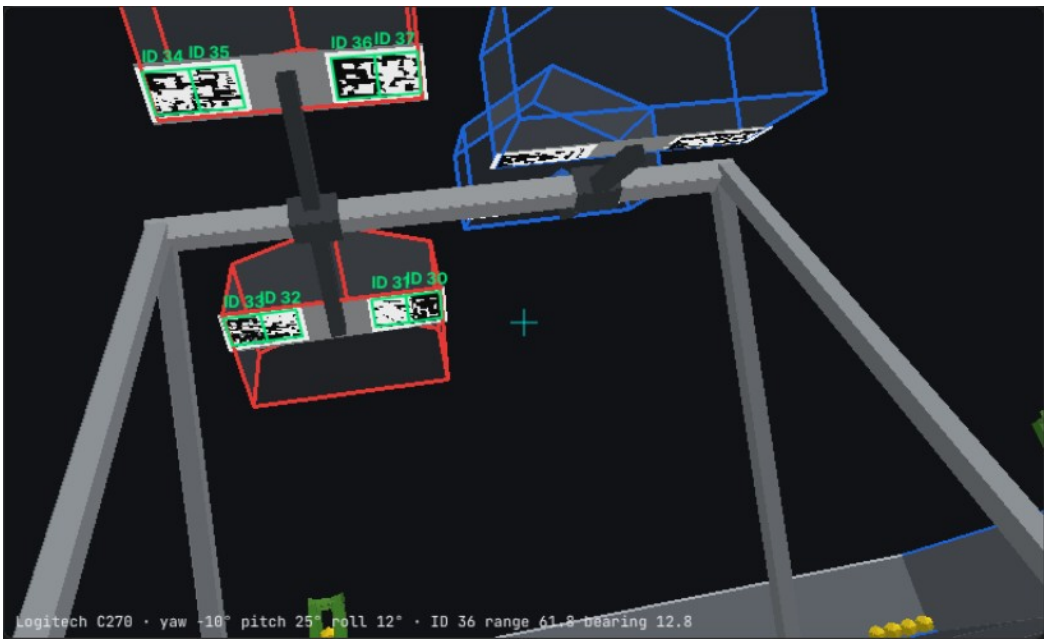
Figura 47 – Montagem das câmeras em seis graus de liberdade, no painel da vista 3D



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

No motor 3D, o yaw soma-se ao heading no giro a do rotatePoint, e o pitch é o giro b. O roll, que é um giro em torno do eixo da lente, é aplicado depois, como um giro da imagem em torno do centro, o que não muda o algoritmo do autor (Figura 48).

Figura 48 – O que a C270 captura com yaw de −10°, pitch de 25° e roll de 12°



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

No SDK, setCameraPose recebe a posição em outro referencial (x para a direita, y para a frente e z para cima) e o pitch medido a partir de uma câmera que aponta para cima: uma câmera olhando para a frente tem pitch de −90°, e a C270 inclinada 25° para cima tem pitch de −65°. O Sandbox calcula o robotPose de cada

detecção como o SDK, com a pose da câmera declarada no código e a posição da tag na biblioteca do jogo. Se a declaração estiver certa, o robotPose fica a menos de 1 pol. da pose real; se o estudante esquecer a inclinação (pitch de −90° em vez de −65°), o mesmo robô aparece a mais de 9 pol. de onde está. A biblioteca do jogo e o mapa da Limelight guardam as tags na posição do começo da partida: quando uma HIVE vira, as tags dela andam, e as poses calculadas por elas saem deslocadas. É o mesmo cuidado que o projeto ViDAR registra ao dizer que as tags das HIVEs não são marcos fixos de localização.

16.6 VALIDAÇÃO

Um teste do lançamento dispara uma peça em 432 situações: com o capô fixo na torreta, quatro tipos de chassi (ideal, mecanum, tank e swerve), dezoito combinações de teclas de direção e de giro (nenhuma, cada uma sozinha e várias juntas, como W + D + ←) e três estados da torreta (parada à frente, parada a 60° e girando a 400°/s durante o disparo); e, no chassi mecanum, as mesmas combinações com os outros lançadores e montagens (capô móvel a 45°, dois volantes e roda em cima na torreta; capô fixo na frente; dois discos e roda em cima atrás). Em todas, a velocidade da peça foi a esperada — a do volante mais a velocidade do ponto de saída no instante em que ela sai, medida pela variação da posição da boca —, a peça seguiu a parábola com desvio menor que 0,05 pol. até encostar em algum obstáculo, nunca bateu no próprio robô e nunca produziu valores inválidos. O mesmo teste mostra o efeito da rotação (Tabela 30); a 70°, os números foram os mesmos nos cinco lançadores, porque todos foram ajustados para o mesmo alcance — a diferença entre eles está no caminho por dentro e nos motores.

Tabela 30 – Alcance do lançamento em função da rotação do volante (robô parado, peça de volta à altura da boca)

Rotação (rpm)	Velocidade de saída	Alcance medido	R = v² · sen 2α / g
1799	95,9 pol/s	15,9 pol.	15,3 pol.
2698	143,9 pol/s	35,3 pol.	34,5 pol.
3598	191,8 pol/s	62,3 pol.	61,3 pol.
4197	223,8 pol/s	84,2 pol.	83,4 pol.
4797	255,8 pol/s	109,4 pol.	108,9 pol.

Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

O ângulo muda o alcance tanto quanto a rotação (Tabela 31): com a mesma rotação, o capô móvel abaixado para 45° manda a peça 56% mais longe que o capô fixo, e levantado para 75°, 23% mais perto.

Tabela 31 – Alcance com a mesma rotação (3598 rpm, 191,8 pol/s) e ângulos de saída diferentes

Lançador	Ângulo de saída	Alcance medido	R = v² · sen 2α / g
Capô móvel, abaixado	44,9°	97,4 pol.	95,3 pol.
Em linha, fixo atrás	60°	83,1 pol.	82,6 pol.
Capô fixo e os outros	70°	62,3 pol.	61,3 pol.
Capô móvel, levantado	75°	48,0 pol.	47,7 pol.

Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

O alcance foi medido no primeiro quadro em que a peça desceu abaixo da altura da boca; a diferença para a fórmula, de até 2 pol. no tiro mais deitado, é o quanto ela anda num quadro. Outros quatro testes conferem os mecanismos, o exemplo e a câmera. No modelo de visão, comandado pelo teclado:

- o intake guarda a peça que encosta no rolo e recusa a quinta (desde a versão 26, cabem 4);
- um lançamento a 49,8 pol. (na horizontal) da CELL vermelha de cima, com a velocidade calculada pela Equação 28 (209 pol/s) e a torreta a 7°, entra na CELL; o segundo, disparado logo em seguida, com o volante ainda sem recuperar a rotação, não entra;
- com os 3 NECTAR do começo, a HIVE vermelha virou depois de 4 POLLEN, e a CELL que desceu despejou os 3 NECTAR, que chegaram ao chão;
- o braço tirou o POLLEN de baixo de uma FLOWER, o intake o guardou e o resto da pilha desceu;
- a perna da estrutura bloqueou o robô, e o robô empurrou um NECTAR;
- com "Seguir o robô", as teclas J, I e U giraram, inclinaram e aproximaram a câmera, o analógico direito do gamepad2 girou e inclinou, o D-pad do gamepad1 girou só com a opção ligada e, com "girar junto com o robô", a vista acompanhou exatamente o giro do robô.

No desenvolvimento completo, um autônomo com os nomes da configuração manteve o volante a 1829 ticks/s, com getVelocity a menos de 0,1% do alvo antes e depois do lançamento, e acertou a CELL; num segundo autônomo, usou o braço e o intake numa FLOWER. O robotPose com o setCameraPose certo ficou a 0,5 pol. da pose real; com a inclinação esquecida, a 9,4 pol. O TeleOp de exemplo compilou em português e em inglês e, começando na posição Vermelho 1, andou para a frente com W, chegou à velocidade pedida, atirou os 2 POLLEN carregados com X, girou a torreta com Q e estendeu o braço com B. Os 10 exemplos, um para cada robô, compilaram em português e em inglês; carregado e rodado no Driver Hub, cada um configurou o shooter2, o servo do capô e a torreta só quando o robô os tinha, chegou à velocidade e

atirou as duas peças, e só o exemplo do robô escolhido ficou no projeto; com o capô móvel, Z baixou a saída de 70° para 56°, e a peça saiu a 56°. Sem o REVERSE no shooter2, as rodas giraram uma contra a outra (+3857 e −3857 rpm) e nenhuma peça saiu. Com a visão, na posição de treino (0; −48), o exemplo da C270 viu 8 tags e deu o robô em (−0,5; −48,0) pol., e o da Limelight, em (0,1; −48,0) pol., com o Camera Stream e a tela da Limelight mostrando a imagem, nas três montagens de câmera. Um último teste dispara 120 peças de lugares, velocidades e ângulos de torreta sorteados e confere, a cada quadro, que nenhuma entra num tubo da estrutura, numa parede ou no chão, nem fica solta dentro de uma CELL de cima, que a energia nunca cresce e que nenhuma para no ar; em 114 dos 119 lançamentos, a primeira batida da peça ficou a menos de 6 pol. do fim do arco previsto, e nos outros 5 o arco parou num toque de raspão, que mal muda a velocidade da peça. Peças jogadas por cima da parede saíram do campo, e as que rolavam no chão voltaram da parede. As seis posições iniciais das alianças ficaram no lado certo, encostadas na parede, fora da LOADING ZONE e sem colisão, e o PARK foi reconhecido com o robô dentro da LOADING ZONE vermelha. Todos os testes anteriores continuam passando.

17. PARTIDA COM ROBÔS, BATERIA E CALIBRAÇÃO

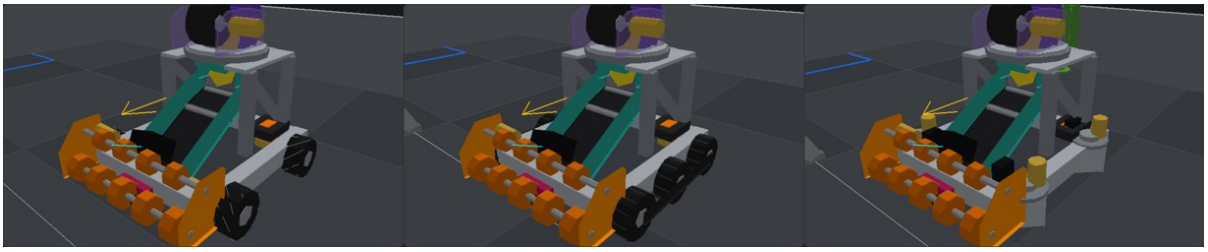
A versão 26 leva o campo BIOBUZZ para mais perto de uma partida de verdade. No desenvolvimento completo, o robô do estudante pode jogar com outros três robôs (um aliado e dois adversários) ou contra um só, e esses robôs têm autônomo e controlado próprios. A bateria passa a poder descarregar, o que muda a força dos motores e torna visível a diferença entre setPower e setVelocity. Um cartão de calibração mede até onde a peça vai para cada potência do volante e gera a função que o estudante leva para o código do robô real. A C270, olhando para a frente, passa a ver as peças no chão com o processador de manchas de cor do SDK. A HIVE vira pela contagem de peças do jogo, o robô leva até 4 peças, os três chassis aparecem na vista 3D e a tela do Sandbox foi reorganizada para que cada opção fique num lugar só.

17.1 OS TRÊS CHASSIS NA VISTA 3D

Até a versão 25, o robô da vista 3D tinha sempre quatro rodas mecanum lisas, qualquer que fosse o chassi escolhido. Agora, cada chassi tem o seu modelo (Figura 49). As rodas mecanum ganharam os roletes, desenhados como nove linhas inclinadas a 45° na face de cada roda, como na peça real: nas rodas da frente à esquerda e de trás à direita, as linhas vão para um lado, e nas outras duas, para o outro, de modo que, vistas de cima, as quatro formam um X. O tank tem três rodas de cada lado, com sulcos, ligadas por uma correia, e os dois motores. O swerve tem quatro módulos, cada um com a placa, a engrenagem de direção, o motor da roda e o

servo; a roda de cada módulo gira de verdade para o ângulo que o módulo está fazendo na simulação, o mesmo que o encoder do módulo mede.

Figura 49 – Os três chassis na vista 3D: mecanum (com os roletes), tank (três rodas de cada lado e a correia) e swerve (módulos girados para o ângulo da simulação)



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

17.2 A PARTIDA: ALIANÇA, RELÓGIO E AUTÔNOMO DOS OUTROS ROBÔS

A partida com robôs fica na configuração da arena, embaixo da vista 3D: desligada, 2 robôs (o do estudante e um adversário) ou 4 robôs (um aliado e dois adversários), e a aliança do estudante, vermelha ou azul. O robô do estudante começa numa das três posições iniciais da sua aliança (a escolhida, se for da aliança; senão, a primeira), o aliado numa das outras duas, e os adversários em posições sorteadas da outra aliança. Os outros robôs têm o mesmo tamanho (18 × 18 pol.) e os mesmos mecanismos do robô do Sandbox, com o lançador de capô fixo a 70° fixo na frente e a placa e a bandeirinha da aliança, e começam com 4 POLLEN. O robô do estudante não os atravessa: para ele, cada um é um obstáculo, como as pernas da estrutura.

A partida segue o OpMode do estudante, como numa competição. O INIT de um Autonomous começa uma partida nova, com as peças e os robôs nas posições iniciais; o START liga o relógio de 30 s, e os outros robôs fazem o seu autônomo. Rodar um TeleOp logo depois do autônomo continua a mesma partida: o START liga os 2 min do controlado, e os últimos 20 s são o fim de jogo. Qualquer outro INIT de TeleOp começa uma partida nova, como se o autônomo tivesse sido pulado. No fim do controlado, se o timer do Driver Hub estiver ligado, o OpMode do estudante é parado. Um placar em cima da arena mostra o período e o tempo que falta e, para cada aliança, quantas vezes a HIVE virou e quanto falta para a próxima virada (Figura 50); o cartão da partida mostra o plano de autônomo de cada robô e, no fim de cada período, quem estava em PARK.

Figura 50 – A arena vista de cima no controlado: o placar (viradas e quanto falta para virar), os outros robôs com a cor da aliança, o nome e quantas peças carregam



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

O autônomo de cada robô é sorteado a cada partida: 50% de chance de só estacionar (PARK), 35% de virar a HIVE da sua aliança — atirar os 4 POLLEN que começam no robô e estacionar — e 15% de ficar parado. Os 4 POLLEN bastam para virar a HIVE porque, com os 3 NECTAR do começo, a CELL de cima já tem 0,6 da virada (Seção 17.4). Em 3000 sorteios, os planos saíram em 49,7%, 35,0% e 15,3% dos robôs. O robô que vai estacionar espera de 0 a 3 s antes de sair, para que os dois robôs da mesma aliança não saiam juntos.

17.3 COMO OS OUTROS ROBÔS JOGAM

Cada outro robô decide o que fazer com uma pequena máquina de estados, refeita sempre que uma tarefa termina:

- **Pegar uma peça no chão:** os candidatos são o POLLEN e o NECTAR da cor da aliança (o intake dos outros robôs tem um sensor de cor e não aceita o NECTAR do adversário), parados, dentro do campo e longe do robô do estudante. A escolha soma o caminho até a peça, um terço da distância da peça até a HIVE da aliança e um pequeno sorteio; o NECTAR ganha um desconto, porque vale mais para virar. O robô vai até um ponto 22 pol. atrás da peça, já virado para ela, e avança devagar com o intake ligado; peças encostadas numa parede são pegas de frente para a parede.

- **Tirar POLLEN de uma FLOWER:** o robô para de frente para a abertura de baixo e repete o ciclo do braço — estender (0,5 s), pegar, recolher (0,5 s) e puxar com o intake —, até encher ou a FLOWER esvaziar.
- **Atirar:** o robô escolhe um ponto de arremesso na frente da CELL de cima da HIVE da sua aliança, vira para ela, liga o volante e atira uma peça a cada 0,45 a 0,65 s. Ele atira quando chega à quantidade que sorteou para aquela volta (2, 3 ou 4 peças) ou quando não há mais peças para pegar.
- **Estacionar:** quando o tempo que falta fica menor que o tempo estimado até a LOADING ZONE mais 5 s, o robô larga o que estava fazendo e vai estacionar. O tempo é estimado pelo comprimento do caminho, a 60% da velocidade máxima; cada aliança tem duas vagas na sua LOADING ZONE, e um robô não escolhe a vaga que outro já escolheu.

O ponto de arremesso é escolhido entre 20 posições numa grade na frente da CELL de cima (de 38 a 56 pol. do meio do campo e até 12 pol. para cada lado da HIVE). Para cada posição, o robô simula a trajetória com a mesma física das peças (Seção 16.4): parte da velocidade da Equação 27 e procura, por varredura e bisseção, a faixa de velocidades em que a peça entra na CELL de cima, sem bater na estrutura, nas paredes da CELL ou na outra HIVE. Essa faixa é a "janela" daquele ponto; ela tem entre 7,9% e 12,6% da velocidade, e o robô mira no meio dela. Como as HIVEs param sempre nos mesmos ±25°, cada janela é calculada uma vez (de 5 a 15 ms) e guardada, no máximo uma por quadro.

Para errar como um robô de verdade, cada arremesso tem um erro de velocidade sorteado de uma distribuição normal com desvio igual a meia janela dividida por um fator z de cada robô, sorteado entre 0,95 e 1,45 (z menor, mais erros), e um erro de mira de 1° a 2,5°. Entre 5% e 9% dos arremessos são "ruins", com o triplo do erro. Como o erro é medido em frações da janela, a chance de acertar depende da mão do robô, e não do ponto de onde ele atira. Uma peça que bate na parede da CELL ainda pode quicar para dentro, como no campo real.

Para andar, o robô usa o algoritmo A* numa grade de 4 pol. (37 × 37 nós). Um nó é proibido se um círculo de 11,5 pol. em volta dele encosta numa perna da estrutura ou numa FLOWER, se o robô sairia do campo ou se está a menos de 19 pol. de outro robô. O caminho é encurtado por linha de visada (pontos a cada 2 pol.) e refeito a cada segundo. O robô segue o caminho com velocidade de até 34 pol/s e aceleração de 70 pol/s², freando para chegar, e gira ao mesmo tempo para o heading pedido, porque o chassi mecanum anda de lado. A cada quadro, a caixa do robô na posição nova é testada contra as paredes, as pernas, as FLOWERs e os outros robôs pelo teorema dos eixos separadores; se bater, ele tenta só andar, só um eixo ou só girar. Um vigia

confere o progresso: se o robô passa 2,5 s sem chegar 1 pol. mais perto do destino, ele refaz o caminho, troca de ponto de arremesso ou desiste da peça.

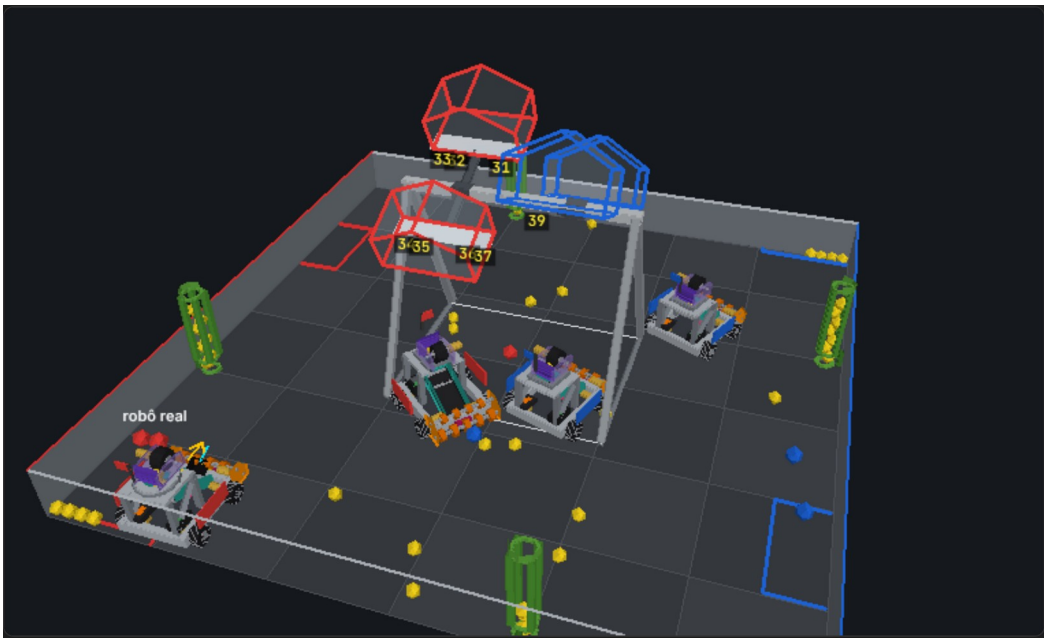
Em 6 partidas completas de 4 robôs (autônomo e controlado), os 18 outros robôs fizeram 592 arremessos e acertaram 475 na CELL de cima (80,2%), viraram as HIVEs 57 vezes e estacionaram em 18 dos 18 fins de partida (Tabela 32). Com os outros robôs, um quadro inteiro do simulador (física, robôs, Driver Hub e arena, sem a vista 3D) levou em média 1,0 ms no computador dos testes; cada outro robô desenhado na vista 3D custa de 1 a 4 ms, e os que aparecem pequenos na imagem são desenhados com um modelo simples.

Tabela 32 – Os outros robôs em partidas completas (4 robôs, aliança vermelha)

Partida	Planos do autônomo	Arremessos (acertos)	Viradas (V / A)	PARK no fim
1	PARK, PARK, virar	R2: 42 (39); B3: 41 (30); B2: 40 (33)	5 / 8	3 de 3
2	virar, virar, PARK	R3: 33 (25); B1: 34 (30); B2: 28 (22)	3 / 6	3 de 3
3	virar, PARK, PARK	R3: 42 (36); B2: 24 (17); B3: 28 (17)	4 / 4	3 de 3
4	PARK, PARK, virar	R3: 35 (26); B3: 21 (16); B2: 21 (17)	3 / 4	3 de 3
5	PARK, virar, virar	R2: 43 (38); B2: 26 (20); B1: 33 (26)	5 / 6	3 de 3
6	virar, virar, virar	R3: 36 (29); B3: 36 (33); B1: 29 (21)	3 / 6	3 de 3

Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

Figura 51 – Uma partida de 4 robôs no controlado, na vista 3D: o robô do estudante (embaixo, à esquerda), o aliado e os dois adversários



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

17.4 A REGRA DE VIRADA DA HIVE E A CAPACIDADE DO ROBÔ

A regra de virada da HIVE passou a ser a de contagem do jogo (Equação 28, na Seção 16.4): 8 POLLEN ou 5 NECTAR na CELL de cima viram a HIVE, e, misturadas, cada POLLEN vale 1/8 e cada NECTAR 1/5. A função bbTipUnits soma essas frações das peças guardadas na CELL de cima, e a trava solta quando a soma chega a 1. A Tabela 33 mostra o teste da regra, com as peças colocadas direto na CELL de cima da HIVE vermelha, sem os 3 NECTAR do começo.

Tabela 33 – Regra de virada da HIVE: P/8 + N/5 ≥ 1

POLLEN	NECTAR	P/8 + N/5	A HIVE virou?
8	0	1,000	sim
7	0	0,875	não
0	5	1,000	sim
0	4	0,800	não
4	3	1,100	sim
3	3	0,975	não
4	2	0,900	não
5	2	1,025	sim
6	1	0,950	não

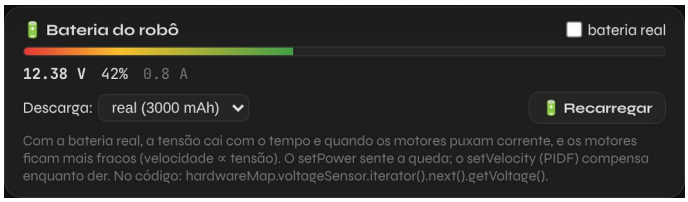
Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

O robô passou a levar até 4 peças de uma vez — POLLEN, NECTAR ou misturadas —, e o robô do estudante pode começar a partida com 0 a 4 POLLEN. Com 4 peças dentro, o intake empurra a quinta para fora, em vez de guardá-la.

17.5 BATERIA REAL E SENSOR DE TENSÃO

Com a opção "bateria real", embaixo do Driver Hub (Figura 52), a bateria de 12 V (NiMH, 3000 mAh) passa a ter carga. A tensão em aberto vai de 13,6 V (cheia) a 11,6 V (vazia), e a tensão nos terminais cai mais quando os motores puxam corrente, por uma resistência interna de 0,08 Ω. A corrente vem do que o robô está fazendo: 0,8 A da eletrônica, até 3,5 A por motor do chassi, até 2,5 A por motor do volante (e mais enquanto ele acelera) e 1,5 A por motor do intake e da esteira. A carga cai com a corrente, na velocidade real ou acelerada 5 ou 20 vezes, e o botão "Recarregar" volta a 100%. A velocidade de cada motor é proporcional à tensão (a razão V / 12), e a tensão do Driver Hub e a do VoltageSensor do SDK (hardwareMap.voltageSensor) mostram o valor da bateria. Sem a opção, a tensão fica em 12,8 V e nada muda em relação às versões anteriores.

Figura 52 – A bateria do robô embaixo do Driver Hub: carga, tensão, corrente, velocidade da descarga e o botão Recarregar



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

A bateria é o melhor jeito de ver por que as equipes usam o controle de velocidade (PIDF) no volante (Tabela 34): com setPower(0,8), o volante gira 11% mais rápido com a bateria cheia e 4% mais devagar com ela a 10%, e o arremesso muda junto; com setVelocity(2240), o PIDF do motor compensa a tensão, e o volante fica em 2240 ticks/s enquanto houver tensão sobrando. O código pode ler a tensão e compensar o setPower (por exemplo, setPower(p · 12 / V)), o que a Referência do SDK do Sandbox mostra.

Tabela 34 – Volante com setPower e com setVelocity, conforme a carga da bateria

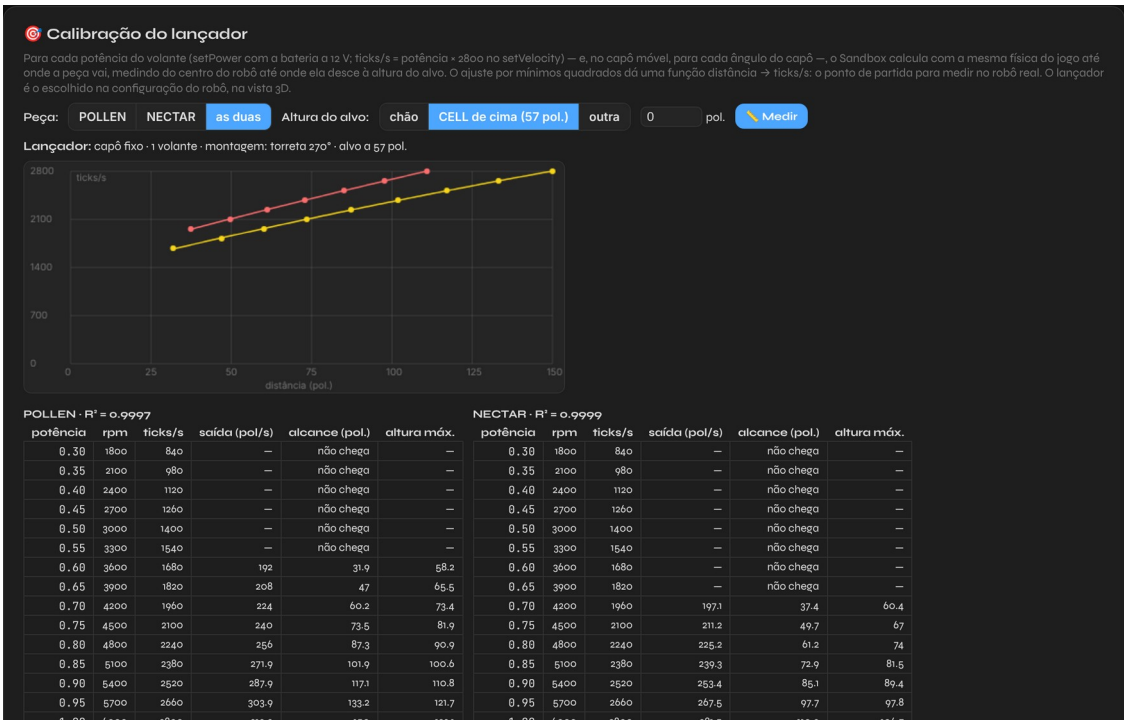
Bateria	Tensão	setPower(0,8)	setVelocity(2240)
desligada (12,8 V fixos)	12,80 V	2233 ticks/s	2240 ticks/s
real, 100%	13,36 V	2484 ticks/s	2240 ticks/s
real, 50%	12,37 V	2301 ticks/s	2240 ticks/s
real, 10%	11,57 V	2154 ticks/s	2240 ticks/s

Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

17.6 CALIBRAÇÃO DO LANÇADOR

O cartão "Calibração do lançador", nas ferramentas do desenvolvimento completo, responde à pergunta que toda equipe faz antes de ir para a mesa de testes: com que potência o volante deve girar para a peça cair a uma certa distância? Para cada potência de 0,30 a 1,00, de 0,05 em 0,05 (com a bateria a 12 V; no setVelocity, ticks/s = potência × 2800), o Sandbox calcula, com a mesma física do lançamento (Equação 27, Seção 16.4), a velocidade de saída, a altura máxima e a distância horizontal do centro do robô até onde a peça desce à altura do alvo — o chão, a CELL de cima (57 pol.) ou outra altura. No capô móvel, o cálculo é feito para cada ângulo de 40° a 75°, de 5° em 5°, e o cartão também mostra, para cada distância de 30 a 130 pol., o ângulo que pede a menor rotação, que é o arremesso menos sensível a erros de rotação. Os pontos ficam num gráfico, e um ajuste por mínimos quadrados de grau 2 dá a função distância → ticks/s, com o R^2, que o cartão escreve em Java, pronta para copiar (Figura 53). Embaixo, uma tabela mostra os últimos arremessos do robô do estudante: a peça, a velocidade, o ângulo, a distância da boca até a primeira batida e onde ela bateu.

Figura 53 – Calibração do lançador de capô fixo para a CELL de cima: o gráfico, as tabelas de POLLEN e NECTAR e a função em Java



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

A função é um ponto de partida: os valores de η e as massas das peças são estimados (Seção 16.4), e no robô real os coeficientes precisam ser medidos. Para conferir que ela funciona no próprio Sandbox, um teste ajustou a função para o chão (R^2 = 0,9985) e atirou um POLLEN com a velocidade que ela dá para 40, 60, 80 e 100

pol., com o robô parado (Tabela 35). A peça caiu a menos de 0,7 pol. do previsto pelo cálculo; a diferença maior, entre a distância pedida e a calculada (até 1,6 pol.), é o erro do ajuste de grau 2.

Tabela 35 – Arremessos com a velocidade dada pela função de calibração (capô fixo, POLLEN, alvo no chão)

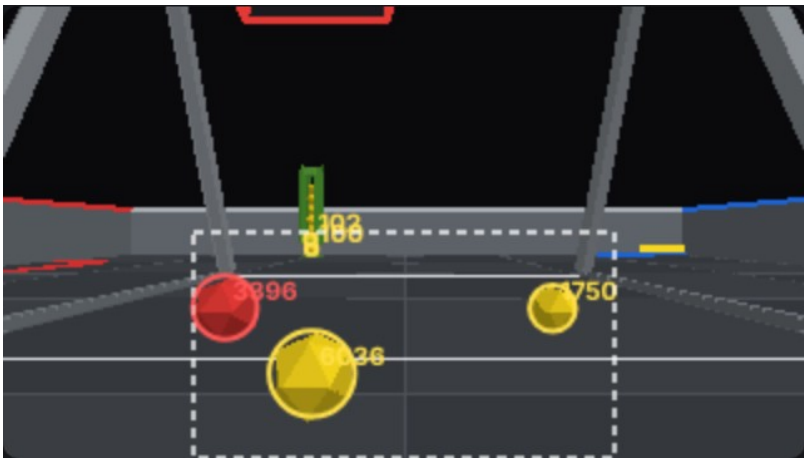
Distância pedida	ticks/s	Calculada (centro do robô)	Medida da boca (calculada)
40 pol.	1308	38,4 pol.	42,6 pol. (42,3 pol.)
60 pol.	1619	58,4 pol.	62,2 pol. (62,3 pol.)
80 pol.	1897	79,8 pol.	84,0 pol. (83,7 pol.)
100 pol.	2144	101,5 pol.	106,1 pol. (105,4 pol.)

Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

17.7 A C270 A 0°: AS PEÇAS PELA COR

O FTC SDK tem, desde a versão 10.1, um processador de visão que acha manchas de uma cor na imagem de uma webcam: o ColorBlobLocatorProcessor, que se junta ao VisionPortal como o AprilTagProcessor. Com ele, a C270 pode reconhecer as peças do BIOBUZZ pela cor — amarelo para o POLLEN, vermelho e azul para o NECTAR. O Sandbox simula esse processador: cada peça solta, numa FLOWER, numa CELL ou no braço, que está dentro do campo de visão, a menos de 120 pol. e sem nada opaco na frente, vira uma mancha circular na imagem, com o centro e o raio da projeção da esfera. O processador tem o Builder do SDK (faixa de cor, modo de contorno, região de interesse, desenho dos contornos e os parâmetros de desfoque e erosão), as manchas (Blob) com área, densidade, proporção, retângulo ajustado (getBoxFit), círculo e circularidade, e as funções de filtro e ordenação do Util. As manchas aparecem desenhadas no Camera Stream do Driver Hub (Figura 54).

Figura 54 – Camera Stream da C270 a 0°: a região de interesse (tracejada) e as manchas do POLLEN e do NECTAR, com a área de cada uma



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

Quando a C270 está montada reta (inclinação de até 10°), o TeleOp de exemplo do robô ganha três desses processadores, um por cor, olhando a metade de baixo da imagem, que é o chão à frente do intake. A peça mais perto é a mancha mais embaixo na imagem; quando ela estava na borda de baixo (y acima de 400 de 480 pixels) e some com o intake puxando, o exemplo conta que o robô pegou aquela peça e mostra a contagem de POLLEN e NECTAR na telemetria. Num teste, o robô andou para a frente com o intake ligado na direção de um POLLEN a 20 pol.: a mancha desceu na imagem (y de 345 para 444), sumiu quando a peça entrou, e a telemetria passou a mostrar "Pegou: POLLEN 1". A contagem é uma estimativa — uma peça que some pela borda de baixo sem entrar também é contada —, como acontece com a câmera de verdade.

17.8 A TELA DO SANDBOX: CADA OPÇÃO NUM LUGAR SÓ

A versão 26 conferiu a tela do Sandbox para que nenhuma opção possa ser escolhida em dois lugares. Até a versão 25, a inclinação da câmera aparecia em três (o campo "C270 inclinada" da montagem do robô, o controle deslizante "Inclinação" e o campo Pitch da vista 3D), e o campo e a câmera, em dois. Cada opção agora tem um lugar só (Tabela 36).

Tabela 36 – Onde fica cada opção no Sandbox (versão 26)

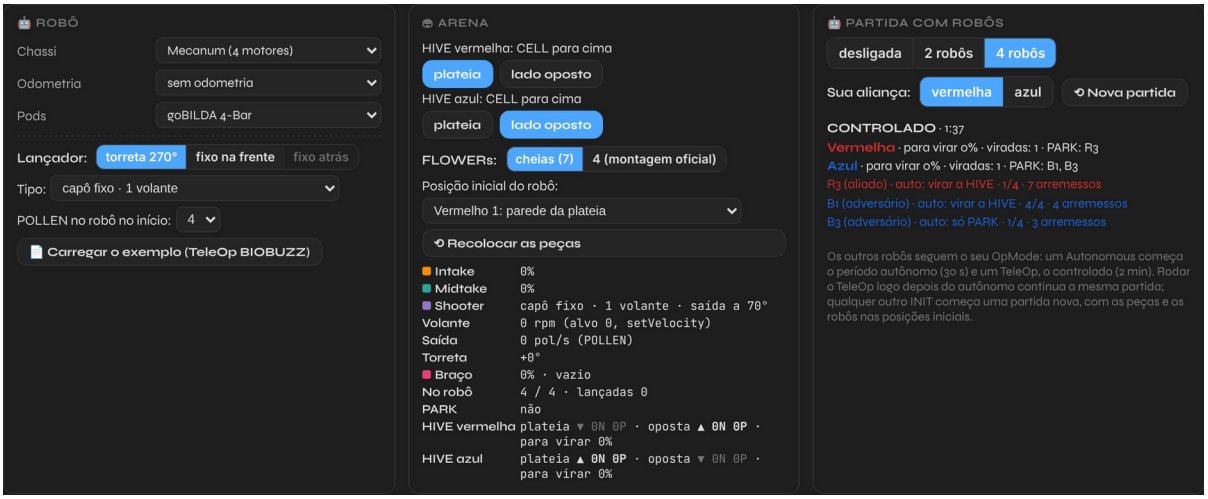
Opção	Desenvolvimento completo	Desenvolvimento pontual
Campo (treino ou BIOBUZZ)	alto da vista 3D	alto da vista 3D (modelo de visão)
Câmeras instaladas	ao lado da vista 3D	barra do código ("Câmera"), porque muda o código-modelo
Montagem e inclinação das câmeras	ao lado da vista 3D (seis números)	ao lado da vista 3D (seis números)
Chassi, odometria e pods	coluna Robô, embaixo da vista 3D	modo de movimento, ao lado da arena
Lançador, peças no robô e exemplo	coluna Robô, embaixo da vista 3D	coluna Robô (modelo de visão)
HIVEs, FLOWERs e posição inicial	coluna Arena, embaixo da vista 3D	coluna Arena (modelo de visão)
Partida com robôs	coluna Partida, embaixo da vista 3D	—
Bateria real	embaixo do Driver Hub	—
Calibração do lançador	depois das ferramentas do Sandbox	—

Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

O cartão da vista 3D foi reorganizado para ocupar o espaço sem deixar buracos: ao lado da imagem ficam as câmeras, a legenda e as tags vistas; embaixo,

em três colunas da mesma altura, o robô, a arena e a partida (Figura 55). Embaixo da arena 2D ficaram só o "colocar o robô na posição inicial" e o botão que faz isso, e embaixo do Driver Hub, a bateria — assim as duas colunas do alto da tela terminam quase na mesma altura. A opção do menu que desliga a vista 3D agora esconde só a imagem: a configuração do robô, da arena e da partida continua no lugar.

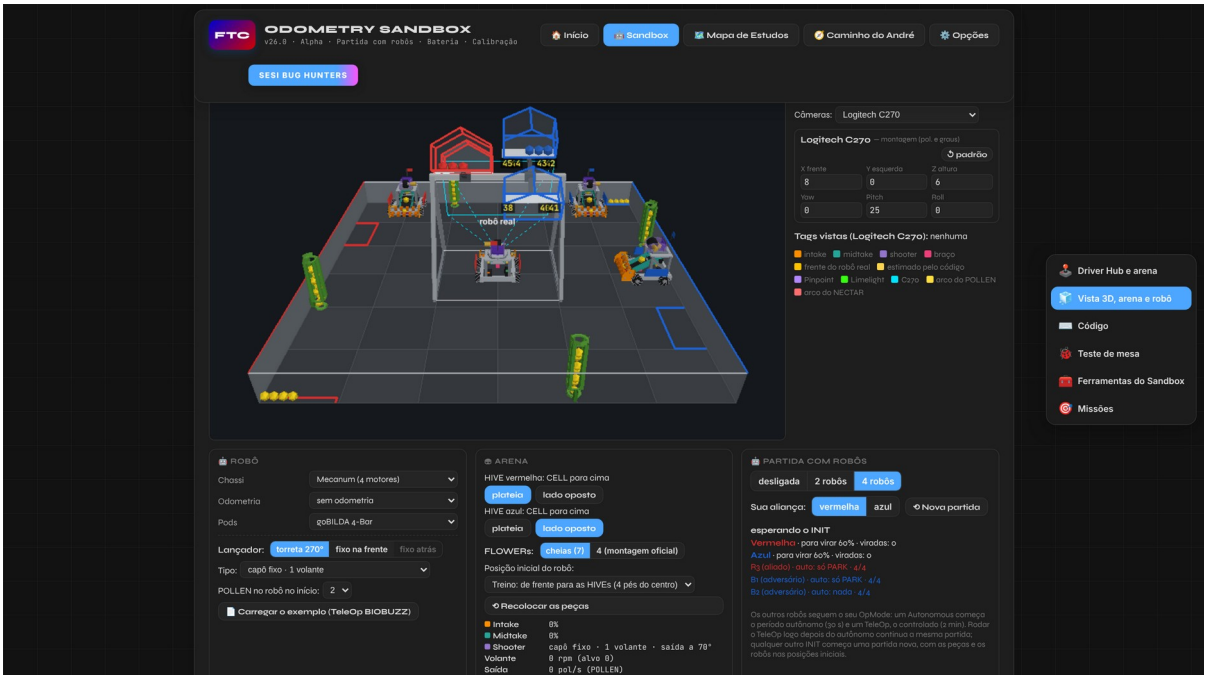
Figura 55 – A configuração embaixo da vista 3D: Robô, Arena e Partida com robôs



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

Uma barra de navegação fica presa à direita da tela do Sandbox (Figura 56). No desenvolvimento completo, ela tem um botão para cada parte: Driver Hub e arena; vista 3D, arena e robô; código; teste de mesa; ferramentas do Sandbox (com a calibração); e missões. No pontual, ela tem a arena e as configurações, o código e o teste de mesa e, no modelo de visão, a arena 3D e o jogo. O botão da parte que está na tela fica aceso e acompanha a rolagem; em telas mais estreitas, a barra mostra só os ícones, com o nome ao passar o mouse, e no celular ela vai para o pé da tela. No modelo de visão do pontual, a arena vista de cima também passou a mostrar o campo de treino (as tags 11 a 14 nas paredes e o sinal) quando ele é o escolhido, como no completo.

Figura 56 – A barra de navegação à direita, com a parte "Vista 3D, arena e robô" acesa



Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

Por fim, o editor de código passou a funcionar melhor com o código rodando. A investigação mostrou duas causas. A primeira: o realce de cores do Android Studio procurava trechos de código novos na página inteira a cada mudança da página — e, com o código rodando, a telemetria, o Driver Hub e a arena mudam a cada quadro, o que fazia a página inteira ser varrida cerca de 54 vezes por segundo, competindo com a digitação. Agora a varredura só acontece quando entra na página algo que pode ter código, e ela caiu para zero com o robô rodando. A segunda: a telemetria e os painéis mudam de altura, e tudo o que está embaixo deles — inclusive o editor — subia e descia enquanto o estudante digitava. A telemetria do Driver Hub passou a ter altura fixa, e, enquanto o editor tem o foco, a página rola junto para ele ficar parado na tela. Além disso, na primeira tecla digitada com um OpMode rodando, um aviso explica que o que se digita só vale depois de parar e dar ▶️ Build, e oferece um botão para parar o OpMode; no pontual, o aviso lembra que o código só muda com "Aplicar Código".

17.9 VALIDAÇÃO

Os testes automatizados da versão 26 conferiram:

- a partida: em 6 partidas de 4 robôs, o relógio, a troca do autônomo para o controlado, os planos sorteados, os arremessos (80,2% de acerto), as viradas e o PARK (Tabela 32); em 3000 sorteios, a distribuição dos planos de autônomo;
- a regra de virada da HIVE em nove combinações de POLLEN e NECTAR, e a capacidade de 4 peças (a quarta entra, a quinta é empurrada);

- a bateria: a tensão do VoltageSensor em 12,81 V sem a bateria real, 12,03 V com 25% e 13,54 V depois de recarregar, e o volante com setPower e setVelocity em três cargas;
- a calibração: o ajuste e os arremessos com a velocidade dada pela função;
- o ColorBlobLocatorProcessor: a C270 a 0° vê o POLLEN a 20 pol. como uma mancha amarela, e o exemplo conta a peça pega;
- a tela: nenhuma opção repetida (o campo, a câmera e a inclinação aparecem uma vez cada), o campo de treino na arena do pontual de visão, e o editor com o código rodando — o texto digitado chega inteiro, o robô não se mexe com as letras digitadas e o realce de cores não varre mais a página a cada quadro;
- e, de novo, os testes das versões anteriores, com as mudanças esperadas (a capacidade de 4 peças e os controles que mudaram de lugar).

18. LIMITAÇÕES DO MODELO

Conhecer os limites do modelo é necessário para interpretar corretamente os resultados acima:

- **Cinemática sem dinâmica:** os chassis são modelados por cinemática; não há massa, forças, transferência de peso ou curva de torque dos motores. A inércia é representada apenas por um limite de variação da potência por ciclo.
- **Deslizamento simplificado:** a perda do Mecanum é um valor fixo de 10%, independente da velocidade e do piso, e o esterçamento do swerve tem velocidade constante.
- **Dead wheels ideais:** as rodas de medição nunca deslizam nem perdem contato com o piso, o que pode ocorrer no robô real durante colisões ou acelerações bruscas.
- **Integração discreta:** o simulador move o robô usando a mesma discretização que o algoritmo esperado, por isso a solução completa atinge drift nulo. No robô real, com ciclos mais lentos e trajetórias curvas, há erro de integração; o Game Manual 0 descreve o uso de pose exponentials para reduzi-lo.
- **Ruído simplificado:** o ruído é aleatório e uniforme, sem calibração com dados reais de sensores.
- **Pinpoint simplificado no modo pontual:** o objeto pinpoint do desenvolvimento pontual não reproduz a comunicação, as velocidades nem as funções de configuração do driver real (no desenvolvimento completo, o Pinpoint V2 segue o driver do SDK; Seção 14.2).

- **Gamepad:** a leitura foi verificada com controles simulados. Controles pouco comuns podem não usar o mapeamento padrão do navegador, e o suporte no Safari é menos consistente que no Chrome e no Edge.
- **Ambiente:** no campo de treino, há colisão apenas com as bordas da arena e não há elementos de jogo (os limites do campo BIOBUZZ estão no item próprio, abaixo). O editor Monaco depende de internet; sem conexão, o simulador usa um editor simples, com as cores do Android Studio, mas sem os recursos do Monaco, como a ajuda ao passar o mouse.
- **Progresso local:** o XP, as lições e os passos ficam salvos apenas no navegador; limpar os dados do navegador apaga o progresso, por isso a tela inicial oferece exportação e importação.
- **Desafios automáticos:** conferem a resposta final, não o raciocínio. As explorações usam modelos simplificados com parâmetros ilustrativos (por exemplo, o elevador do PID é uma massa com atrito e gravidade constante), e as práticas com o robô real, previstas no plano de cada encontro, continuam necessárias.
- **Idiomas do conteúdo:** o Mapa de Estudos e o Caminho do André estão em português e em inglês; nos outros idiomas da interface, o conteúdo aparece em inglês.
- **Seguidor GVF simplificado:** o código do Caminho do André usa só o campo interno ξ_1 do artigo (sem a mistura com ξ_2 longe do caminho), um controle de heading apenas proporcional, a velocidade estimada pela diferença entre poses e constantes em unidades por ciclo. Ele serve para entender o algoritmo e não substitui a biblioteca nem o seu tuning no robô real.
- **API do Pedro Pathing:** os códigos "no robô real" seguem a documentação das versões 2.x e 3.x da biblioteca; nomes de classes e métodos podem mudar entre versões.
- **Vibração do controle:** depende do navegador e do controle; `runRumbleEffect` aceita apenas efeitos já montados como lista de passos, e o construtor `Gamepad.RumbleEffect.Builder` do SDK não está disponível no Sandbox.
- **Sensores de localização:** o Pinpoint V2 segue a API do driver do SDK (os recursos novos da V2, como pitch, roll e verificação CRC, não são simulados) e integra a cada ciclo da física, não a 1500 Hz. A Limelight tem só o pipeline de AprilTags, com o mapa das tags na posição do começo da partida (no BIOBUZZ, quando uma HIVE vira, as tags dela saem do lugar do mapa); pipelines de cor, Python e redes neurais não são simulados, e as convenções

de "target space" seguem a documentação da Limelight de forma aproximada. As câmeras não sofrem desfoque de movimento.

- **Robô Completo:** o compilador aceita o subconjunto de Java usado em OpModes de competição. Não há classes anônimas, lambdas, classes genéricas próprias nem records; a divisão entre inteiros não é emulada. Os dispositivos simulados são motores, servos, a IMU e a webcam com AprilTags, o Pinpoint, pods de odometria, a Limelight 3A, servos contínuos e entradas analógicas (usados no swerve e, o servo contínuo, no rolo do braço do robô BIOBUZZ); sensores de cor, de distância e de toque ainda não existem nessa tela.
- **Driver Hub:** reproduz o fluxo do aplicativo (configuração, listas de OpModes, INIT, START, STOP, timer, Restart Robot), mas não o pareamento dos controles (Start + A), as várias configurações salvas nem as telas de atualização de firmware.
- **Campo BIOBUZZ e física:** a forma da HIVE segue o modelo 3D do autor e as medidas do manual, mas o comprimento e a inclinação da barra, a força da trava e as massas das peças são estimados, e a leitura das cotas da Figura 9-15 é uma interpretação; os lados das FLOWERs, das GARDENs e das LOADING ZONEs foram tirados da figura das posições iniciais (Figura 11-1). A física é deliberadamente simples: as peças são esferas perfeitas, sem giro nem arrasto do ar; dentro da CELL de cima elas ocupam vagas fixas e não rolam; o robô é um retângulo que empurra as peças e só colide com as pernas da estrutura, as FLOWERs e as paredes; e a contagem de pontos e as faltas não são simuladas (o relógio da partida, as viradas das HIVEs e o PARK são só indicados). As peças batem nos tubos da estrutura como em cápsulas, e as colisões são conferidas a cada 1/120 s: um toque de raspão pode passar sem efeito, e uma peça que sai do campo fica parada onde cair. Dentro do lançador, a peça segue um caminho fixo (o arco do capô ou a reta entre as rodas) com aceleração uniforme: a compressão, o giro da peça e o escorregamento estão resumidos no coeficiente η, que foi estimado, e não medido num lançador real. O capô móvel é um trecho curvo que gira em volta do eixo do volante, e não um capô flexível; nos lançadores de duas rodas, a peça sai com a média das duas velocidades, sem o efeito (giro) que rodas a velocidades diferentes dariam a ela.
- **Partida com robôs, bateria e cor:** os outros robôs seguem regras simples (uma máquina de estados, o A* numa grade e um erro de arremesso sorteado), não se defendem nem cometem faltas, e não aparecem para as AprilTags nem para o processador de cor; os pontos não são contados. A bateria é um modelo

linear (tensão em aberto pela carga e uma resistência interna), sem a queda de tensão por temperatura nem os picos de corrente reais; a velocidade dos motores é proporcional à tensão. O processador de manchas de cor trata cada peça como um círculo perfeito, sem sombra, reflexo nem variação de luz, e a calibração usa os η e as massas estimados das peças.

- **Vista 3D e câmeras:** o desenho é feito por software, sem texturas além das tags, sem sombras e com uma transparência simples (o vidro das CELLs). As câmeras não têm distorção de lente, desfoque, variação de luz nem latência, e a detecção segue regras geométricas (centro na imagem, ângulo de até 72°, alcance, tamanho mínimo e oclusão pelas peças) em vez de um detector de AprilTags sobre a imagem.
- **Desafios de código:** o compilador aceita o subconjunto de Java usado nas lições. Um índice fora do vetor não gera ArrayIndexOutOfBoundsException (o valor lido fica vazio), e o System.out.println de um número decimal inteiro mostra 3 em vez de 3.0, porque o tipo da variável não chega até a execução; a divisão entre inteiros (`7 / 2` resultar em 3) não é reproduzida, por isso os desafios usam `Math.floor` ou `(int)` quando precisam dela. Os testes conferem o resultado do método, não o estilo do código, e os enunciados estão em português e em inglês.

19. CONCLUSÃO E DIRETRIZES DE VALIDAÇÃO NO HARDWARE REAL

A validação das trajetórias e da programação do chassi no simulador encerra a fase conceitual e prepara a migração para o robô de competição da equipe SESI Bug Hunters. Como essa lógica começou a ser consolidada no off-season e não depende das regras de um jogo, a base matemática do robô pode ser reutilizada em qualquer temporada; as partes específicas do BIOBUZZ servem para treinar a temporada em curso. Para a transição ao hardware, estabelecem-se as seguintes diretrizes:

- **Treinar com o chassi da equipe:** selecionar o modo de movimento igual ao do robô real e praticar o TeleOp com o mesmo controle usado na Driver Station, usando os campos de `gamepad1` e `gamepad2` que serão usados no código final.
- **Usar o modo FTC como referência:** as equações e convenções desse modo são as mesmas da IMU e do Road Runner, o que evita inversões de sinal na migração.
- **Preferir dead wheels aos encoders de tração:** a Seção 8.2 mostrou que os encoders de tração acumulam o deslizamento das rodas, principalmente no Mecanum.

- **Migrar do controle proporcional para PID:** o exemplo do Go To Target usa apenas o termo proporcional. No robô real, um controlador PID, ou ao menos um controle proporcional com potência mínima, é necessário para lidar com a inércia e o atrito estático perto do alvo.
- **Calibrar as constantes físicas:** medir os offsets reais das rodas (girando o robô no lugar várias vezes e ajustando até a posição calculada permanecer constante), o diâmetro efetivo das rodas (movendo o robô por uma distância medida com trena) e o sentido de contagem de cada encoder.
- **Seguir o Mapa de Estudos nos encontros da equipe:** cada lição corresponde a um encontro de três horas do plano; o Sandbox e as explorações substituem o robô quando ele não está disponível, e as revisões mostram quem precisa refazer qual encontro.
- **Treinar o ciclo do jogo antes de construir o robô:** usar o robô de poucos polígonos para testar, no código, o caminho inteiro de uma peça — pegar na FLOWER com o braço, guardar com o intake, mirar com a torreta e acertar a velocidade do volante pela parábola — e decidir, com números, entre uma torreta e um lançador fixo na frente ou atrás, e entre capô fixo, capô móvel, dois volantes ou um lançador em linha, antes de cortar a primeira peça.
- **Treinar a partida inteira antes do primeiro campeonato:** jogar com os outros robôs para treinar a rotina de autônomo e controlado, o PARK no fim e a convivência com outros robôs no campo; usar a calibração do lançador para chegar à mesa de testes com uma função inicial de velocidade por distância; e deixar a bateria real ligada nos treinos de controle de velocidade, para que a equipe veja, com números, por que o PIDF do volante compensa a bateria fraca e o setPower não.
- **Treinar a visão do jogo antes de ter o campo:** usar o modelo pontual de visão e o campo BIOBUZZ para decidir a altura e a inclinação da câmera e testar a fusão de odometria e AprilTags antes de montar o suporte da câmera no robô real.
- **Comparar os sensores antes de comprar ou montar:** a Montagem do robô permite testar, com o mesmo autônomo, a diferença entre usar só os encoders dos motores, o Pinpoint ou três pods, e entre a C270 e a Limelight, antes de decidir o que instalar no robô real.
- **Treinar a rotina de competição no Robô Completo:** antes de mexer no robô real, cada novo programador deve conseguir montar do zero a configuração, o TeleOp e o autônomo com visão e completar as dez missões; os erros mais comuns de competição (nome da configuração diferente do código, motor

invertido, telemetria sem update, laço que não respeita o STOP) aparecem ali sem risco para o robô.

- **Programar antes de avançar:** usar os desafios de código como tarefa de cada encontro da equipe; quando um estudante travar, a tabela de testes mostra exatamente em qual caso o método falha, o que orienta a conversa com o mentor.
- **Construir o seguidor antes de ajustar a biblioteca:** percorrer o Caminho do André até o passo P11 antes do tuning do Pedro Pathing no robô real, para que cada constante ajustada tenha um significado conhecido pela equipe.

Como próximos registros no Caderno de Engenharia, a equipe documentará o uso do simulador pelos demais integrantes, os testes com o controle físico e a validação no robô real, comparando as posições calculadas pela odometria com posições medidas no campo.

20. HISTÓRICO DE DESENVOLVIMENTO (CHANGELOG)

O ciclo evolutivo do FTC Odometry Sandbox está registrado na Tabela 37, do motor gráfico inicial até a versão atual.

Tabela 37 – Matriz de versionamento técnico (fase Alpha)

Versão	Denominação / Marco	Descrição das implementações e impacto pedagógico
v1.0	O Nascimento	Criação da arena de renderização, física básica de colisão com as bordas e movimentação manual do chassi pelo teclado.
v2.0	Interatividade	Primeiro módulo de edição de código em painel lateral (IDE), permitindo alterar a saída de dados sem comprometer o motor principal da aplicação.
v3.0	Alinhamento Matemático	Correção dos eixos cartesianos virtuais e sincronização da telemetria com o campo da FTC, padronizado em 1000 unidades para fins didáticos.
v4.0	Lógica de Fluxo	Variáveis de memória e histórico (prevEncoder), que permitem calcular a variação das leituras entre ciclos consecutivos. Marco que viabilizou as equações de odometria.
v5.0	Estabilidade e Segurança	Botão "Aplicar Código" e tratamento de erros em tempo de execução, tornando o ambiente tolerante a erros de sintaxe de estudantes iniciantes.
v6.0	Ferramentas de Equipe	Salvar e abrir arquivos de código, permitindo que os estudantes guardem e compartilhem o próprio progresso.
v7.0	O Motor Autônomo	Comandos autônomos e funções acionadas por teclas (por exemplo, a tecla "R"), tornando o robô programável além da localização passiva.
v8.0	Integração Multilíngue	Suporte a 6 idiomas e rotina "Smart Swap", que preserva o código do estudante ao trocar de idioma.

Versão	Denominação / Marco	Descrição das implementações e impacto pedagógico
v9.0	Rastro Visual	Rastros da posição real e da odometria calculada em cores diferentes, para identificar onde as coordenadas começam a divergir.
v10.0	Modo de Física Realista	Botão para introduzir desvios holonômicos, deslizamento lateral e ruído de leitura, exigindo correção e calibração.
v11.0	Suporte a Java	Transpilador interno de sintaxe Java para JavaScript, aproximando a escrita do código ao ambiente usado no Control Hub.
v12.0	Monaco Editor	Substituição da caixa de texto pelo editor do VS Code (Monaco), com destaque de sintaxe Java e tema escuro personalizado.
v13.0	Console de Telemetria	Console simulado com os comandos telemetry.addData e telemetry.update, sincronizado ao ciclo da simulação.
v14.0	Diagnóstico e Personalização	Fantasmas da pose calculada e bruta, gráfico de drift, temas claro, escuro e OLED, controles de offset X/Y e de ruído (substituindo o botão do v10), modo de 3 dead wheels, 14 idiomas com suporte a escrita da direita para a esquerda e indicação da linha de erro no editor.
v15.0	Consistência e Realismo	Correção do teclado que movia o robô durante a digitação no editor; IMU separada do gabarito, com ruído próprio e opção desligada; Calc Heading e erro de heading; modo de 3 rodas completo; modo de coordenadas FTC; leituras em ticks; Pinpoint simulado; reimplementação dos rastros; transpilador com conversões de tipo e parênteses aninhados; editor alternativo sem internet; revisão do memorial (fórmula do arco e matriz de rotação).
v16.0	Chassis e Navegação	Interface reorganizada (barra de ações acima da arena, seletor de movimento com painel de motores, abas e atalhos de navegação); quatro modos de movimento (Ideal, Tank, Mecanum e Swerve) com cinemática inversa e direta, inércia dos motores e esterçamento limitado; objeto motors, encoders de tração e modelos de TeleOp para cada chassi.
v17.0	Gamepad	Leitura de controles reais pela Gamepad API (gamepad1 e gamepad2, com os nomes de campo do FTC SDK e zona morta), indicador de conexão com a posição dos sticks, variável prevDriveEncoders e atualização deste memorial para os novos modos.
v18.0	Tela Inicial	O simulador passa a fazer parte de um aplicativo com telas e navegação no cabeçalho (endereços #/tela/item, botão Continuar), pausa da simulação fora da tela do Sandbox e sistema de progresso com XP, níveis, medalhas, exportação e importação.
v19.0	Mapa de Estudos	Os 86 encontros do plano de estudo como lições interativas em português e inglês, com 50 explorações, 199 desafios de cinco tipos, revisões e checkpoints montados a partir das lições, regras de liberação, medalhas de fase e botão "Abrir no Sandbox"; `String.equals` no transpilador.

Versão	Denominação / Marco	Descrição das implementações e impacto pedagógico
v20.0	Caminho do André	Guia de 27 passos (TeleOp, autônomo e Pedro Pathing do zero) com 74 termos de vocabulário verificados, códigos testados no Sandbox e equivalentes para o robô real; seguidor GVF de segunda ordem construído termo a termo; API de desenho na arena; `new double[n]` e `Math.signum` no transpilador.
v21.0	Desafios de código	Toda lição, revisão e passo termina com um método em Java a ser escrito e testado (101 desafios, 452 casos de teste), executado num Web Worker com limite de tempo; código inicial do swerve refeito (giro, strafe no modo FTC e compensação do cosseno); `.length()` no transpilador.
v21.1	Botões do controle	Atalhos do controle para as funções do Sandbox (iniciar/pausar, resetar, aplicar código, fantasma, eixos, rastros, modo de movimento), com aviso de conflito com o código; métodos de borda do FTC SDK (`aWasPressed()` e demais), nomes do PlayStation e vibração (`rumble`, `rumbleBlips`).
v22.0	Robô Completo	Nova tela em que o estudante escreve os arquivos Java inteiros (package, imports, anotações, OpMode, hardwareMap, visão e autônomo), compilados por um compilador próprio que confere as regras do Java e executa o LinearOpMode com geradores; FTC SDK simulado (DcMotor com encoders e RUN_TO_POSITION, Servo, IMU com orientação, telemetria, VisionPortal e AprilTagProcessor); Driver Hub com Configure Robot, listas de OpModes, INIT, START, STOP, timer de 30 s, preselectTeleOp e Restart Robot; oito missões e a medalha Robô completo.
v23.0	Sensores de localização	Montagem do robô escolhida na tela; goBILDA Pinpoint V2 com o driver do SDK (offsets, resolução, direções, calibração, posição mantida entre OpModes); três pods de odometria nas portas de encoder do Expansion Hub; Limelight 3A com AprilTags, tx/ty/ta, MegaTag1 e MegaTag2, imagem no Driver Hub e campo de visão na arena; Configure Robot com I2C Bus 1, Expansion Hub e dispositivos USB; duas missões novas; o Robô Completo vira o modo "desenvolvimento completo" do Sandbox, com as ferramentas do Sandbox (pausa, rastro, eixos, fantasmas do Pinpoint e da Limelight, controles de chassi e ruído); mensagens de erro mais claras para método declarado dentro de outro; editor com sugestões de todas as classes, campos e métodos simulados do SDK, import automático, nomes da configuração e ajuda de parâmetros; menu Projeto para salvar e abrir projetos (.zip), importar arquivos .java ou .zip (também arrastando) e baixar um arquivo; renomear abas com duplo clique ou dois toques; chassi swerve no modo completo (4 motores, 4 servos de esterçamento como CRServo ou Servo e 4 encoders absolutos como AnalogInput); barra de ferramentas alinhada com a arena nos dois modos.

Versão	Denominação / Marco	Descrição das implementações e impacto pedagógico
v24.0	Editor e console dos desafios de código	Editor dos desafios das lições com números de linha, indentação automática, comentários com Ctrl+/ e sugestões do Java da linguagem; botão Executar com caixa de chamada e console (System.out.println, valor devolvido, exceções com a linha); coluna de saída e botão ▶️ em cada teste; correção pelo compilador de Java do modo completo, com mensagens para os erros mais comuns de quem está começando; teste de mesa nos desafios das lições e depurador passo a passo nos dois modos do Sandbox, com uma página por ciclo do laço; cores do Android Studio (Darcula e IntelliJ Light) em todos os editores e trechos de código; menu de opções na barra de navegação (tema, idioma, depurador, sugestões, funções, ajuda e painel do teclado); organização do código em várias classes no modo completo (herança, classes abstratas, interfaces, sobrecarga, import static, pastas e classes internas); cartão com as funções de cada desafio de código.

Versão	Denominação / Marco	Descrição das implementações e impacto pedagógico
v25.0	Visão 3D e campo BIOBUZZ jogável	Campo BIOBUZZ 2026-2027 com as HIVEs (CELLs em forma de casinha, transparentes, com a armação na cor da aliança, ligadas por uma barra reta que passa pelo pivô), as 16 AprilTags 36h11 de 3,25 pol. (IDs 30 a 45) com o padrão de bits real, as FLOWERs com anéis vazados, as GARDENs e as LOADING ZONEs (PARK); vista 3D desenhada por software com a reta de Bresenham e o rotatePoint do autor, z-buffer, recorte e vidro, com a arena em órbita e a primeira pessoa da C270 e da Limelight; robô real e estimado (myState, Pinpoint, MegaTag, robotPose e pose da telemetria); câmeras em seis graus de liberdade e escolha entre C270, Limelight ou as duas; detecção em 3D com oclusão; getCurrentGameTagLibrary com nomes e posições e robotPose calculado com o setCameraPose declarado; Camera Stream e Limelight do Driver Hub com a imagem 3D; POLLEN e NECTAR como icosaedros no tamanho oficial; robô com intake, midtake com subida vertical pelo furo da torreta, lançador em cinco tipos — capô fixo, capô móvel com servo (40° a 75°), dois volantes com dois motores, e em linha com dois discos (dois motores, fixo atrás) ou uma roda em cima — numa torreta de 270°, fixo na frente ou, os em linha, fixo atrás seguindo a rampa, com o caminho da peça por dentro e laterais transparentes, e braço com rolo, ligados ao Expansion Hub ou ao teclado; física das peças, do lançamento em parábola, com os arcos previstos do POLLEN e do NECTAR, que se ligam separadamente, das CELLs e do tombamento das HIVEs (com o despejo da CELL que desce); colisão do robô com a estrutura e as FLOWERs; peças que batem em todos os tubos das HIVEs e passam por cima das paredes; arcos previstos que param no primeiro obstáculo; exemplos que ligam a C270 (VisionPortal com setCameraPose) e a Limelight; posições iniciais das regras e de treino; um TeleOp de exemplo para cada um dos 10 robôs possíveis, carregado de acordo com o robô escolhido; câmera em terceira pessoa que segue o robô, controlada pelo teclado ou pelo controle; lançamento com a velocidade do ponto de saída (andar, girar e torreta) e integração exata; modelo "Visão" no desenvolvimento pontual com odometria pronta (com deriva) ou escrita pelo estudante; dicas em escada nos desafios de código (ideia, plano, esqueleto e peças).

Versão	Denominação / Marco	Descrição das implementações e impacto pedagógico
v26.0 [Atual]	Partida com robôs, bateria e calibração	Partida no desenvolvimento completo com 2 ou 4 robôs (aliança vermelha ou azul), seguindo o OpMode do estudante (autônomo de 30 s e controlado de 2 min, com fim de jogo); outros robôs com autônomo sorteado (50% PARK, 35% virar a HIVE com os 4 POLLEN do robô, 15% nada) e controlado próprio (pegar peças no chão e nas FLOWERs, atirar de pontos calculados pela física com erros sorteados, estacionar no fim), navegação por A* e colisões com o robô do estudante; placar na arena; HIVE vira pela contagem P/8 + N/5 ≥ 1; até 4 peças no robô; modelos 3D dos chassis tank e swerve e roletes nas rodas mecanum; bateria real com descarga, queda de tensão, motores mais fracos e recarga, e o VoltageSensor do SDK; calibração do lançador (alcance por potência e ângulo, ajuste por mínimos quadrados e função em Java) e registro dos arremessos; ColorBlobLocatorProcessor na C270, com o exemplo que conta as peças pegas quando a câmera está a 0°; cada opção num lugar só, configuração em colunas embaixo da vista 3D, bateria embaixo do Driver Hub e barra de navegação à direita; campo de treino na arena do pontual de visão; editor estável com o código rodando, com aviso.

Fonte: Elaborada por Flor e equipe SESI Bug Hunters (2026).

REFERÊNCIAS

APRILROBOTICS. **AprilTag 3**: biblioteca de detecção de AprilTags (tag36h11.c). Disponível em: https://github.com/AprilRobotics/apriltag. Acesso em: 16 out. 2026.

BRESENHAM, J. E. Algorithm for computer control of a digital plotter. **IBM Systems Journal**, v. 4, n. 1, p. 25-30, 1965.

FIRST TECH CHALLENGE. **Class Gamepad**: RobotCore 12.0.0 API. Disponível em: https://javadoc.io/doc/org.firstinspires.ftc/RobotCore/latest/com/qualcomm/robotcore/hardware/Gamepad.html. Acesso em: 12 out. 2026.

FIRST TECH CHALLENGE. **FTC Game Manual – DECODE Season: Traditional Events**. Manchester: FIRST, 2026. (Documento consultado para fins de contextualização do período cronológico de desenvolvimento).

FIRST. **2026-2027 FIRST Tech Challenge Competition Manual**: BIOBUZZ, seções 10 e 11 (pontuação, PARK, regra G304 e Figura 11-1). Disponível em: https://ftc-resources.firstinspires.org/ftc/archive/2027/game/manual-10. Acesso em: 16 out. 2026.

FIRST. **2026-2027 FIRST Tech Challenge Competition Manual**: BIOBUZZ, seções 9.6 a 9.8 (HIVE, FLOWER e elementos de pontuação). Disponível em: https://ftc-resources.firstinspires.org/ftc/archive/2027/game/manual-09. Acesso em: 16 out. 2026.

FIRST. **2026-2027 FIRST Tech Challenge Competition Manual**: BIOBUZZ. Versão TU04, seção 9 (Arena). Disponível em: https://ftc-resources.firstinspires.org/ftc/game/manual. Acesso em: 16 out. 2026.

FIRST. **2026-2027 FIRST Tech Challenge Event Field Setup Guide**. Disponível em: https://ftc-resources.firstinspires.org/ftc/archive/2027/field/eventfieldguide. Acesso em: 16 out. 2026.

FIRST. **FTC Docs**: AprilTag Localization (setCameraPose). Disponível em: https://ftc-docs.firstinspires.org/en/latest/apriltag/vision_portal/apriltag_localization/apriltag-localization.html. Acesso em: 16 out. 2026.

FIRST. **FTC Docs**: Color Processing — ColorBlobLocatorProcessor. Disponível em: https://ftc-docs.firstinspires.org/en/latest/color_processing/index.html. Acesso em: 17 out. 2026.

FIRST. **FTC Docs**: documentação do FIRST Tech Challenge (VisionPortal, AprilTag, IMU, Driver Station). Disponível em: https://ftc-docs.firstinspires.org/. Acesso em: 12 out. 2026.

FIRST. **RobotCore 10.3.0 API**: documentação Javadoc do FTC SDK. Disponível em: https://javadoc.io/doc/org.firstinspires.ftc/RobotCore/. Acesso em: 12 out. 2026.

GAME MANUAL 0. **Mecanum TeleOp**. Disponível em: https://gm0.org/en/latest/docs/software/tutorials/mecanum-drive.html. Acesso em: 12 out. 2026.

GAME MANUAL 0. **Odometry**. Disponível em: https://gm0.org/en/latest/docs/software/concepts/odometry.html. Acesso em: 12 out. 2026.

GOBILDA. **goBILDA Pinpoint Odometry Computer User Guide**: Hardware Overview and API Integration. goBILDA, 2025. Documento técnico de referência.

GOBILDA. **Pinpoint V2 Odometry Computer (IMU Sensor Fusion for 2 Wheel Odometry)**. Disponível em: https://www.gobilda.com/pinpoint-v2-odometry-computer-imu-sensor-fusion-for-2-wheel-odometry/. Acesso em: 12 out. 2026.

HART, P. E.; NILSSON, N. J.; RAPHAEL, B. A formal basis for the heuristic determination of minimum cost paths. **IEEE Transactions on Systems Science and Cybernetics**, v. 4, n. 2, p. 100-107, 1968.

LIMELIGHT VISION. **FTC Programming Guide**. Disponível em: https://docs.limelightvision.io/docs/docs-limelight/apis/ftc-programming. Acesso em: 12 out. 2026.

MOZILLA. **Using the Gamepad API**. MDN Web Docs. Disponível em: https://developer.mozilla.org/en-US/docs/Web/API/Gamepad_API/Using_the_Gamepad_API. Acesso em: 12 out. 2026.

PEDRO PATHING. **Pedro Pathing Docs**. Disponível em: https://pedropathing.com/. Acesso em: 12 out. 2026.

ROAD RUNNER. **Tuning**. Road Runner Docs. Disponível em: https://rr.brott.dev/docs/v1-0/tuning/. Acesso em: 12 out. 2026.

SESI BUG HUNTERS. **Plano de estudo do programador novo de FTC**: 60 encontros. São Bernardo do Campo, 2026. Documento interno da equipe.

SESI BUG HUNTERS. **Plano de estudo do programador novo de FTC**: nível 4, encontros 61–86. São Bernardo do Campo, 2026. Documento interno da equipe.

SRIPADA, Havish; HENDERSON, Baron. **Second-Order Multidimensional Guiding Vector Field for Parametric Path-Following Under Holonomic Constraints**. Pedro Pathing, 2026. Artigo técnico.

THE ALLSPARKS. **ViDAR**: transcrição do campo BIOBUZZ (config/seasons/2026-biobuzz.json). Disponível em: https://github.com/The-Allsparks/ViDAR. Acesso em: 16 out. 2026.

WPILIB. **Differential Drive Kinematics**. FIRST Robotics Competition Documentation. Disponível em: https://docs.wpilib.org/en/stable/docs/software/kinematics-and-odometry/differential-drive-kinematics.html. Acesso em: 12 out. 2026.

WPILIB. **Swerve Drive Kinematics**. FIRST Robotics Competition Documentation. Disponível em: https://docs.wpilib.org/en/stable/docs/software/kinematics-and-odometry/swerve-drive-kinematics.html. Acesso em: 12 out. 2026.